

VOORWOORD

Deze cursusnota's geven de inhoud weer van het opleidingsonderdeel Uitbatingssystemen. In het bijzonder wordt er ingegaan op de onderliggende principes die gehanteerd worden door hedendaagse besturingssystemen. Zoals in de meeste takken van de informatica is het ook hier wenselijk een deel van de Engelse terminologie over te nemen. Wanneer de Nederlandse vertaling erg sterk lijkt op het Engelse begrip, dan zal de Nederlandse term veelal gebruikt worden, bv. proces, segmentatie, etc. Toch worden deze Nederlandse woorden in bepaalde samenstellingen als twee woorden geschreven, hoewel dit in wezen niet helemaal correct is, bv. proces tabel. Tenslotte dank ik bij deze ook Gino Peeters, Robbe Block en Wouter Minnebo, die reeds vele kleine correcties en suggesties deden die de inhoud van de cursus enkel ten goede kwamen.

Benny Van Houdt
December 2014

Inhoudsopgave

PROCESSEN EN THREADS	5
1 Processen	7
1.1 Proces attributen	7
1.2 Proces toestanden	10
1.3 Kernel, state en mode switching	11
2 Threads	14
2.1 User-level en kernel-level threads	16
CACHE GEHEUGEN	18
1 Computergeheugen: een overzicht	18
2 Cache geheugenorganisatie	19
2.1 Direct mapping	22
2.2 Associative en set-associative mapping	24
2.3 Replacement algoritmen	27
3 Geheugen updates: write policy	29
4 Aantal caches in hedendaagse systemen	29
5 Cache benchmarking	31
GEHEUGEN MANAGEMENT	34
1 Partitionering	37
1.1 Fixed partitioning	37
1.2 Dynamic partitioning	38
1.3 Buddy systeem	43
2 Paging	44
3 Segmentation	46
VIRTUEEL GEHEUGEN	50
1 Virtuele geheugenlayout	51
2 Page table organisatie	53
2.1 Multilevel page tables	54
2.2 Inverted page tables	55
2.3 Translation lookaside buffer	57
3 Virtueel geheugen: software support	59
3.1 Page replacement algoritmen	60
3.2 Resident set management	64
4 Optimaliteit van het MIN algoritme: een bewijs	66
UNIPROCESSOR SCHEDULING	69
1 Non-real-time short-term scheduling	71
1.1 Performantie maten	71

1.2	Notatie van scheduling modellen	72
1.3	Scheduling algoritmen	72
1.4	Traditionele UNIX scheduler	81
1.5	Fair-share scheduler	83
2	Real-time short-term scheduling	84
2.1	Deadline scheduling	85
2.2	Rate monotonic scheduling (RMS)	90
	DISK MANAGEMENT	92
1	Magnetische Disks	92
2	Disk Scheduling	96
3	RAID: redundant arrays of independent discs	99
3.1	RAID 0	100
3.2	RAID 1	101
3.3	RAID 3 en 4	102
3.4	RAID 5 en 6	104
3.5	RAID $X + Y$	104
	CONCURRENCY	108
1	Mutual exclusion, deadlocks en starvation	108
2	Softwarematige oplossingen	110
2.1	Algoritme van Dekker	110
2.2	Algoritme van Peterson	114
3	Hardwarematige oplossingen	115
4	Semaforen	117
4.1	Mutual exclusion met semaforen	118
4.2	Het producer/consumer probleem met semaforen	119
4.3	Semaforen implementeren	121
5	Het readers/writers probleem	122
	FILE SYSTEMEN	125
1	File attributen, operaties en types	125
2	De directory structuur	130
2.1	Boomgestructureerde directories	130
2.2	Graafgestructureerde directories	131
2.3	Directory structuur implementatie	133
3	Access en allocatie methodes	134
3.1	Allocatie methodes	134
3.2	UNIX inodes	139
3.3	Free-space management	141
4	Fast file systeem voor UNIX	143

4.1	Het traditionele UNIX file systeem	143
4.2	Het Fast file systeem	144
5	Journaling file systemen	149
MULTIPROCESSOR ISSUES		152
1	Symmetrische multiprocessor (SMP) organisatie	152
2	Cache coherency: het MESI protocol	154
2.1	Read miss en hit	155
2.2	Write miss en hit	156
2.3	L1-L2 cache consistency	158
3	Multiprocessor scheduling	158
3.1	De maximale completion time	158
3.2	SPN en EDF scheduling	163

PROCESSEN EN THREADS

In dit hoofdstuk worden enkele fundamentele concepten van operating systemen geïntroduceerd. Aangezien dit het inleidende hoofdstuk vormt van de cursus, zullen we op tal van plaatsen vrij vage bewoordingen hanteren. In latere hoofdstukken zullen vele van deze beschrijvingen verder geconcretiseerd worden.

Computerarchitectuur in een notendop

Een computersysteem bestaat logisch gezien uit vier componenten: de processor, het geheugen, de input/output (I/O) devices en een verbindingselement dat toelaat informatie uit te wisselen tussen de eerste drie componenten. De processor is de entiteit die de instructies effectief zal uitvoeren. Deze instructies, alsook de data die erdoor gemanipuleerd wordt, bevinden zich (veelal) in het geheugen. De I/O devices (bv. de hard disk, grafische kaart, netwerk kaart, keyboard, audio devices) laten toe informatie met het systeem uit te wisselen.

Het geheugen is een verzameling bits die opgedeeld wordt in woorden van vaste lengte (bv. 16, 32 of 64 bits). Deze woorden worden genummerd, startende vanaf nul, waarbij we verwijzen naar het woordnummer door te spreken van het adres van een geheugenwoord. Elk van deze geheugenwoorden kan data of een instructie bevatten. Wanneer het om een instructie gaat, dan geven de eerste bits, die we de *opcode* noemen, aan om welke instructie het gaat, terwijl de overige bits een geheugenadres bevatten. Bijvoorbeeld wanneer er een datawoord uit het geheugen moet gelezen worden, dan zal de opcode aangeven dat het om een leesoperatie gaat, terwijl het adres aangeeft welk woord er gelezen moet worden.

De processor heeft als doel de instructies waaruit een programma bestaat achtereenvolgens uit te voeren. Om dit te realiseren beschikt de processor onder andere over een beperkt aantal registers (die in aantal veel kleiner zijn dan het aantal geheugenwoorden). De processor moet weten welke instructie uitgevoerd moet worden, deze wordt steeds in het *instruction* register

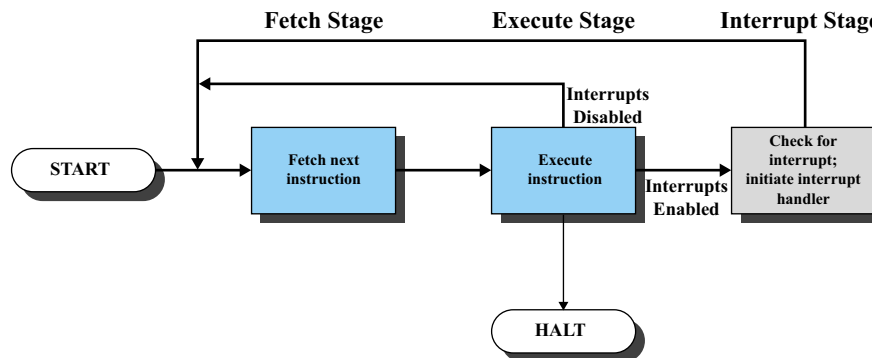


Fig. 1: Instructiecyclus met interrupts [Stallings 03]

opgeslagen. Het geheugenadres van de volgende instructie wordt bijgehouden door de *program counter*, waarvan de waarde eveneens in een register bijgehouden wordt.

Logisch gezien doorloopt de processor dus een cyclus waarbij (a) telkens een instructie wordt uitgevoerd en (b) de processor de volgende instructie uit het geheugen opvraagt. De program counter wordt bij het uitvoeren van de instructie steeds met één verhoogd. Dit betekent dat een groot deel van de instructies zich in volgorde in het geheugen bevinden. Daarnaast kunnen er ook *branch* instructies plaatsvinden die de waarde van de program counter gelijk stellen aan de meegeleverde waarde (om o.a. function calls af te handelen).

Om de capaciteit van de processor goed te benutten, moet er voor gezorgd worden dat de processor niet te lang hoeft te wachten tussen de uitvoering van twee instructies in. Echter, wanneer bij een operatie een I/O device betrokken is, dan zal het doorgaans duizenden cycli duren voordat het I/O device de vraag van de processor verwerkt heeft. Dit maakt dat het operating systeem het huidige proces tijdelijk zal stopzetten zodat een ander proces nuttig gebruik kan maken van de processor. Om aan te geven dat de I/O operatie klaar is, zal er gebruik gemaakt worden van het *interrupt* mechanisme.

Logisch gezien zal de cyclus van de processor dan bestaan uit drie fases (zie ook Figuur 1): (a) een instructie uitvoeren, (b) controleren of er een interrupt heeft plaats gevonden en deze mogelijk verwerken en (c) de volgende instructie opvragen. Dit interrupt mechanisme wordt niet enkel gebruikt voor I/O operaties, maar voor alle *system calls*, *exceptions* en *device*

interrupts. Met behulp van een system call kan een user programma aan het operating systeem vragen om een bepaalde service, zoals het openen van een file. Exceptions zijn illegale operaties, zoals delen door nul, of het opvragen van een ongeldig geheugen adres. Device interrupts worden gebruikt om aan te geven dat een bepaald hardware device aandacht nodig heeft van de processor. Dit is bijvoorbeeld het geval wanneer data is gelezen van de hard disk of netwerk kaart, maar kan net zo goed komen van de zogenaamde *clock chip* die een aantal keer per seconde een clock interrupt genereert om de uitvoering van verschillende processes met elkaar te verweven. Wanneer de interrupt afkomstig is van een user programma omwille van een system call of exception die is opgetreden spreekt men ook van een *trap*.

De code voor het afhandelen van een interrupt, dat is de *interrupt handler*, alsook de code die gebruikt wordt om te bepalen welke interrupts worden doorgestuurd naar welke processor maken deel uit van het operating systeem.

1 Processen

Ruwweg kunnen we stellen dat een proces overeenstemt met een programma in uitvoering. De taak van het operating systeem bestaat er dan ook in om meerdere processen te beheren. Zo moet het er voor zorgen dat de processorcapaciteit goed wordt benut. Het operating systeem speelt hierbij een bepalende rol aangezien het zal beslissen in welke volgorde de processen over de processor kunnen beschikken. In het onderdeel *Scheduling* keren we op deze problematiek terug. Verder moeten de processen alle beschikbare resources, en in het bijzonder het geheugen, met elkaar delen. Het operating systeem zal de beschikbare resources verdelen en moet de mogelijke conflicten die hieruit voortvloeien oplossen. In de hoofdstukken over Geheugen Management en *Concurrency* gaan we hier dieper op in.

1.1 Proces attributen

Laten we eerst even stilstaan bij de fysieke manifestatie van een proces. Ten eerste bestaat een proces uit een verzameling instructies, of nog, de code van het proces. Verder is er de data die door de code wordt gemanipuleerd. De omvang van deze data kan reeds bij de compilatie vastliggen of dynamisch wijzigen in de tijd. In het eerste geval spreken we van statische data, anders van dynamische data.

De data en code wordt bij het opstarten van een proces in het (secundaire) geheugen geladen (voor meer info verwijzen we naar het onderdeel Geheugen Management). Verder zal elk proces gebruik maken van een *user stack* en een *heap* gedeelte. De user stack wordt gebruikt voor het afhandelen van procedure calls en wordt georganiseerd in de vorm van een stackstructuur, terwijl geheugen dat wordt gealloceerd tijdens de uitvoering van een proces wordt toegevoegd aan het heap gedeelte.

Stel dat we tijdens de uitvoering van een proces een procedure aanroepen. Dan zal de program counter aangepast worden zodat deze wijst naar het begin van de procedure code. Wanneer de procedure eindigt, moet de processor terugkeren naar de locatie waarvan de call plaats vond. Om de program counter te bepalen moet het return adres dus opgeslagen worden. Hiervoor wordt een stackstructuur gebruikt. Namelijk, telkens wanneer er een procedure call optreedt, plaatsen we het return adres bovenaan de stack en wanneer een procedure eindigt dan keren we terug naar het adres bovenaan op de stack en verwijderen vervolgens dit adres. Op deze manier kunnen we geneste en recursieve procedure calls afhandelen.

Verder kan men op de stack ook de lokale variabelen (en meegegeven parameters) van de procedure plaatsen. Dit is erg handig aangezien men niet kan voorzien hoeveel procedure oproepen er precies zullen plaats vinden. De stack bevat ook een frame pointer om een variabele omvang (in aantal of grootte) van de lokale parameters toe te laten (zie Figuur 2). Return parameters worden ook via de stack doorgegeven. Het adres van de top van de stack wordt bijgehouden in één van de registers van de processor. Kortom, de stack vormt een dynamische component van een proces, die op een efficiënte manier gebruik maakt van het geheugen. Doorgaans is er wel een maximale grootte voor de user stack.

Naast de code, data, de systeem stack en het heap gedeelte moet er voor elk proces ook informatie bijgehouden worden die door het operating systeem gebruikt wordt om de processen te beheren. Deze informatie, het proces *control block*, wordt vaak opgedeeld in drie onderdelen:

1. *Process Identification*: Elk proces krijgt bij het opstarten een uniek nummer. Dit nummer wordt niet enkel gebruikt voor inter-proces communicatie, maar ook als index tot de proces tabellen van het operating systeem om snel informatie op te zoeken (zie verder). Wanneer een proces wordt gecreëerd door een ander proces (het *parent* proces), wordt ook de identifier hiervan bijgehouden. Dit is bijvoorbeeld het geval

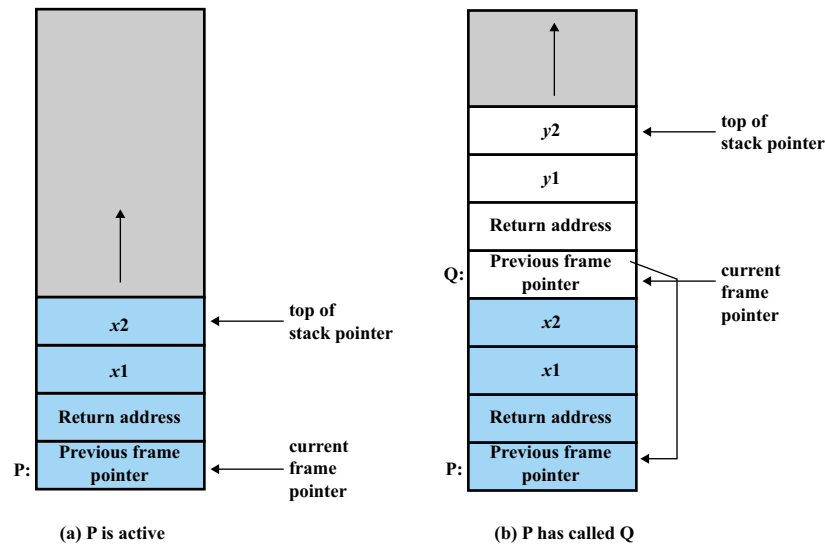


Fig. 2: Evolutie van de systeem stack van een proces [Stallings 03]

wanneer we een commando uitvoeren vanuit de *shell*. In multi-user systemen heeft elk proces ook een user identifier.

2. *Processor State Information*: Deze data weerspiegelt de inhoud van de processor registers. Wanneer het proces in uitvoering is, bevindt deze info zich in de processor zelf. Wordt het proces onderbroken door een ander proces, dan zal de inhoud van de registers hier worden weggeschreven.
3. *Process Control Information*: Alle extra informatie die het operating systeem nog behoeft voor het beheren van de processen vinden we hier terug. Hier wordt onder andere de toestand van het proces bijgehouden en de scheduling informatie (zie verder), informatie met betrekking tot het geheugen waarover het proces kan beschikken, de privileges van het proces, velden nodig om de proces tabellen op te bouwen, etc.

Het geheel van de code, data, stack en proces control block noemen we het *proces image*.

Het operating systeem zal de proces informatie die vervat zit in de control blocks van alle processen organiseren in een aantal tabellen. Zo zijn er geheugentabellen, I/O tabellen, file tabellen en een proces tabel die zo

georganiseerd is dat de proces id als index tot de tabel gebruikt kan worden. De geheugentabellen geven bijvoorbeeld aan waar en in welke vorm de proces images van elk van de processen zich bevinden in het (virtueel) geheugen, verder in de cursus bespreken we enkele mogelijke structuren. I/O tabellen geven aan welke resources in gebruik zijn en houden informatie bij met betrekking tot de datastructuren nodig voor het ondersteunen van I/O management.

1.2 Proces toestanden

Wanneer het operating systeem meerdere processen beheert, moet het bijhouden welk proces in uitvoering is, welke op een I/O event wachten en welke processen verder uitgevoerd kunnen worden (indien het proces in uitvoering de processor vrijgeeft). Hiervoor wordt gebruik gemaakt van de proces toestand (*state*).

De mogelijke toestanden verschillen van systeem tot systeem. Echter, een aantal toestanden keren, zij het onder verschillende namen, terug in de meeste systemen. Het proces dat over de processor kan beschikken bevindt zich in de toestand *Running*. Een proces in uitvoering kan om verscheidene redenen onderbroken worden, zonder dat het op een I/O event moet wachten. Bijvoorbeeld wanneer er een interrupt optreedt doordat een toets is ingedrukt op het keyboard. Verder zal een proces in de meeste systemen maar een beperkte tijd ononderbroken over de processor mogen beschikken (men hanteert hiervoor de term *time-sharing* systemen). Om dit te realiseren zal de clock chip regelmatig (bv. elke 100ms) een clock interrupt genereren, waardoor het proces in uitvoering onderbroken kan worden. We komen in detail terug op deze materie in het onderdeel Scheduling. Wanneer een proces onderbroken wordt door het operating systeem terwijl het eigenlijk nog verder nuttig gebruik kan maken van de processor, dan wijzigt de toestand van het proces in *Ready*. Het proces is niet langer in uitvoering, maar kan wel verder uitgevoerd worden mocht de processor vrij zijn.

Een proces kan ook onderbroken worden omdat het moet wachten op een I/O event (of op een andere service aangeboden door het operating systeem). Soms moet een proces ook wachten op informatie van een ander proces of tot een gedeelde resource vrijkomt (bv. een deel van het geheugen dat gemanipuleerd wordt door meerdere processen). In al deze gevallen wordt het proces onderbroken en stellen we de toestand van het proces gelijk aan *Blocked*. De meeste systemen hebben verder een *New* en *Exit* toestand (zie Figuur 3).

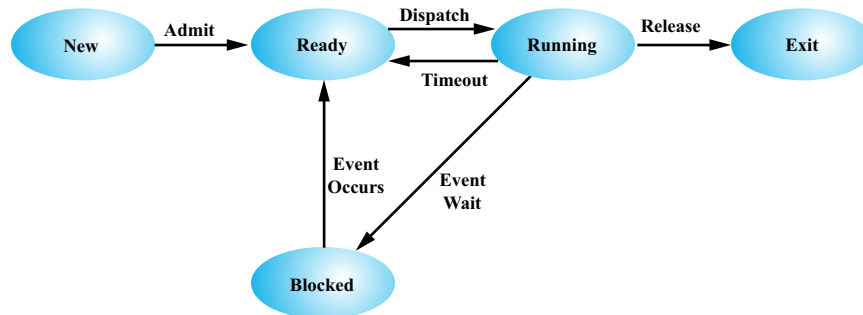


Fig. 3: Proces model met vijf toestanden [Stallings 03]

Een proces bevindt zich in de *New* toestand vlak nadat het gecreëerd is en blijft in deze toestand tot het klaar is voor uitvoering. De *Exit* toestand wordt bereikt wanneer het proces eindigt (mogelijk door het optreden van een fout). Wanneer een proces zich in de *Exit* toestand bevindt is het proces afgelopen maar is er nog data aanwezig die opgevraagd kan worden door het parent process (bv. voor debugging).

Een andere veelgebruikte toestand is de *Suspend* state. Het proces image van een proces dat in deze toestand verzeild geraakt, zal weggeschreven worden naar het secundaire geheugen (typisch naar de swap space van een disk). Om de capaciteit van de processor goed te benutten, moeten er vaak veel actieve processen zijn. Deze vergen echter allemaal geheugen, wat een eerder schaarse resource is. Hoewel het proces image niet volledig in het geheugen aanwezig moet zijn (zie onderdeel Virtueel Geheugen), is het vaak nodig om een aantal process images (bij voorkeur deze in een *Blocked* state) tijdelijk naar disk te schrijven. Er bestaan ook andere redenen om een proces tijdelijk uit het geheugen te verwijderen. Een proces kan bijvoorbeeld periodisch worden uitgevoerd en in de tussenperiodes in de *Suspend* toestand gaan. In sommige gevallen kan ook een *Ready* proces naar de swap space verwezen worden (voornamelijk wanneer een ander proces een erg groot blok van het geheugen vereist). Vandaar dat ook de opsplitsing *Ready*, *suspend* en *Blocked*, *suspend* nuttig kan zijn.

1.3 Kernel, state en mode switching

De *kernel* wordt gevormd door de belangrijkste functies van het operating systeem. Men spreekt van een *monolitische* kernel indien het volledige ope-

rating systeem overeenstemt met de kernel (zoals bijvoorbeeld in Linux), terwijl men anders spreekt van een *microkernel*. Om het systeem te beschermen tegen programmeerfouten, zal de processor gebruik maken van minstens 2 privilege levels: user en kernel level (de x86 architectuur ondersteunt 4 privilege levels) en niet alle instructies (en interrupts) mogen worden uitgevoerd in user mode. Wanneer een user proces een system call uitvoert, dan zal de processor bijgevolg eerst overgaan naar de kernel mode. Op deze manier zorgt men er onder meer voor dat alle data die samen het proces control block vormt, niet kan worden gewijzigd door een user process.

We gaven reeds aan dat een proces onderbroken kan worden omwille van een interrupt. Wanneer een ander proces hierdoor de processor inneemt, moet de state information van het onderbroken proces uit de registers verwijderd worden (en opgeslagen worden in het geheugen). Verder moeten ook de nodige velden in de proces tabellen worden aangepast voor zowel het inkomende als uitgaande proces. Echter, niet elke interrupt hoeft aanleiding te geven tot een *proces switch*. Bijvoorbeeld wanneer het process een system call oproept of er een keyboard interrupt optreedt, kan het lopende proces mogelijk gewoon verder gezet worden wanneer de interrupt is afgehandeld. In dit geval moet enkel de data uit de registers worden opgeslagen en hoeft de overige data in het proces control block niet aangepast te worden (het proces blijft dus de hele tijd in de *Running* toestand). In dit geval spreken we van een *mode switch*, wat minder werk vereist dan een proces switch.

Zelfs de gehanteerde *page table*, die we later uitvoerig zullen bepreken in het hoofdstuk Virtueel Geheugen, blijft ongewijzigd bij een mode switch. Dit komt doordat de kernel functionaliteit deel uitmaakt van elk user proces. Meer bepaald alle kernel functionaliteit zal zich bevinden in de virtuele adresruimte van elk process (doorgaans in de bovenste 2 Gbyte in een 32 bit machine). De system calls worden dus uitgevoerd als zijnde deel uitmakend van het user proces. Het volstaat natuurlijk wel om een enkel exemplaar van de code van deze routines op te slaan in het geheugen (door de page tables zo te configureren). Wanneer een user process overgaat naar kernel mode zal er voor eventuele functie oproepen gebruik gemaakt worden van een aparte stack: de *kernel stack*. Deze bevat eveneens de info die zich in de registers van de processor bevond vlak voor de mode switch optrad. Eén van de redenen dat er een aparte kernel stack is en we niet gewoon de user stack hanteren voor deze oproepen, is dat deze reeds corrupt kan zijn door programmeerfouten in de code van het user process. Wanneer een process zich in kernel mode bevindt, kan het nog steeds onderbroken worden, tenzij

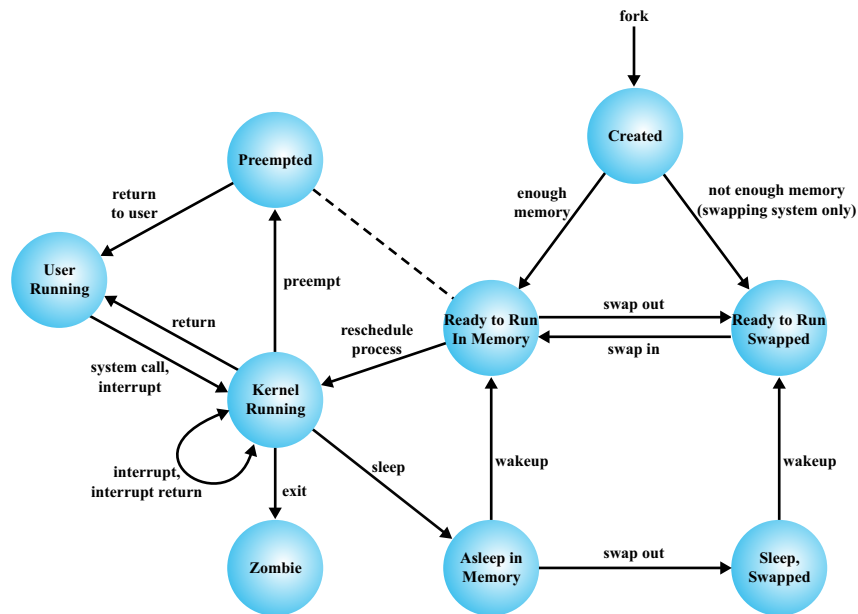


Fig. 4: UNIX toestanddiagram [Stallings 03]

de zogenaamde *interrupt flag* in de processor is gezet.

Een voorbeeld van een toestanddiagram van een UNIX systeem zien we in Figuur 4. In dit systeem is de *Running* toestand opgesplitst in twee toestanden: *User Running* en *Kernel Running*. Hierdoor wordt er aangegeven of het proces zich in de *kernel* of *user mode* bevindt. Ook de *Ready* toestand is opgedeeld in de *Preempted* en *Ready to Run* toestanden. Wanneer een proces van de kernel mode terug overgaat naar de user mode, maar de scheduler beslist om een ander process aan de beurt te laten dan treedt het onderbroken process de *Preempted* toestand binnen. Dit is bijvoorbeeld het geval wanneer er een clock interrupt optreedt wanneer een process in user mode is. In dit geval gaat het process eerst naar kernel mode (omdat de scheduler deel uitmaakt van de kernel) en als deze vervolgens beslist een ander process aan de beurt te laten gaat het process naar de *Preempted* toestand. De processen in de *Preempted* toestand zijn eveneens *Ready to Run* en deze twee toestanden vormen samen één queue. Tenslotte is de blocked toestand opgesplitst in *Sleep, Swapped* en *Asleep in Memory*. Merk op dat een transitie naar deze toestanden eveneens verloopt via de *Kernel Running* toestand.

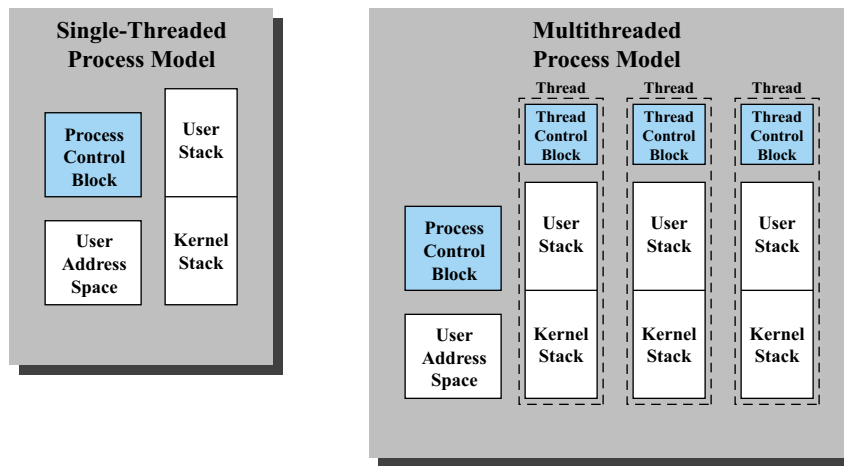


Fig. 5: Single en multithreaded processen [Stallings 03]

2 Threads

Met een proces associëren we enerzijds de data gerelateerd met de resources waarover het beschikt, bv. het (virtueel) geheugen, I/O devices, files, etc. Anderzijds, is er ook de sequentie van instructies die doorlopen wordt en die mogelijk meermaals onderbroken wordt. Bij het doorlopen van de instructies wijzigt ook de toestand van het proces regelmatig.

Heel wat programma's zouden we kunnen opdelen in meerdere processen waarmee dezelfde resources geassocieerd zijn, maar die elk een ander pad van instructies doorlopen. Bijvoorbeeld in het geval van een webserver zouden we een proces per request kunnen creëren zodat we meerdere files simultaan kunnen opvragen (d.i., het wachten van de I/O requests parallel maken). Bij een tekstverwerker zouden we een automatische *save* functie kunnen aanbieden in de vorm van een apart proces dat regelmatig de tekst opslaat in een I/O buffer. Bij een spreadsheet kan men het hertekenen van het scherm en het verwerken van de user input als apart proces runnen (om zo de reactiesnelheid van het programma te verhogen).

Deze verschillende processen hebben dan elk hun eigen proces image. De gezamenlijke code van uitvoering en zelfs de data kan wel gedeeld worden. Echter, omwille van protectie zal de processor in kernel mode moeten gaan om in de gedeelde geheugengebieden te mogen schrijven. Verder heeft elk proces zijn eigen proces control block en moeten er meerdere proces wissels plaats

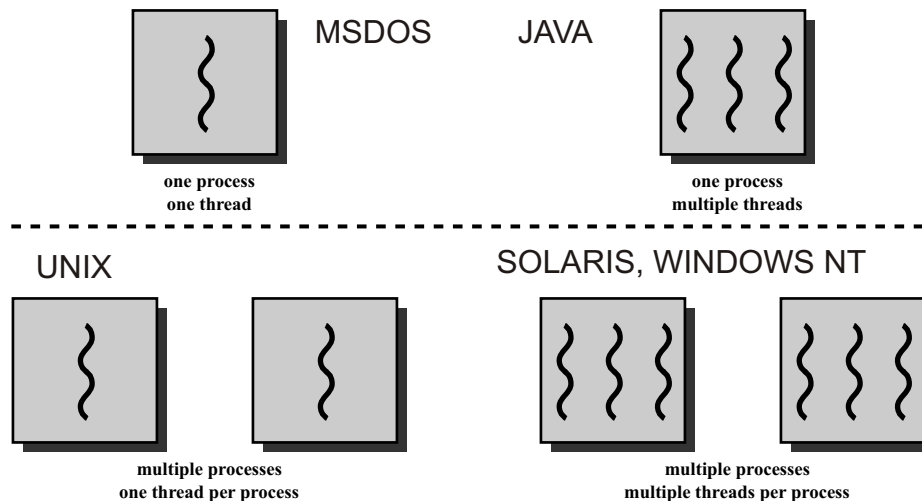


Fig. 6: Processen en Threads [Stallings 03]

vinden, willen we de uitvoering van de processen interleaven. Daarnaast is de levensduur van deze processen soms erg kort, waardoor de overhead voor het aanmaken en tenietdoen van het proces erg groot is. Kortom, dit alles komt de efficiëntie niet ten goede.

Om een programma toch te kunnen opdelen op een meer performante wijze, kan men gebruik maken van *threads*. Threads die deel uitmaken van éénzelfde proces, zullen dezelfde resources delen (d.i., de code/data en het proces control block), maar kunnen elk een eigen pad van uitvoering volgen. Om dit te realiseren beschikken ze wel over een eigen user (en kernel) stack, zie Figuur 5. Een proces bestaat dus uit meerdere threads die elk met elkaar kunnen worden verweven. Vandaar dat elke thread ook een eigen toestand heeft.

Mogelijke toestanden zijn *Running*, *Ready* en *Blocked*. De swapped toestand heeft weinig nut, want een enkele thread wordt niet gewapt. Enkel processen worden gewapt en wanneer dit gebeurt dan zullen alle bijhorende threads eveneens inactief zijn, daar het code en data gedeelte nodig voor uitvoering zich op disk bevindt. Ook wanneer het proces eindigt, eindigen alle threads. In het thread control block vinden we deze toestand terug, samen met andere info om de threads te scheduleren en te synchroniseren. De inhoud van registers (van de thread) moet natuurlijk ook opgeslagen worden per thread, aangezien threads elkaar kunnen onderbreken. Net zoals bij

de gedeelde resources tussen processen, moet de gezamenlijke data van de threads mogelijk tegen gelijktijdige access beschermd worden (zie hoofdstuk Concurrency).

Wanneer een thread data wenst uit te wisselen met andere threads, dan hoeft deze de info maar weg te schrijven in het datagedeelte. Hierdoor wordt deze info onmiddellijk beschikbaar voor alle overige processen. Als een andere thread later actief wordt en de info leest, dan hebben we dus info uitgewisseld zonder dat er een proces wissel of zelfs een mode wissel heeft plaats gevonden. Ook wanneer een thread een bepaalde file (of een andere I/O resource) opent, dan is deze direct beschikbaar voor alle overige threads behorende tot het proces.

2.1 User-level en kernel-level threads

Threads kunnen ruwweg op twee manieren geïmplementeerd worden: user-level threads (ULTs) en kernel-level threads (KLTs). In het geval van ULTs wordt het volledige thread beheer verricht door het proces waartoe de threads behoren. Meer zelfs, de kernel weet niet eens dat het proces uit meerdere threads bestaat.

In dit geval wordt de nodige software om threads aan te maken en te beheren aangeboden door middel van een *thread library*, wat een verzameling routines is om threads aan te maken, te beëindigen, te scheduleren of om boodschappen tussen threads uit te wisselen (bv. de Java virtual machine).

Bij het opstarten van een proces wordt er meestal één thread aangemaakt en deze zal, via de library routines, de andere threads aanmaken. Wanneer de kernel beslist om het proces in uitvoering te onderbreken, bv. omwille van een I/O interrupt of een clock interrupt, dan zal de kernel de processor opeisen en mogelijk later dit of een ander proces laten verder gaan. Gedurende deze hele tijd zal de toestand van de thread die in uitvoering was gelijk aan *Running* blijven. Kijkend naar de thread toestanden lijkt het dus alsof de overige processen niet bestaan en we enkel deze threads uitvoeren op een *dedicated* machine. Merk op dat een proces zelfs kan worden onderbroken tijdens een thread wissel.

ULTs hebben een aantal voordelen:

1. Thread wissels gebeuren in de context van een proces, zonder dat de kernel hierbij betrokken wordt, wat zeer efficiënt is.

2. Het scheduleren van de threads wordt niet beïnvloed door de kernel. Een gepaste scheduling strategie kan hierdoor ontworpen worden voor elke applicatie.
3. ULTs zijn processor onafhankelijk en vereisen geen kernel wijzigingen.

Er zijn natuurlijk ook enkele beperkingen:

1. Wanneer een thread een kernel functie aanroept, dan zal de kernel in vele gevallen het proces dat de aanroep deed in de *Blocked* state plaatsen tot het bijhorende event optreedt. Hierdoor zijn alle threads ineens geblokkeerd.
2. Wanneer het systeem over meerdere processoren beschikt dan zal er toch maar steeds één thread in uitvoering zijn.

Het probleem van blocking kan deels worden opgelost door gebruik te maken van *jacketing*. Een I/O jacket routine van een applicatie zal de blocking call omzetten in een non-blocking call door eerst te checken of het I/O device beschikbaar is. Is dit niet het geval dan zal de thread, via de thread library, de controle overgeven aan een andere thread en zijn toestand op *Ready* plaatsen (en niet op *Blocked*).

Kernel-level threads zijn wel zichtbaar voor de kernel, meer zelfs, de kernel zal instaan voor het thread management. Alle scheduling beslissingen zullen gebeuren op thread niveau (ipv op proces niveau). Deze aanpak lost natuurlijk beide nadelen van ULTs op, maar heeft zijn eigen beperkingen.

De voornaamste beperking is de efficiëntie: een enkele thread wissel vereist inmenging van de kernel en een mode switch is dus telkens noodzakelijk. Experimenten hebben uitgewezen dat de overhead voor het aanmaken, scheduleren, uitvoeren en beëindigen van KLTs 10 tot 30 maal trager verloopt dan ULTs. Dezelfde operaties op proces niveau zijn echter nog tot 10 maal trager. De keuze tussen ULTs en KLTs is vaak erg applicatie afhankelijk. Als de meeste thread switches toch inmenging van de kernel vereisen dan gaat het performantievoordeel van ULTs al snel verloren.

CACHE GEHEUGEN

In dit hoofdstuk zullen we een aantal basisprincipes van *cache* geheugen bespreken en kort ingaan op de performantie ervan. Om te komen tot de noodzaak van een cache geheugen, starten we met een overzicht van verschillende soorten computergeheugen.

1 Computergeheugen: een overzicht

Sinds de opkomst van de allereerste computersystemen is gebleken dat de snelheid van computergeheugen meestal ontoereikend is om te voldoen aan de snelheid waarmee de processor informatie uit het geheugen opvraagt. Sindsdien is deze discrepantie alleen nog maar toegenomen. Het type geheugen dat wel aan deze vereisten kan voldoen, is erg duur en is in omvang erg beperkt. Bijgevolg lijkt het of men zich moet beperken tot systemen met een zeer schaarse hoeveelheid geheugen of tot trage processoren. Om deze beperkingen te omzeilen, heeft het computergeheugen een hiërarchische structuur gekregen.

Geheugen wordt typisch opgedeeld in *intern* en *extern* geheugen. Bij het interne geheugen denken we aan de *registers* aanwezig in de processor, het *cache* geheugen en het *main memory* of RAM geheugen. Dit geheugen is veelal *volatiel*, wat wil zeggen dat het de informatie die het draagt verliest zodra er geen elektrische stroom meer aanwezig is. Dit in tegenstelling tot extern geheugen dat bestaat uit externe apparatuur zoals *disks*, *flash* geheugen en *tapes*.

Geheugenhiërarchie

Zoals aangegeven in voorgaande sectie bestaan er vele types van geheugen. De meeste hedendaagse computersystemen beschikken dan ook over een hele waaier van types computergeheugen. Het typische gebruik wordt goed weergegeven door volgende geheugenhiërarchie (zie Figuur 7). Naarmate we af dalen in deze hiërarchie gebeuren volgende zaken:

1. De prijs per bit neemt af,
2. De omvang van het geheugen neemt toe,
3. De access time (dat is, tijd om het geheugen te raadplegen) neemt toe en
4. De frequentie waarmee de processor data uit het geheugen opvraagt neemt af.

Dit laatste vormt de sleutel tot het succes van de geheugenorganisatie in onze huidige computersystemen. Tenzij in de meerderheid van de gevallen de opgevraagde data ook aanwezig is in het snelle geheugen, heeft het weinig nut om over dit dure geheugen te beschikken. Gelukkig bestaat er zoiets als *locality of reference*, wat betekent dat een bepaalde instructie of bepaalde data vaak veelvuldig en kort na elkaar wordt opgevraagd (denk aan lussen) of dat instructies of data in de buurt van de huidige instructie/data ook met vrij veel kans zal worden opgevraagd in de nabije toekomst (denk aan het vaak sequentieel doorlopen van de code, of aan arrays, etc.). We zullen op dit idee veelvuldig terugkeren in het verdere verloop van dit en andere hoofdstukken.

Voor de omvang van cache moeten we denken in termen van een Kbyte tot enkele Mbytes. Voor geheugen gaan we al snel tot enkele Gbytes, terwijl disks vandaag enkele Tbytes kunnen bevatten. De access time voor een on-chip cache (enkele nanoseconden) is tot 40 keer sneller dan de access time van het main geheugen, terwijl een main memory access vaak 10000 keer sneller is dan een disk access (enkele milliseconden). We komen hier later in dit hoofdstuk op terug.

2 Cache geheugenorganisatie

Cache geheugen is doorgaans het snelste geheugen dat zelf geen deel uitmaakt van de processor (hoewel het zich doorgaans op de chip van de processor bevindt). Naast de zeer lage access time die dit type geheugen kan realiseren (bv. 4 cycles), is het ook belangrijk dat de verbinding tussen de cache en de processor erg snel is. Wanneer het cache geheugen een bus zou moeten delen met het main memory, de I/O apparatuur, de netwerk apparatuur, etc. dan blijft er weinig winst over van de lage access tijd. Bijgevolg wordt een aparte, zeer snelle bus gebruikt om de processor en de cache te verbinden.

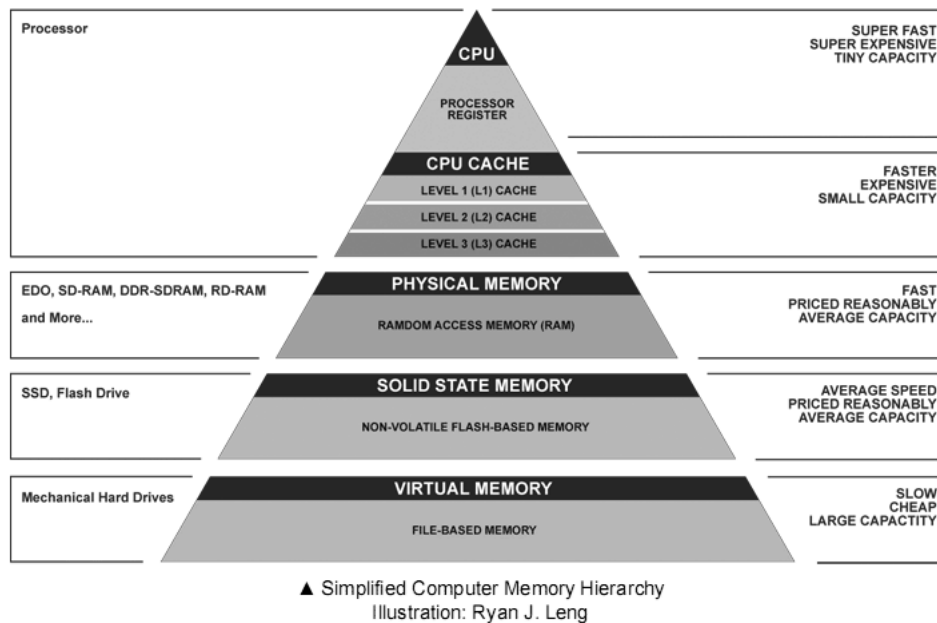


Fig. 7: Geheuenhiërarchie

Het main memory zal dan via een tragere bus met de cache verbonden zijn, die mogelijk wel met andere apparatuur wordt gedeeld (zie Figuur 8).

Een ander onderscheid tussen beide bussen is dat de eenheid waarmee de processor en cache data uitwisselen overeenstemt met de eerder genoemde *word size*, terwijl men tussen het geheugen en de cache grotere blokken zal uitwisselen (bestaande uit K woorden). Dit is ook logisch gezien de zogenaamde locality of reference eigenschap, alsook het feit dat de tragere bus wordt gedeeld en men dus best wat meer data uitwisselt wanneer men over de bus kan beschikken.

De inhoud van de cache bevat een (kleine) deelverzameling van de informatie in het main memory. Laat het main geheugen een totaal van 2^n adresseerbare woorden bevatten (dus de omvang van het geheugen is gelijk aan 2^n maal de woord size). Dan wordt dit geheugen gepartitioneerd in een verzameling van blokken bestaande uit K opeenvolgende woorden: het eerste blok bevat de eerste K woorden, het tweede de volgende K , enz. Het main memory bevat dus precies $M = 2^n/K$ van zulke blokken. Een cache bestaande uit C lines (lijnen) biedt plaats aan precies $C \ll M$ van deze blokken (die elk uit K woorden bestaan). Naast deze K woorden bevat elke

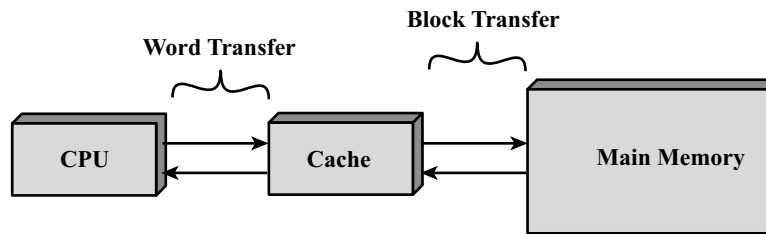
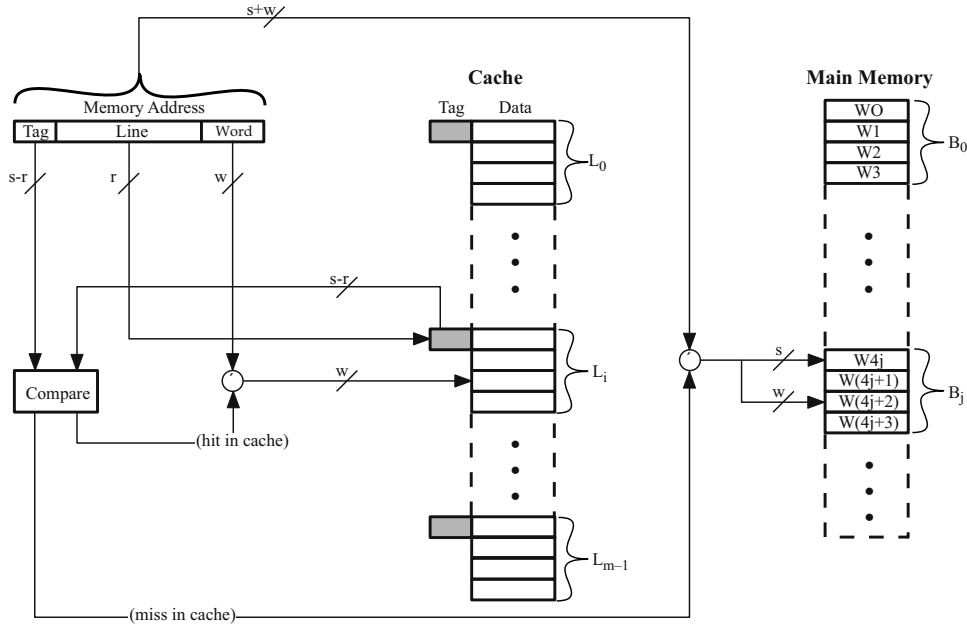


Fig. 8: Cache verbindingen [Stallings 03]

lijn ook een *tag*, die nodig is om te identificeren welk blok uit het main geheugen aanwezig is op deze lijn in de cache. De cache bevat ook een enkele bit per lijn die aangeeft of de lijn in gebruik is en dus betekenisvolle data bevat (alook een UPDATE bit, etc.).

Wanneer de processor een bepaald woord uit het geheugen wenst te lezen, dan zal eerst de cache geraadpleegd worden. Hoe de cache controller, die deze vraag ontvangt (waarin het adres van het woord vervat zit), dit nagaat is afhankelijk van de manier waarop de cache ontworpen is. Verder in dit hoofdstuk zullen we drie van deze ontwerpen bespreken. Indien het gevraagde woord gevonden wordt, we spreken hier van een *hit*, dan zal de cache de gevraagde informatie direct aan de processor doorgeven. In dit geval wordt het tragere main geheugen dus niet geraadpleegd en hoeven we ook geen gebruik te maken van de gedeelde, minder snelle bus. Wanneer het blok, dat het gevraagde woord bevat, ontbreekt, zal de cache het blok dat deze informatie bevat opvragen uit het main geheugen en het opslagen in één van de cache lijnen voor mogelijk later hergebruik.

Voor een efficiënte werking van de cache structuur is het van vitaal belang dat de kans H dat een gevraagd woord zich in de cache bevindt, groot is. Zonder de eerder vermelde locality of reference is deze kans, de *hit ratio* genaamd, slechts gelijk aan $C/M \approx 0$ en moeten we dus bijna altijd terugvallen op het main geheugen. Deze kans H wordt natuurlijk ook beïnvloed door de keuze van de lijn waarin we een nieuw blok in de cache opslaan. Verder in dit hoofdstuk zullen we hiervoor een aantal strategieën bespreken. Daarnaast zal H ook een functie zijn van het aantal lijnen C in de cache. Hoe groter de cache, hoe waarschijnlijker het is dat een bepaald woord zich ook daadwerkelijk in de cache bevindt. Echter hoe groter C , hoe duurder (en trager) de cache wordt.

Fig. 9: Direct Mapping [Hwang 93], $m = C$

2.1 Direct mapping

Zoals we eerder opmerkten, is het aantal lijnen in de cache $C = 2^r$ beduidend kleiner dan het aantal blokken M in het main geheugen. De *mapping function* zal aangeven waar we blok nummer j in de cache moeten plaatsen. Vanuit wiskundig oogpunt is de naam functie niet goed gekozen, daar de meeste mappings toelaten dat een blok op verschillende plaatsen kan geschreven worden (er is dus een keuzemogelijkheid). De eenvoudigste mapping, *direct mapping* genaamd, zal echter elk blok op een enkele lijn i afbeelden door de eenvoudige relatie

$$i = j \mod 2^r (= C). \quad (1)$$

Stel $K = 2^w$, dus elk blok (en elke lijn in de cache) bevat precies 2^w woorden. Dan zullen de laatste w bits, die we de minst significante noemen, van een adres aangeven met het hoeveelste woord in een blok dit adres overeenstemt.

Kijken we nu naar het adres van een bepaald woord, dat bestaat uit $a = s + w$ bits, dan zullen de overige s bits, de eerste dus, het nummer j van

het blok vormen. Wanneer we nu het overeenkomstige lijnnummer wensen te berekenen, volstaat het te kijken naar de r minst significante bits van de eerste s bits. Alle adressen die dus dezelfde $r + w$ laatste bits hebben, worden allemaal op dezelfde lijn van de cache afgebeeld. De tag, die identificeert welk blok zich effectief in het geheugen bevindt, komt dus precies overeen met de $s - r$ eerste bits van het adres. Merk op dat we door deze keuze van de mapping, ervoor zorgen dat de blokken die op dezelfde lijn worden afgebeeld zich ook vrij ver van elkaar bevinden in het geheugen (namelijk precies 2^{r+w} plaatsen).

De werking van de cache en de wijze waarop de controller nagaat of het gevraagde woord zich in de cache bevindt, wordt gedemonstreerd aan de hand van Figuur 9. Eerst zal de controller de cache lijn bepalen door naar de r middelste bits te kijken. Vervolgens wordt de tag van deze lijn vergeleken met de tag van het gevraagde adres. Stemt deze overeen dan vinden we het gevraagde woord in de cache door te kijken naar de w minst significante bits van het adres. Anders moeten we het main geheugen raadplegen.

De direct mapping methode kunnen we dus als volgt samenvatten:

- Adres lengte $= a = s + w$ bits,
- Aantal adresseerbare woorden is gelijk aan 2^a ,
- De blok grootte $=$ lijn grootte $K = 2^w$ woorden,
- Aantal blokken in het geheugen $M = 2^a / 2^w = 2^s$,
- Aantal lijnen in de cache $C = 2^r$,
- Aantal bits in de tag $= s - r$ bits.

Hoewel deze techniek erg eenvoudig is, brengt ze een aantal nadelen met zich mee. Wanneer men bijvoorbeeld afwisselend gegevens nodig heeft die zich in twee blokken bevinden die op dezelfde lijn worden afgebeeld, dan zal het ene blok steeds het andere overschrijven, en moeten we dus telkens terugvallen op het main geheugen. Dit fenomeen wordt *thrashing* genoemd. Heel erg veel komt dit niet voor daar geheugen blokken die dezelfde cache lijn gebruiken vrij ver uit elkaar liggen en we het locality of reference fenomeen hebben. Echter, wanneer we bijvoorbeeld een eenvoudige lus uitvoeren waarbij in het ene blok de instructies staan en in het andere toevallig de data, dan kan dit nadeel zeer sterk doorwegen. Een ander voorbeeld dat op

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
    sum += a[i]*b[i];
```

The generic system executes this loop as follows:

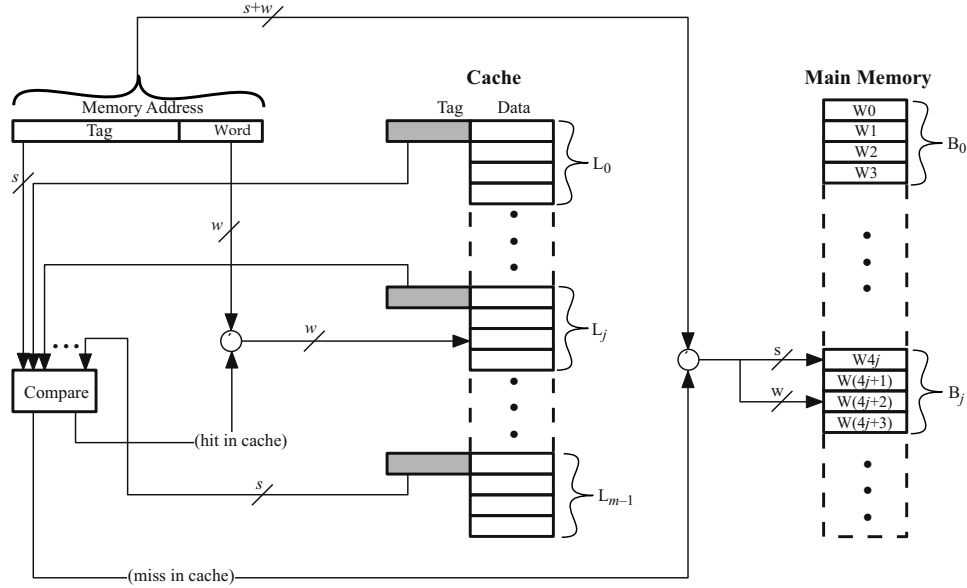
i	Operation	Status	In cache	Comment
0	load a[0]	miss	a[0..7]	assume a[] was not cached yet
	load b[0]	miss	b[0..7]	assume b[] was not cached yet
	t=a[0]*b[0]			
	sum += t			
1	load a[1]	hit	a[0..7]	previous load brought it in
	load b[1]	hit	b[0..7]	previous load brought it in
	t=a[1]*b[1]			
	sum += t			
	 etc			
7	load a[7]	hit	a[0..7]	previous load brought it in
	load b[7]	hit	b[0..7]	previous load brought it in
	t=a[7]*b[7]			
	sum += t			
8	load a[8]	miss	a[8..15]	this line was not cached yet
	load b[8]	miss	b[8..15]	this line was not cached yet
	t=a[8]*b[8]			
	sum += t			
9	load a[9]	hit	a[8..15]	previous load brought it in
	load b[9]	hit	b[8..15]	previous load brought it in
	t=a[9]*b[9]			
	sum += t			
				

Fig. 10: Cache trashing voorbeeld

het eerste zicht een goede cache hit rate realiseert zien we in Figuur 10. We veronderstellen hier dat we over een 4 Kbyte cache beschikken met blokken van 32 bytes. Een float heeft een grootte van 4 bytes, dus elke cache lijn bevat 8 floats en we onderstellen dat zowel de variabele **a** als **b** nog niet in de cache zitten. Het lijkt er op dat we een hit rate van 87.5 procent realiseren, wat meestal ook correct is. Echter, wanneer **a** en **b** zich vlak na elkaar in het geheugen bevinden dan is **a[i]** precies 4 Kbyte van **b[i]** verwijderd en krijgen we een hit rate van 0 procent. Merk op dat hetzelfde probleem zich ook zou voordoen wanneer de arrays **a** en **b** een veelvoud van 1024 als grootte hebben.

2.2 Associative en set-associative mapping

In geval van associative mapping mag elk geheugenblok op elke mogelijke lijn in de cache geschreven worden. De lijn wordt dus niet langer door de r middelste bits bepaald. Bijgevolg zal het thrashing fenomeen niet meer in dezelfde mate optreden. Echter, bij associative mapping heeft de tag een lengte van s bits (ipv $s - r$). Daar elk blok zich nu overal in de cache kan bevinden, moeten we nu het volledige bloknummer onthouden, hetgeen

Fig. 11: Associative Mapping [Hwang 93], $m = C$

precies overeenstemt met de eerste s bits van het adres.

Wanneer we wensen na te gaan of een bepaald woord zich in de cache bevindt, moeten we de tag van het gevraagde woord vergelijken met alle tags in de cache (zie Figuur 11). Dit zal gebeuren door alle tags in parallel te vergelijken met de gevraagde tag. Dit type (cache) geheugen access noemen we *associative*, in tegenstelling tot het voorgaande dat we *direct* access noemen. Wanneer het woord zich niet in de cache bevindt, zullen we het blok waarvan het deel uitmaakt, opvragen uit het main geheugen en opslaan in de cache. De plaats waar we dit blok opslaan, kunnen we nu vrij kiezen in tegenstelling tot de direct mapping techniek. Algoritmen hiervoor die trachten de hit ratio te maximaliseren zullen we verder bespreken.

Associative geheugen access is erg complex en duur, vandaar dat er ook een tussenvorm van cache geheugen werd ontwikkeld, genaamd *set-associative* caching. In dit geval is de locatie van een blok in de cache niet vastgelegd zoals bij direct caching, maar ook niet meer volledig vrij zoals bij associative caching. De cache wordt echter opgedeeld in $v = 2^d$ sets die elk precies $k = C/v = 2^{r-d}$ lijnen bevatten (de eerste set bevat de eerste k lijnen, de

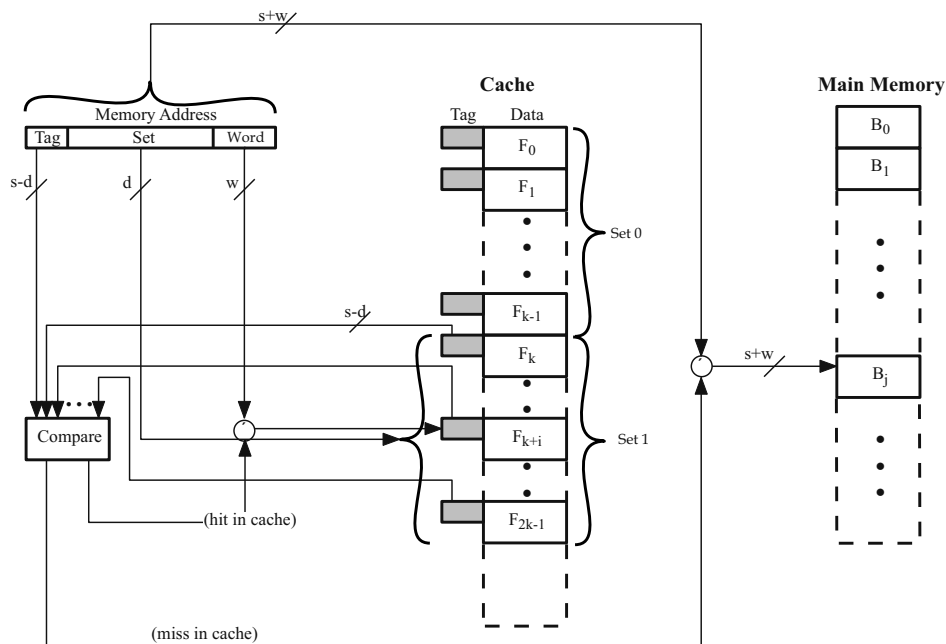


Fig. 12: Set Associative Mapping [Stallings 03]

tweede de volgende k , enz.). Elk blok zal nu precies gebruik mogen maken van één van deze sets. In het bijzonder, wanneer we kijken naar de eerste s bits van een adres, die het bloknummer aangeven, dan zullen de laatste d van deze bits de set identificeren waarbinnen we het blok mogen schrijven. Daar alle blokken die in een set kunnen staan dezelfde laatste d bits bevatten, volstaat het dat de tag lengte $s - d$ heeft. In dit geval spreekt men van een *k-way set-associative cache*.

Wanneer nu een bepaald woord wordt opgezocht, zal de controller eerst de set bepalen door naar de d middelste bits te kijken. Vervolgens wordt de tag van het adres vergeleken met de k tags in de set om te bepalen of het gevraagde woord zich in de cache bevindt. Wanneer $k = 1$ valt deze methode samen met direct caching, terwijl $k = C$ aangeeft dat we te maken hebben met associative caching. Deze tussenoplossing is minder duur, daar er minder tags simultaan vergeleken hoeven te worden. Daarnaast lost deze tussenoplossing ook het probleem van de trashing voor een groot deel op. Wanneer $k = 4$ moet men al herhaaldelijk 5 of meer blokken aanspreken die mappen op dezelfde set. Anderzijds merken we ook dat de blokken die

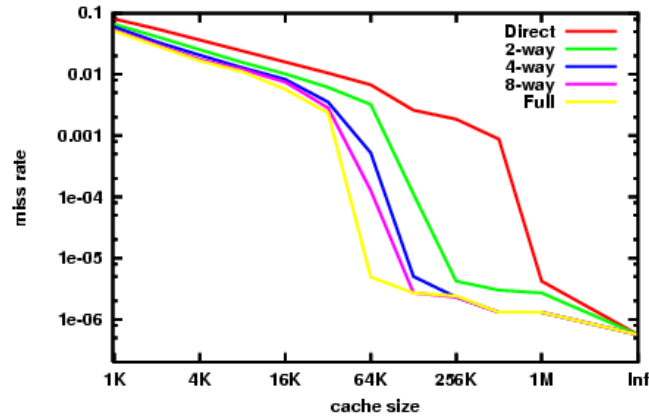


Fig. 13: Miss rate with set-associative caching in a PowerPC G3/4

mappen op dezelfde set dicht bij elkaar gelegen zijn in het main geheugen. In de praktijk is k meestal erg klein, bijvoorbeeld 2, 4, 8, etc. Zulke kleine waarden volstaan reeds om de meeste gevallen van trashing te vermijden. Dit wordt geïllustreerd door Figuur 13 voor de PowerPC G3/4 die een 8-way associative cache (van 2×32 Kbyte) hanteert.

2.3 Replacement algoritmen

Tenzij men gebruik maakt van direct mapping, bestaat er steeds een keuze wanneer we een nieuw blok in het cache geheugen plaatsen. Het algoritme dat we hanteren om deze keuze te maken, noemen we het *replacement* algoritme. Idealerweise willen we steeds dat blok verwijderen dat we in de nabije toekomst het minst waarschijnlijk terug zullen aanspreken. In het kader van de reference of locality eigenschap ligt het dus voor de hand om een *least recently used* (LRU) strategie te hanteren. Dit algoritme geeft aan dat we uit de set het blok kiezen dat het langst niet meer is opgevraagd door de processor. Natuurlijk vergt het enige moeite om bij te houden wanneer we welke blokken hebben aangeroepen en dit in elke set (in geval van set-associative caching). Een voor de hand liggende implementatie van LRU, is gebruik te maken van een stack. Wanneer data uit een bepaald blok binnen een set wordt gevraagd, verwijderen we dit bloknummer uit de stack en plaatsen het bovenaan de stack. Deze stack vergt $k \log_2 k$ bits, er zijn namelijk k elementen in de set die we elk op hun beurt identificeren met $\log_2 k$ bits en dit voor

elke set.

Een eenvoudigere oplossing is de *first in first out* (FIFO) methode, waarbij we steeds het oudste blok verwijderen. Dit kunnen we eenvoudig met een circulaire buffer implementeren, waardoor we slechts $\log_2 k$ bits nodig hebben, per set. In geval van FIFO veronderstellen we dat de oudste blokken ook het minst recent gebruikt zijn.

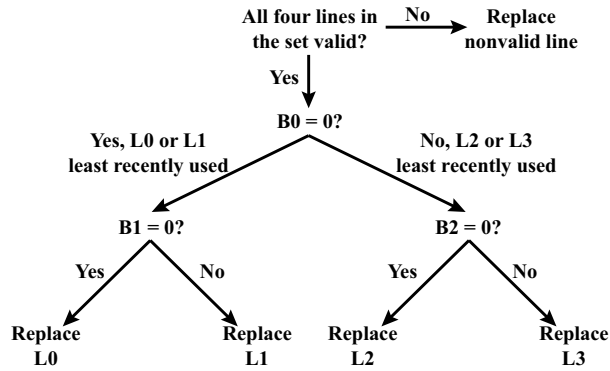


Fig. 14: Pseudo LRU cache replacement strategy on the Intel 40486 on-chip cache [Stallings 03]

Daarnaast maakt men soms ook gebruik van *pseudo LRU* algoritmen. Twee belangrijke varianten hiervan zijn de *tree-based* (TB) en *most recently used* (MRU) technieken. In geval van de TB wordt er een lijst van $k - 1$ bits bijgehouden: $B_0, B_1, \dots, B_{k-2}$. De eerste bit geeft aan of het de eerste $k/2$ lijnen of de laatste $k/2$ lijnen zijn die het minst recent gebruikt zijn. De tweede bit geeft aan welke helft van de eerste $k/2$ lijnen het minst recent is gebruikt, terwijl de derde bit dit doet voor de laatste $k/2$ lijnen, etc (zie Figuur 14). Wanneer we een nieuw blok wensen weg te schrijven in een set, doorlopen we de boom om het lijnnummer te bepalen. Terwijl we de boom doorlopen zetten we ook steeds de overeenkomstige bit zodat de tak die we niet nemen als de minst recent gebruikte wordt aangeduid (we switchen de bit dus om). Wanneer er een cache hit optreedt, doorlopen we ook de boom naar de gevraagde cache locatie toe en passen zo nodig de bits onderweg aan. Men kan aantonen dat dit algoritme nooit één van de $\log_2 k = r - d$ meest recent gebruikte blokken (of lijnen) zal verwijderen (voor $k \geq 2$). De Intel 80486 on-chip cache maakte gebruik van het TB pseudo LRU algoritme met $k = 4$.

De MRU variant zal een lijst van k bits bijhouden. Wanneer een hit gebeurt op lijn i dan zetten we de i -de bit gelijk aan 1 (tenzij hij al gelijk aan 1 was). Wanneer we nu een nieuw blok wensen in te voegen in de set, dan zoeken we een blok waarvoor de overeenkomstige bit gelijk is aan 0 en vervangen dit blok (en zetten de bit ervan op 1). Wanneer we de laatste 0 vervangen door een 1, zetten we alle andere bits terug op nul. Merk op dat we nooit de meest recent gebruikte blok overschrijven, maar mogelijk wel de tweede meest recent gebruikte. Wanneer we tijdens het invoegen van een nieuw blok de laatste 0 invullen, dan overschrijven we zeker het minst recent gebruikte blok. Voor $k = 2$ vallen beide pseudo LRU algoritmen samen met LRU.

3 Geheugen updates: write policy

Naast het lezen van data uit de cache, wordt er natuurlijk ook in de cache geschreven. Zoals we eerder opmerkten, zit de informatie aanwezig in de cache ook in het main geheugen. Wanneer we enkel de data in de cache updaten ontstaat er dus een discrepantie tussen beide geheugens. Dit kan voor problemen zorgen daar ook andere componenten, zoals de I/O apparatuur, deze data kan aanspreken zonder beroep te doen op de processor (via *direct memory access* (DMA)).

Men onderscheidt in het algemeen twee technieken: *write through* en *write back*. In het eerste, eenvoudigste maar ook minst efficiënte geval, zal elke update in de cache ook onmiddellijk worden doorgevoerd in het main geheugen. Wat voor extra verkeer zorgt op de bus wanneer bepaalde data frequent wordt geüpdatet. Bij *write back* zal men bij het aanpassen van de data in de cache een UPDATE bit zetten, om aan te duiden dat deze informatie verschilt van het main geheugen. Wanneer het blok uiteindelijk uit de cache verwijderd wordt, zal nagekeken worden of de UPDATE bit staat, is dit het geval dan wordt het main geheugen aangepast. Extra complicaties kunnen zich voordoen wanneer er meerdere processoren zijn die elk een cache bevatten met daarin mogelijk dezelfde informatie (zie verder).

4 Aantal caches in hedendaagse systemen

Hedendaagse computersystemen maken gebruik van meerdere caches. Zo is er (per core) doorgaans een cache van beperkte omvang (bv. 32 Kbyte)

<i>L1</i> CACHE hit	≈ 4 cycles
<i>L2</i> CACHE hit	≈ 10 cycles
<i>L3</i> CACHE hit	line unshared ≈ 40 cycles
<i>L3</i> CACHE hit	shared line in another core ≈ 65 cycles
<i>L3</i> CACHE hit	modified in another core ≈ 75 cycles
Dram	$\approx 60 - 100$ ns

Tab. 1: Intel Core i7 Xeon 5500 Series Data Source Latency

voorzien op de chip van de processor, de *L1* cache genaamd. Bevat de *L1* cache de gevraagde data niet, dan wordt de *L2* cache geraadpleegd. De *L2* cache is doorgaans wat groter (bv. 256 Kbyte), maar trager en elke core beschikt vaak over zijn eigen *L2* cache. Tot slot is er de *L3* cache die veelal gedeeld wordt tussen de verschillende cores. De *L3* cache bevat enkele Mbyte data, maar is ook de traagste van alle caches.

Wanneer er meerdere caches gehanteerd worden, is de vraag of dezelfde data zich in slechts één of in meerdere caches kan/mag bevinden. Bij een *inclusieve* cache zal de inhoud van de *L1* cache steeds een deelverzameling vormen van de *L2* cache (en analoog voor de *L2* en *L3* cache). *Exclusieve* caches laten echter niet toe dat data zich in verschillende caches tegelijk bevinden en een hit in de *L2* cache zal in dit geval ervoor zorgen dat de lijn uit de *L2* cache wordt gewisseld met een lijn uit de *L1* cache.

In de praktijk maakt men eveneens gebruik van een aparte cache voor instructies en data, wat men aanduidt met de term *split cache*. Het nadeel is dat een deel van de cache capaciteit verloren gaat. Soms is er namelijk meer nood aan data, terwijl gedurende andere periodes er veelal gebruik gemaakt wordt van instructies. Er zijn echter ook sterke voordelen: men kan uit beide caches gelijktijdig informatie opvragen. Dit is erg efficiënt wanneer de processor aan *prefetching* doet, wat aangeeft dat de processor tracht reeds een deel van de toekomstige instructies en data in de registers te laden om zo de access time van het geheugen te reduceren.

In het hoofdstuk Virtueel Geheugen zullen we zien dat hedendaagse processoren eveneens zijn uitgerust met TLB caches die bedoeld zijn snel logische in fysieke geheugenadressen om te zetten. Tot slot geeft Tabel 1 ter informatie de tijden weer die ongeveer nodig zijn om de verschillende caches te raadplegen op een Intel Core i7 machine. Het verschil tussen de drie gevallen van een *L3* cache hit zal duidelijk worden wanneer we het MESI protocol bespreken (verder in de cursus).

5 Cache benchmarking

In deze sectie bespreken we een eenvoudige test om de verschillende eigenschappen van het cache systeem op je machine te ontdekken. Het idee bestaat erin om een aantal arrays van integers, met $2^{C_{MIN}}$ tot $2^{C_{MAX}}$ entries, elk een aantal maal te doorlopen. Tijdens de eerste reeks van passages worden alle elementen opgevraagd. Bij de tweede reeks vragen we enkel elementen 2, 4, 6, etc. op. In het algemeen, bij de i -de reeks passages lezen we enkel entries $2^i, 2 \cdot 2^i, 3 \cdot 2^i$, etc. We verwijzen naar de stapgrootte van een reeks passages als de *stride*. Door de gemiddelde access tijd als functie van de stride waarde uit te zetten voor elke array grootte verkrijgen we een plot zoals in Figuur 15. Deze figuur is wat geïdealiseerd, tijdens de oefeningen zal er een meer realistische plot worden bekeken.

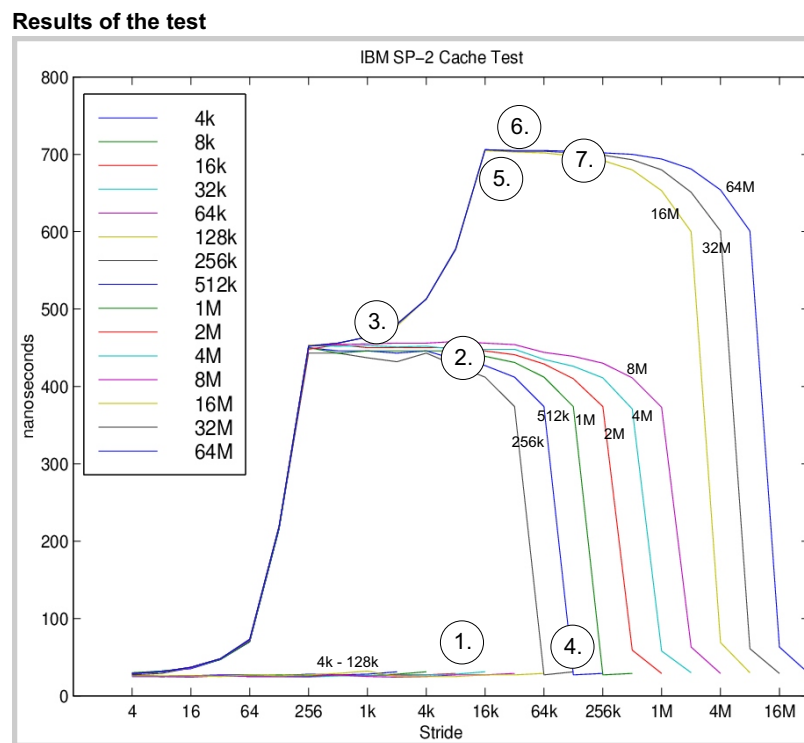


Fig. 15: Resultaten van de benchmarking test

We kunnen uit deze plot heel wat leren over onze cache (de replacement

policy mag je LRU of FIFO veronderstellen):

1. **De grootte van de $L1$ cache is 128 Kbyte.** Namelijk, voor de arrays met een omvang van 128K of minder is de access tijd rond de 25 nsec. Bij de eerste passage worden alle elementen in de cache gezet, zonder dat deze door de latere passages nog verwijderd worden. Er zijn bijgevolg geen misses meer. Vanaf 256K treden er nog wel misses op, daar de gemiddelde access tijd sterk toeneemt. Dit geeft aan dat de array niet meer in de cache past. Voor een array van 256K zal de tweede helft van de array steeds de eerste helft overschrijven.
2. De extra rekentijd als gevolg van een $L1$ cache miss is ongeveer 425 nanoseconden.
3. **Een enkele cache lijn bevat 256 bytes.** Dit zien we het eenvoudigste aan de resultaten van de arrays met een omvang van minstens 256K. Wanneer de stride half zo groot is als een enkele cache lijn, dan is er minstens één hit bij elke twee woorden. De access tijd ligt dus halfweg tussen de 25 en de 450 nsec. Is de stride een vierde dan zijn er minstens drie hits per miss. Bij een stride van 256 bytes zijn er enkel nog misses, elke woord behoort bijgevolg tot een andere lijn.
4. **De $L1$ cache is 4-way associatief.** Wanneer de stride bijna even groot is als de array zelf, dan worden er tijdens een enkele passage bijna geen elementen meer gelezen. Voor de array van 256K zal een stride van 128K resulteren in het lezen van slechts twee elementen, die precies 128K uit elkaar gelegen zijn. Deze twee elementen worden bijgevolg steeds op dezelfde set afgebeeld (daar de $r + w$ meest rechtse bits identiek zijn). In de figuur zien we dat deze twee elementen wel steeds in hits resulteren, vandaar dat de associativiteit van de cache minstens twee moet zijn. Omdat de sets minstens twee elementen bevatten, worden ook de vier elementen van de 256K array, die we lezen bij een stride van 64K, op dezelfde set afgebeeld (want hun laatste $r - 1 + w$ meest rechtse bits zijn identiek). Opnieuw zien we dat deze vier elementen veelal in hits resulteren, waardoor een set minstens vier elementen moet bevatten. Deze redenering kunnen we herhalen, tot op het moment dat er plots wel meestal cache misses optreden, wat aan geeft dat een enkele set niet meer al de te lezen elementen kan bevatten (in de veronderstelling dat er een niet al te dom replacement algoritme

wordt gehanteerd). In dit voorbeeld gebeurt dit bij een stride van $32K$, wat met 8 elementen overeenstemt. Hierdoor kunnen we besluiten dat de associativiteit van de $L1$ cache 4 moet zijn.

5. **Er is geen L2 cache.** De plot bevat slechts 3 niveaus: een $L1$ hit (25 nsec), een $L1$ miss met een hit in de TLB cache (450 nsec, zie Hoofdstuk Virtueel Geheugen voor meer info over de TLB cache) en een $L1$ miss met een miss in de TLB cache (700 nsec). Indien er nog een extra niveau zou zijn, dan was er sprake van een $L2$ cache.
6. **De TLB page size is 16K, de TLB cache bevat 512 entries en is 4-way associatief.** De overgang van niveau twee naar niveau drie voltrekt zich bij een stride van $16K$. Voor kleinere strides zal de vertaling van het virtuele adres naar het fysieke adres vaak via de TLB cache kunnen verlopen (omdat meerdere elementen zich op dezelfde page bevinden). Aangezien de array van 8M nog geen TLB misses lijkt te veroorzaken, moet de TLB cache minstens 512 entries bevatten, daar 512 maal $16K$ gelijk is aan 8M. Voor de 16M array zijn er wel misses, waaruit we mogen besluiten dat de TLB cache naar alle waarschijnlijkheid geen 1024 entries bevat. Wat de associativiteit betreft observeren we hetzelfde gedrag voor bij grote arrays en strides als bij de $256K$ array, wat aangeeft dat de TLB cache eveneens 4-way associatief is. Mocht de TLB cache een grotere associativiteit hebben dan zouden de tijden eerst terugvallen tot 450ns, omdat de TLB cache wel de te lezen elementen in een enkele set kan bevatten, terwijl dit niet het geval is voor de $L1$ cache.
7. De extra rekentijd omwille van een TLB miss is 250 nsec.

GEHEUGEN MANAGEMENT

De tijd dat er slechts één programma gelijktijdig in het geheugen van een computersysteem werd geladen, is natuurlijk al lang voorbij. Vandaag de dag zal, zoals we al eerder aangaven, het operating systeem moeten instaan voor het beheren van vele processen, gezien anders het grootste deel van de processorcapaciteit zou verloren gaan. Eén van de voornaamste en meest complexe taken bestaat er dan ook in om al deze processen op een efficiënte wijze het geheugen, dat sinds de ontwikkeling van de eerste computersystemen schaars is, te laten delen. Gezien het grote aantal processen zal slechts een deel van hen (gedeeltelijk) aanwezig zijn in het main geheugen op elk moment in de tijd. Het operating systeem staat dus in voor het bepalen van enerzijds welke processen in het geheugen aanwezig zijn (iets waar we later op terug komen) en anderzijds op welke wijze de aanwezige processen (d.i., hun images) logisch en fysisch georganiseerd worden, wat het onderwerp is van dit hoofdstuk. We weten reeds dat er gebruik gemaakt wordt van secundair, of nog virtueel, geheugen, typisch een deel van de diskspace. Processen moeten dus, nadat ze opgestart zijn en in het geheugen werden geladen, mogelijk meermaals plaats maken voor andere processen. Wanneer ze, door een scheduling beslissing van het operating systeem, terug in het main geheugen worden geladen, hoeft dit niet noodzakelijk op dezelfde locatie te zijn. Daarnaast moet de geheugenruimte van de processen ook beschermd worden voor niet geautoriseerde access door andere processen en moet de mogelijkheid bestaan om gebieden van het geheugen te delen.

Loading en linking

Een eerste item dat nauw samenhangt met geheugenbeheer is de wijze waarop de object code (d.i., de instructies plus data sectie) van een proces in het geheugen wordt geladen. Hiervoor bestaan er een aantal mogelijkheden. Ten eerste is het mogelijk dat de object code gebruik maakt van *fysieke/absolute* adressen, bepaald door de compiler (of programmeur). In dit geval is er voor de *loader*, die instaat voor het laden van de object code in het geheugen,

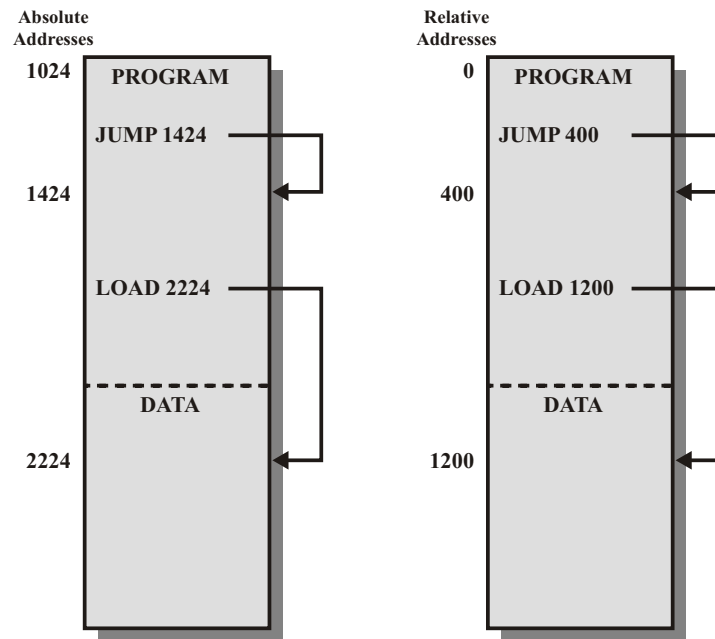


Fig. 16: Absolute en Relocatable load modules [Stallings 03]

weinig keuze: de code kan slechts op één plaats worden ingelezen, daar de adressen in de *branch* instructies en data referenties anders niet kloppen. De loader heeft in dit geval natuurlijk een erg eenvoudige taak.

Een iets meer flexibele mogelijkheid is dat de object code wordt aangemaakt door de compiler met *logische/relatieve* adressen. In dit geval zal elk adres relatief zijn ten opzichte van een bepaalde plaats in het geheugen, meestal het beginpunt (zie Figuur 16). Deze logische adressen moeten nog vertaald worden naar fysieke adressen. Wanneer dit gebeurt bij het laden van de object code, spreekt men van *relocatable loading*. De load module zal elk logisch adres vervangen door een fysiek adres daar deze weet waar de code zal opgeslagen worden. Informatie die aangeeft waar de te vertalen adressen zich bevinden, wordt door de compiler aangemaakt (d.i., de *relocation directory*). Relocatable loaders hebben het voordeel dat de locatie van een proces in het geheugen niet op voorhand is vastgelegd. Echter, eens geladen liggen de fysieke adressen vast en moet het proces, telkens wanneer het een tijdje naar de swap space was verwezen, opnieuw op dezelfde locatie worden ingeladen.

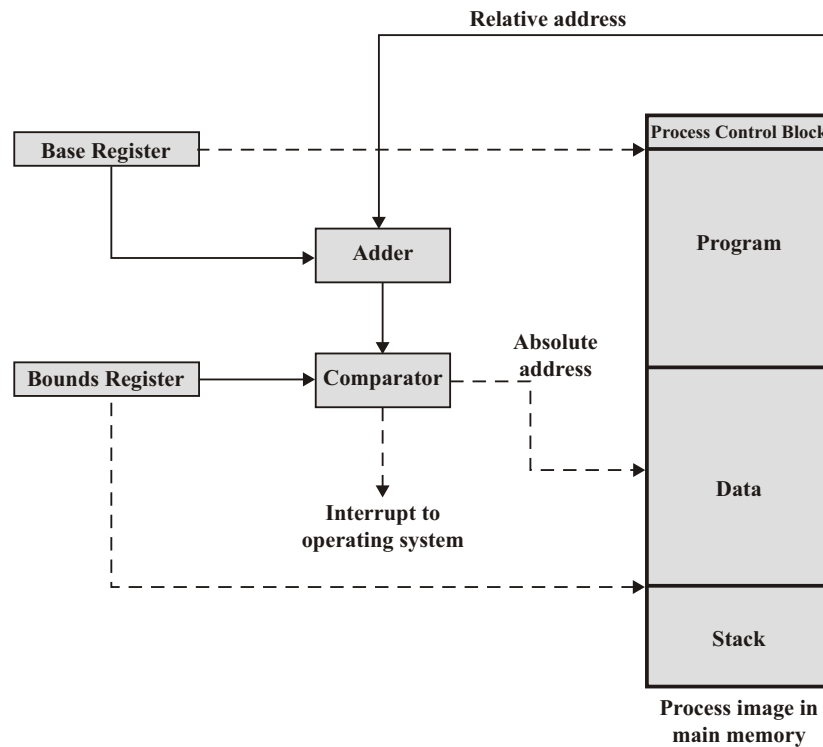


Fig. 17: Hardware ondersteuning voor adresvertaling [Stallings 03]

Dit geeft aanleiding tot een erg inefficiënt gebruik van het geheugen.

De oplossing hiertoe bestaat erin de adresvertaling pas door te voeren *at run-time*, of dus wanneer de instructie effectief wordt uitgevoerd. De loader zal dan de object code met logische adressen inladen en de processor zelf zal instaan voor de vertaling door gebruik te maken van speciaal hiervoor ontworpen hardware. Dit heeft als groot voordeel dat een zelfde proces, gedurende zijn levensduur, nu zonder problemen kan worden ingeladen op verschillende geheugenlocaties. Het operating systeem heeft hierdoor de mogelijkheid om aan *relocation* te doen. Een grafische voorstelling van de hardware support voor relocation zien we in Figuur 17, waar we voor de eenvoud onderstellen dat het proces image ononderbroken aanwezig is in het geheugen.

De code van een proces zal in heel wat gevallen natuurlijk niet bestaan uit een enkele object code file, maar uit meerdere zulke files (wat het hergebruik van code aanzienlijk vereenvoudigt). In dat geval moeten de verschillende object code files ook gelinkt worden met elkaar door de zogenaamde *linker*

module, zodat ook referenties naar de code en data van de andere modules correct verloopt. Dit kan gebeuren bij de compilatie of bij het inladen van de module. Tenslotte merken we ook op dat het proces control block en de proces stack samen met de ingeladen object code het proces image vormen.

1 Partitionering

Nu we deze beschouwingen achter de rug hebben, kunnen we ons richten op de hoofdzaak van dit hoofdstuk: de geheugenorganisatie. Er zijn historisch drie belangrijke manieren om het geheugen van een computersysteem te organiseren: *partitioning*, *paging* en *segmentation*. Daar de twee laatste in grote mate de eerste mogelijkheid overtreffen in termen van performantie, vinden we partitioneren nog enkel terug in het kernel gedeelte van een aantal operating systemen. Toch zullen we deze organisatiemethode ook van naderbij bekijken, daar ze een aantal belangrijke problemen aankaart, die door de twee laatstgenoemde technieken vermeden worden. Partitioneren is ook de eenvoudigste en oudste van deze technieken.

In de meeste systemen zal een deel van het geheugen voorbehouden zijn aan het operating systeem. Onze aandacht gaat voornamelijk uit naar het beheer van het overblijvende geheugen dat gedeeld moet worden door de verschillende user processen. Daarnaast zal er heel vaak ook gebruik worden gemaakt van secundair, of virtueel geheugen, een onderwerp waar we later in de cursus op terugkomen. Voorlopig beperken we ons tot het main geheugen.

1.1 Fixed partitioning

Een eerste optie is het geheugen in een aantal delen, partities genaamd, te verdelen. Deze kunnen van vaste of variabele grootte zijn. Een proces dat moet ingeladen worden in het geheugen, zal één van deze partities ter zijner beschikking krijgen. Wanneer alle partities even groot zijn is het inladen van processen eenvoudig: is er een partitie vrij dan gebruiken we deze voor het te laden proces. Wat we hierbij opmerken is dat elk proces een even groot deel van het geheugen krijgt, ongeacht hoeveel geheugen het vergt. Dit geeft aanleiding tot wat men *interne fragmentatie* noemt: zelfs wanneer een partitie is ingenomen door een proces blijft, gemiddeld gezien, de helft van de partitie ongebruikt. Daarnaast kan het ook gebeuren dat een proces te groot is om in een partitie te passen.

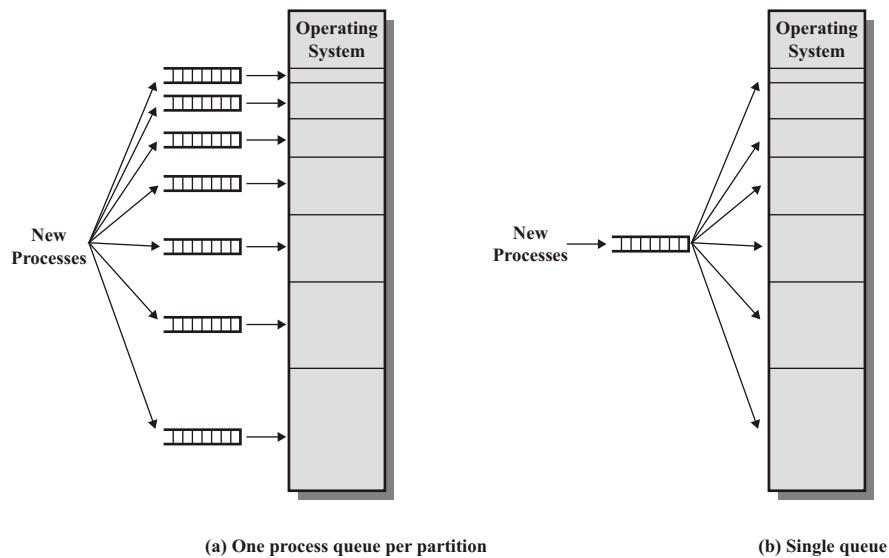


Fig. 18: Vaste partities met variabele grootte: proces placement [Stallings 03]

Partities van variabele lengte lossen deze beide problemen gedeeltelijk op. Door een variatie van partitiegroottes aan te bieden gaat er minder geheugen verloren en kunnen we grotere processen ondersteunen. Het blijft wel erg moeilijk om op voorhand te bepalen hoe we de partities dan best kiezen. Een bijkomende vraag is hoe we een partitie selecteren wanneer we een nieuw proces wensen te laden in het geheugen. Om interne fragmentatie te vermijden kunnen we best de kleinst mogelijke partitiegrootte selecteren die het proces image omsluit. Mogelijk zijn alle partities van deze grootte al in gebruik. We hebben dan twee opties (zie Figuur 18): (1) ofwel laten we het proces wachten tot één van deze partities vrijkomt, (2) we kunnen ook opteren voor een andere, vrije partitie die voldoende groot is. De eerste keuze maximaliseert het geheugengebruik binnen een partitie, terwijl optie twee aanleiding kan geven tot een beter gebruik in termen van het aantal partities.

1.2 Dynamic partitioning

Daar vaste partities aanleiding geven tot *interne fragmentatie*, lijkt de oplossing eenvoudig: we maken de partities dynamisch aan zodat ze exact de

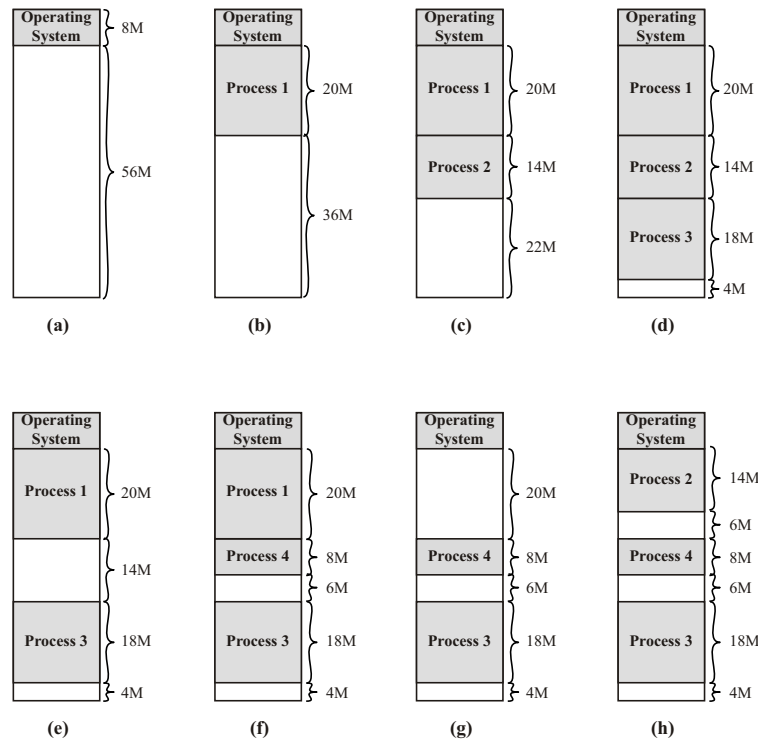


Fig. 19: Evolutie van geheugen onder dynamische partitionering [Stallings 03]

vereiste grootte hebben. We moeten hierbij wel rekening houden met het feit dat er steeds nieuwe processen bijkomen en eindigen. Een voorbeeld zien we terug in Figuur 19, merk op dat proces 2 in stap (h) ook een nieuw proces kan zijn (proces 5).

We stellen vast dat er ongebruikte gebieden in het geheugen ontstaan tussen de partities in, die we *holes* (of leegtes) noemen. Het ontstaan van deze leegtes, noemt men *externe fragmentatie*. Wanneer er zich een nieuw proces aandient, moeten we ook bepalen waar we de nieuwe partitie zullen plaatsen. Hierbij kunnen we trachten de leegtes die ontstaan te dichten. Eens we bepaald hebben in welke leegte we de partitie plaatsen, zorgen we ervoor dat deze aansluit bij de vorige partitie zodat we geen extra leegte creëren.

Hoeveel leegtes kunnen we verwachten in zulk een systeem? Stel dat we het geheugen een lange tijd observeren en noteer p_i als de kans dat een proces image omgeven wordt door i leegtes, voor $i = 0, 1$ en 2 . Dan zal het aantal

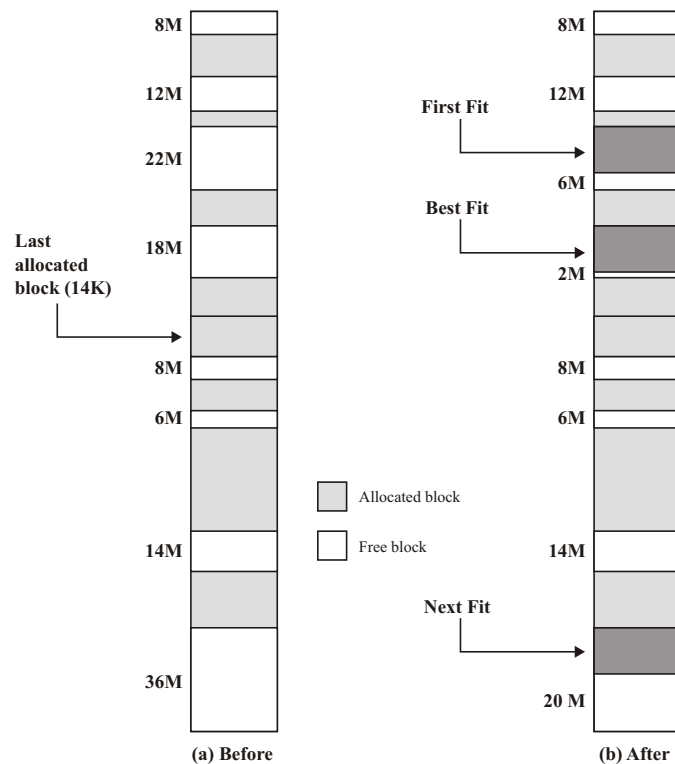


Fig. 20: Placement algoritmen onder dynamische partitionering [Stallings 03]

leegtes gemiddeld met $p_0 - p_2$ toenemen wanneer een proces het geheugen verlaat. Namelijk als er twee leegtes zijn, dan worden deze samengevoegd, waardoor het aantal met één afneemt. Zijn er geen leegtes, dan komt er één bij. Bij het toevoegen zal, op de uitzondering na dat een partitie precies past, het aantal ongewijzigd blijven. Het gemiddeld aantal leegtes zal over de tijd gezien niet wijzigen, dus moet p_0 gelijk zijn aan p_2 . Het gemiddeld aantal leegtes rondom een proces is precies gelijk aan $p_1 + 2p_2 = p_1 + p_2 + p_0 = 1$, door gebruik te maken van de afgeleide gelijkheid. Kortom, er zijn gemiddeld gezien twee maal zoveel processen aanwezig als leegtes (daar een leegte steeds omgeven is door twee processen). Dit resultaat is gekend als de 50 procent regel.

Het plaatsen van partities kan gebeuren aan de hand van een aantal algoritmen (zie ook Figuur 20). Zo kunnen we zoeken naar de kleinste mogelijke leegte die voldoende groot is (Best-fit), we kunnen de eerste leegte lokaliseren

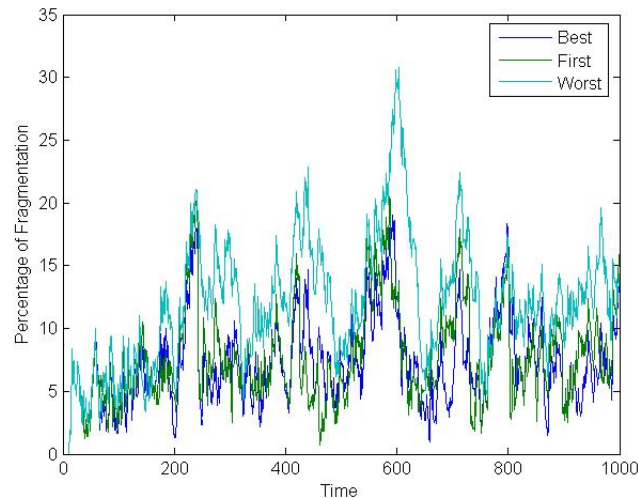


Fig. 21: Gemeten fragmentatie voor Uniform verdeelde proces lengtes tussen 1 en 40 eenheden

die volstaat (First-fit), de volgende ten opzichte van de laatst gealloceerde partitie (Next-fit) of zelfs de grootste leegte (Worst-fit). Op het eerste zicht lijkt het Best-fit algoritme het meest efficiënt, aangezien het de gaten zo goed mogelijk tracht te vullen. Echter, Best-fit creëert erg veel eerder kleine leegtes, die mogelijk weinig bruikbaar zijn om andere processen in te laden en zodanig is deze methode niet altijd de meest efficiënte. Neem bijvoorbeeld dat we twee leegtes hebben van grootte 20 en 70 Kbyte en alle processen vergen minstens 12 Kbyte. Als het eerstvolgende proces 12 Kbyte vraagt, dan zal Best-fit een leegte creëren van 8 Kbyte, dewelke onbruikbaar is voor alle overige processen. In dit geval is het beter te wachten tot er zich een proces aandient dat de 20 Kbyte beter opvult. Best-fit maakt dit type van leegtes frequent aan.

Het andere extreem is de Worst-fit, deze zal de grootste leegte kiezen, precies om het bovengenoemde fenomeen tegen te gaan. Echter door steeds voor de grootste leegte te opteren, zullen grote processen vaak geen geschikte leegte meer vinden, waardoor er achteraan een grote extra hoeveelheid geheugen moet worden gealloceerd. Het hoeft dus niet te verbazen dat de First-fit en Next-fit strategieën regelmatig de meest efficiënte zijn (hoewel alles erg afhankelijk is van het scenario). First-fit zal vooral vooraan vele kleine leegtes

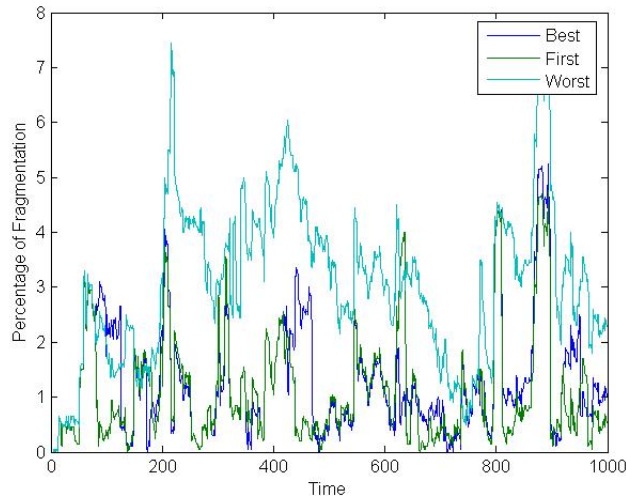


Fig. 22: Gemeten fragmentatie voor Spiky verdeelde proces lengtes 1, 2, 4, 8, 16 en 32 eenheden (waarbij de kans steeds halveert bij het verdubbelen van de lengte)

creëren. Next-fit is doorgaans sneller in uitvoering omdat het in tegenstelling tot First-fit niet steeds de reeks kleine leegtes moet overlopen die vooraan in het geheugen gelokaliseerd zijn. In het verleden is de Next-fit strategie dan ook veelvuldig gebruikt.

Experimentele bevestiging van bovenstaande beschouwingen voor Best-fit, First-fit en Worst-fit wordt in Figuur 21 en 22 weergegeven. In deze setup dient er zich op elk tijdstip een nieuw proces aan, de processen hebben allemaal een looptijd van geometrische duur met een gemiddelde van 20 tijdstippen. Er is voldoende geheugen voorzien zodat er nooit een proces niet kan toetreden tot het geheugen. De gemeten fragmentatie stemt overeen met de som van alle leegtes die zich tussen de processen bevinden, inclus de eventuele leegte vooraan in het geheugen (maar natuurlijk zonder de grote leegte achteraan).

Men kan het voorkomen van de leegtes ook aanpakken door *compaction*: regelmatig zal het operating systeem alle processen reloceren zodat hun partities terug een aaneengesloten blok vormen. Deze operatie is vanzelfsprekend erg duur in termen van procestijd.

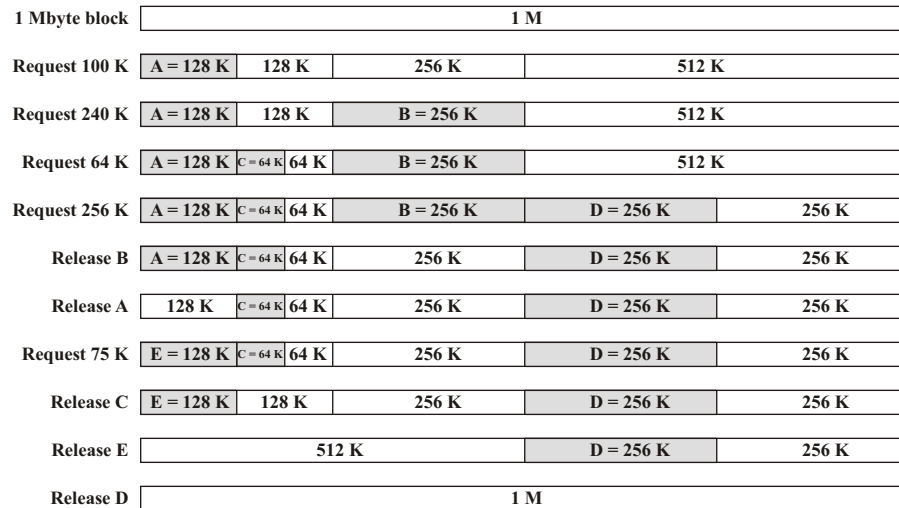


Fig. 23: Voorbeeld van de evolutie van het Buddy systeem [Stallings 03]

1.3 Buddy systeem

Het *buddy* systeem is een interessante variant op fixed en dynamic partitioning. Het idee is dat de geheugenomvang gelijk is aan een macht van 2, stel 2^H . De omvang van elke partitie is eveneens een macht van 2 gelegen tussen 2^L en 2^H . Voor elk van deze $H - L + 1$ blokgroottes zal het systeem een lijst bijhouden van de (vrije) blokken met deze omvang. Initieel is er slechts één blok, met grootte 2^H , en zijn alle overige lijsten dus ook leeg. Wanneer een proces een blok van grootte k vereist, met $2^{i-1} < k \leq 2^i$, dan wordt de volgende recursieve procedure gehanteerd:

```

procedure get hole(i);
begin
  if i_list empty then begin
    get hole(i+1);
    split hole into buddies;
    put buddies on i_list;
  end;
  take first hole on i_list
end;

```

Indien er geen blok van de juiste grootte aanwezig is, zal een groter vrij

blok (recursief) opgedeeld worden tot er een blok van de juiste omvang aangemaakt is (zie ook Figuur 23). De toestand van het buddy systeem kan men eenvoudig weergeven door middel van een boomstructuur. Indien een blok weer vrijgegeven wordt, doordat een proces uit het geheugen verwijderd wordt, dan wordt dit blok mogelijk met zijn *buddy* weer samengevoegd tot een groter blok, ten minste als de buddy zelf ook (volledig) vrij is. In de boomvoorstelling is er dus steeds één van beide kinderen (deels) gevuld. Een variant van het Buddy systeem wordt bij de kernel geheugenallocatie van bepaalde UNIX systemen gebruikt.

We kunnen ook eenvoudig inschatten hoeveel extra geheugen een proces image toebedeeld krijgt, in de veronderstelling dat de grootte van het proces image een random lengte heeft kleiner dan het beschikbare geheugen. Namelijk, als het image b bytes lang is, dan zal er een stuk van $2^{\lceil \log_2 b \rceil}$ bytes worden toegekend (merk op, $2^{\lceil \log_2 b \rceil}$ is de kleinste macht van 2 die groter is dan b). De eerste helft van dit stuk wordt zeker benut, terwijl we gemiddeld gezien de tweede helft voor 50% benutten. Kortom, $3/4$ van het toegekende deel wordt effectief gebruikt door het image. Met andere woorden, een image krijgt 33% meer geheugen toebedeeld dan nodig (d.i., $4/3 \approx 1.33$).

2 Paging

Een alternatieve werkwijze voor partitionering bestaat erin het volledige geheugen op te delen in kleine blokken van vaste grootte, die we *page frames* of kortweg *frames* noemen. Het image van een proces wordt eveneens in een aantal stukken verdeeld, die we *pages* noemen. Deze pages hebben dezelfde grootte als de frames waarin het geheugen is opgedeeld. Wanneer we nu het proces wensen te laden in het geheugen zullen er precies voldoende frames gealloceerd worden. Een belangrijk verschil met partitionering is dat deze frames geen aaneengesloten blok hoeven te vormen in het geheugen. Deze techniek wordt *paging* genaamd.

Bij paging hebben we dus geen *externe* fragmentatie, er is wel nog een erg beperkte vorm van interne fragmentatie daar de omvang van het proces image niet noodzakelijk een veelvoud hoeft te zijn van de frame/page grootte. Gemiddeld gaat het echter maar om de helft van een page per proces, in tegenstelling tot partitionering.

Paging vergt natuurlijk extra werk bij het vertalen van de logische naar fysieke adressen. Een adres bestaat nu immers uit een page nummer en een

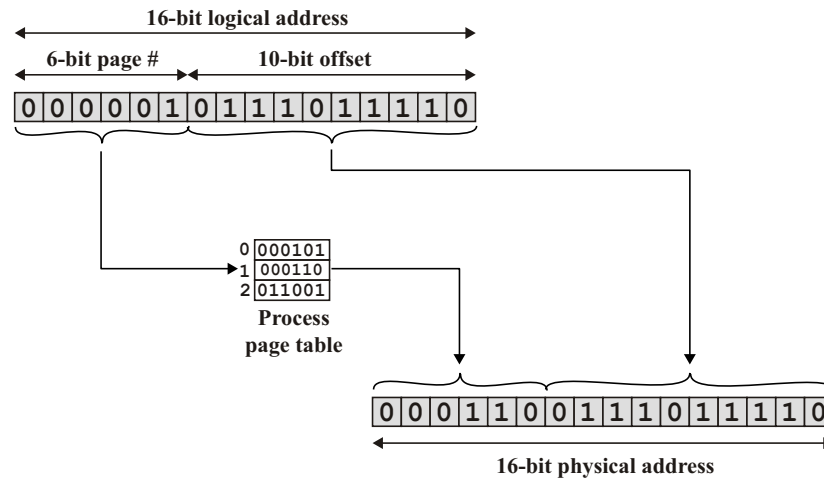


Fig. 24: Adresvertaling voor paging [Stallings 03]

offset waarde binnen deze page. De processor kan niet meer eenvoudigweg het logische adres bij de base register waarde optellen, aangezien de pages verspreid liggen over het geheugen. Om het juiste adres te bepalen zal er daarom, per proces, een *page table* bijgehouden worden. Deze table bevat de nummers van alle pagina's van een proces (op de i -de plaats vinden we de page frame nummer van page i terug, wat snel opzoeken toelaat). De table wordt dus telkens geraadpleegd wanneer we een fysiek adres moeten bepalen.

Stel dat een logisch adres bestaat uit $n + m$ bits en 2^m is de grootte van een page frame in bytes. Dan zullen de eerste n bits het page nummer vormen, terwijl de overige m bits de offset binnen de page aangeven. Om een logisch adres te vertalen volstaat het de eerste n bits van het logisch adres te nemen en deze als index tot de page table te gebruiken om het frame nummer te bekomen. Dit frame nummer vormt de eerste n bits van het fysieke adres. Vervolgens voegt men gewoon de m laatste bits van het logische adres toe (zie Figuur 24) om het fysieke adres te bekomen.

Het vertalen van de logische naar fysieke adressen gebeurt volledig hardwarematig. De hardware zal de eerste n bits van het logische adres nemen, de page table raadplegen (die is opgeslagen in het RAM geheugen) en vervolgens, wanneer er een frame nummer is bekomen, de offset eraan plakken. Doordat dit volledig in hardware gebeurt, is het ook de hardware die het formaat van een page table entry vastlegt.

Op 32 bit machines die gebruik maken van de x86 architectuur is een

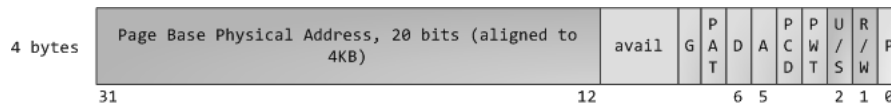


Fig. 25: Page table entry op 32 bit x86 architectuur

page 4096 bytes groot en bestaat een page nummer uit 20 bits, wat toelaat tot 4 Gbyte geheugen aan te spreken. Een page table entry bestaat in dit geval, zoals aangegeven in Figuur 25, uit 4 bytes en bevat niet enkel het 20 bit frame nummer, maar bevat ook enkele extra bits. De P bit geeft aan of de page aanwezig is in het geheugen, zo niet dan mogen alle resterende 31 bits vrij gebruikt worden. De U/S bit geeft aan of een page al dan niet mag worden opgevraagd in user mode, wat er voor zorgt dat het geheugen dat gebruikt wordt door de kernel niet kan worden geraadpleegd door user code. Verder is er de R/W een bit die aangeeft of er data geschreven mag worden op een page. Dit is nuttig omdat er op deze manier voor gezorgd kan worden dat instructies van een process niet per ongeluk overschreven kunnen worden door fouten in de user code. De D bit geeft aan of aan page *dirty* is, wat wil zeggen dat ze is aangepast en de A bit geeft aan of de page al is geraadpleegd. Merk op dat de page table er eveneens voor zorgt dat een process niet kan schrijven in de geheugenruimte van een ander process. Daar de page nummers bestaan uit 20 bits, bevat de volledige page table 2^{20} entries. Dit maakt dat een page table 4 Mbyte geheugen zou vergen per process, wat erg veel is. In het hoofdstuk Virtueel Geheugen zullen we aangeven hoe de vereiste hoeveelheid geheugen per page table beperkt kan worden.

Wanneer we gebruik maken van paging dan lijkt het voor de programmeur/compiler alsof het image van het proces als een samenhangend blok in het geheugen aanwezig is. Vandaar dat men zegt dat paging transparant is naar de compiler/programmeur toe.

3 Segmentation

Paging kan men interpreteren als een fixed partitionering schema met vaste partitiegroottes, met dat verschil dat een proces nu kan beschikken over meerdere partities die niet naast elkaar gelegen hoeven te zijn. Hierdoor treedt er ook enkel interne fragmentatie op. Segmentatie verhoudt zich op

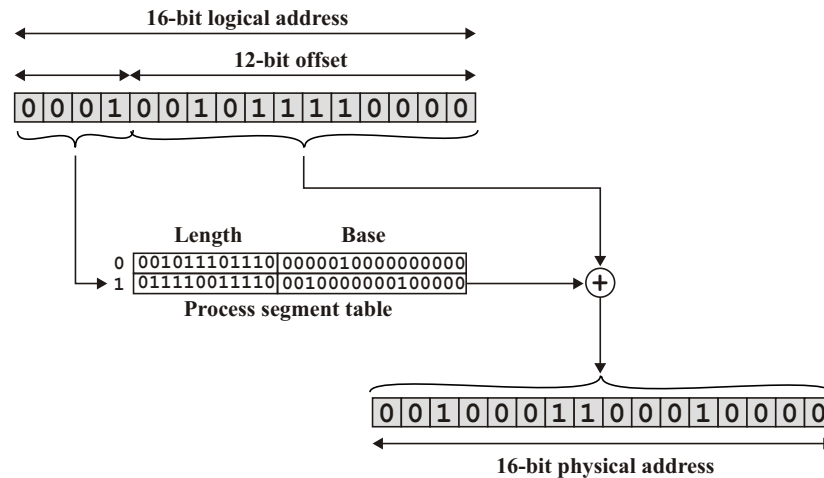


Fig. 26: Adresvertaling voor segmentatie [Stallings 03]

dezelfde wijze tot dynamisch partitioneren als paging tot fixed partitioneren. Namelijk, een proces kan beschikken over één of meer segmenten, die variabel in grootte kunnen zijn en die niet noodzakelijk als een aaneengesloten blok in het geheugen aanwezig zijn. Interne fragmentatie wordt door deze aanpak vermeden, er kan echter wel externe fragmentatie optreden. In vergelijking met dynamische partities zal deze minder sterk doorwegen, gezien een proces image over meerdere segmenten gespreid kan worden.

Een logisch adres van $n + m$ bits bestaat in het geval van segmentatie, uit een segmentgedeelte dat de n meest linkse bits omvat en een offset die gevormd wordt door de overige m bits. Om dit logische adres om te zetten in een fysiek adres, wordt er gebruik gemaakt van een segment table. Deze table bevat per segment het startadres van het segment, de *base value* genaamd, alsook de lengte. Deze lengte wordt opgeslagen om de geldigheid van een adres te kunnen controleren. Wanneer blijkt dat de offset gevormd door de m meest rechtse bits kleiner is dan de segmentlengte, wordt de offset bij de base value opgeteld om te komen tot het overeenkomstige fysieke adres (zie Figuur 26).

Segmentatie werd initieel geïntroduceerd op de x86 architectuur om ervoor te zorgen dat er meer dan 64 Kbyte geheugen kon worden ondersteund op een 16 bit machine (tot 1 Mbyte). Op de x86 architectuur wordt een logisch adres eerst omgezet in een lineair adres via segmentatie en daarna

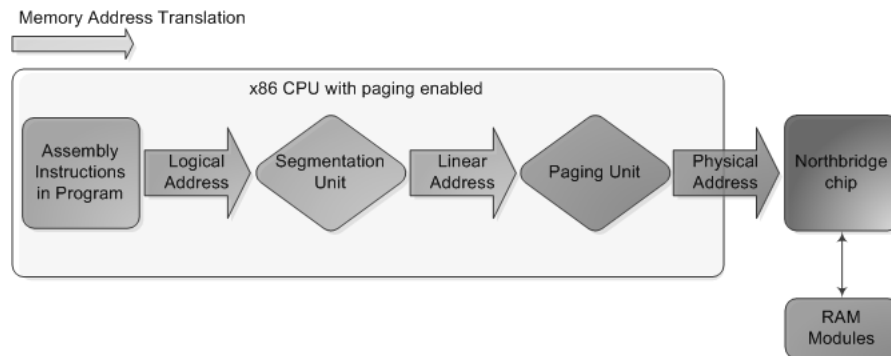


Fig. 27: Adresvertaling op x86 architectuur

wordt het fysieke adres bekomen via paging (zie Figuur 27). De omzetting van het logische naar lineaire adres via segmentatie verloopt echter iets anders dan eerder aangegeven. Zo wordt er gebruik gemaakt van 4 segment registers die elks 16 bits groot zijn. Deze 16 bits bepalen (onrechtstreeks) het segment dat gebruikt wordt zoals verder aangegeven. De keuze van het segment register dat wordt gebruikt bij het bepalen van het lineaire adres wordt impliciet bepaald door de instructie. Zo is er een register voor de stack, voor programma code en twee registers voor data.

Om het logische adres om te zetten in een fysiek adres op de x86 zijn er twee mogelijke modes: *real mode* en *protected mode*. In *real mode* zal de waarde in het segment register worden vermenigvuldigd met 16 en wordt vervolgens het logische adres hierbij opgeteld om het lineaire 20 bit adres te bekomen. Merk op, in dit geval kan hetzelfde lineaire adres bekomen worden door verschillende combinaties van een logische adres en een segment nummer (iets waar MS-DOS programmeurs vaak gebruik van maakte, zodat ze niet steeds de waarde in de segment registers moesten aanpassen). Verder kan men op deze manier ook adressen vormen die net iets groter zijn dan 1 Mbyte. Aangezien er op een machine met 1 Mbyte geheugen slechts 20 bits kunnen worden doorgegeven aan het RAM geheugen (via de Northbridge chip), werd de 21-ste bit impliciet gelijk gesteld aan nul.

De programmeur kan (op uitzondering van het segment register voor programma code) ook expliciet de waarde van een segment register invullen (via de MOV instructie) en in dit opzicht is segmentatie niet transparant naar de programmeur toe. In *protected mode* wordt er wel gewerkt met een segment table en wordt de inhoud van het gebruikte segment register gebruikt als

index tot de segment table. De segment table voorziet eveneens protectie door enerzijds te werken met een limiet op de segment grootte (zoals eerder aangegeven) en anderzijds door ook bij te houden welk privilege level vereist is om dit segment te gebruiken. Op de x86 zijn er 4 privilege levels, maar de meeste operating systemen gebruiken slechts twee levels: level 0 (kernel mode) en level 3 (user mode).

Real mode segmentatie wordt gebruikt wanneer de machine opstart en daarna wordt overgeschakeld op protected mode segmentatie (tenslotte moet de segment table eerst nog worden opgesteld). Hoewel de x86 architectuur nog steeds werkt met segmentatie, wordt de waarde van de segment registers in protected mode sinds de introductie van 32 bit machines vaak gelijk gesteld aan 0 (en de segment limiet aan 4 Gbyte), waardoor de logische en lineaire adressen identiek zijn. In dit geval zegt men dat de segmentatie werkt in *flat* mode, wat op 64 bit machines steeds het geval is.

VIRTUEEL GEHEUGEN

Een eerste belangrijke stap in geheugenmanagement bestond erin niet langer vast te houden aan het idee dat een proces (image) als een enkel aangesloten blok in het geheugen moet worden opgeslagen, wat aanleiding gaf tot het ontstaan van paging en segmentatie. De volgende stap zou men dan kunnen aanduiden door het besef dat mogelijk niet alle pages die behoren tot een enkel proces gelijktijdig in het geheugen aanwezig hoeven te zijn. We zagen reeds bij het onderdeel over caches dat er zoiets bestaat als het locality of reference principe. In de context van paging doet ditzelfde principe het vermoeden ontstaan dat, over niet al te lange tijdspannes, mogelijk slechts een beperkt aantal pages nodig zijn. Kortom, zonder in te boeten aan snelheid zou men een deel van de pages van een proces kunnen opslaan in het secundaire geheugen (swap space). Wanneer een page is geswapt, dan kan men gebruik maken van de overeenkomstige page table entry om aan te geven waar de page terug te vinden is (op het I/O device).

Het idee dat niet alle pages gelijktijdig in het geheugen aanwezig moeten zijn, laat niet enkel toe dat we meer processen (gedeeltelijk) in het geheugen kunnen laden, maar dat een enkel proces zelfs een grotere adresruimte kan aanspreken dan deze die fysiek aanwezig is. Vandaar dat we de logische adressen, die worden aangemaakt door de compiler, eveneens aanduiden met de term virtuele adressen.

In dit hoofdstuk richten we ons op paging systemen. De situatie bij het gebruik van virtueel geheugen vertoont enkele gelijkenissen met de caching problematiek uit de voorgaande hoofdstukken. De pages van een proces zijn slechts gedeeltelijk aanwezig in het geheugen, wat snellere access toelaat, de overige pages vinden we terug in de swap space. Wanneer een page aanwezig is in het geheugen, kunnen we onmiddellijk over de page beschikken, anders moet deze ingeladen worden. In plaats van een *cache miss*, spreken we in het geval dat de page niet aanwezig is, van een *page fault*. Het opvangen van de page faults is de verantwoordelijkheid van het operating systeem. Om te weten welke page de page fault veroorzaakte zal dit page nummer door de paging hardware worden opgeslagen in een register. Een belangrijk verschil

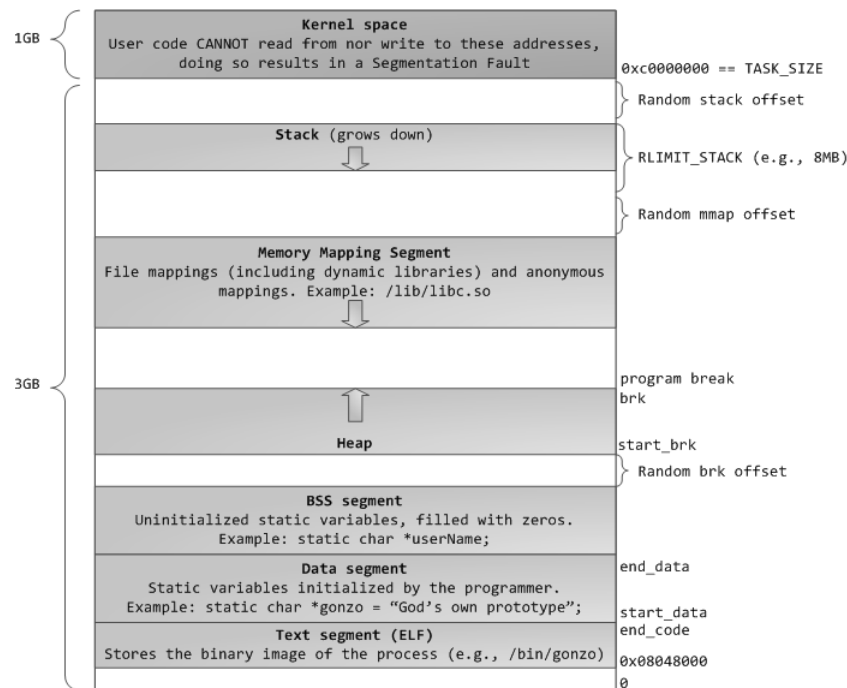


Fig. 28: Virtuele geheugenlayout in Linux

met caching is natuurlijk dat het opzoeken van de pages via een page table verloopt, en niet via een directe/associatieve mapping.

1 Virtuele geheugenlayout

Voor we in meer detail kijken naar de manier waarop we page tables kunnen implementeren zonder dat deze al te veel geheugen vergen, kijken we eerst even naar de wijze waarop een process typisch het virtuele geheugen opdeelt. We kijken hierbij naar een typische geheugenlayout in Linux op een 32 bit machine. Figuur 28 geeft aan dat de virtuele adresruimte van een process wordt opgedeeld in verschillende segmenten. Deze segmenten hebben niets te maken met de segmentatie besproken in Sectie 3 van het voorgaande hoofdstuk.

We zien dat de bovenste GByte (in Windows standaard 2 Gbyte) gereserveerd is voor het mappen van het geheugen gebruikt door de kernel. Dit deel van de mapping is aanwezig in de page table van elk process en is ook

identiek voor elk proces (wat er voor zorgt dat de kernel code steeds correct kan functioneren). Met andere woorden enkel de mapping van de onderste 3 Gbyte moet worden aangepast bij een proceswissel. In het user gedeelte zien we een stack, memory mapped, heap, twee data en een text segment.

Het stack segment wordt gebruikt om de user stack te implementeren en heeft een maximale grootte (bv. 8Mbyte). De hoeveelheid geheugen toegekend aan de stack zal tijdens de uitvoering van een process enkel toenemen, terwijl de inhoud op de stack natuurlijk zowel kan krimpen als groeien. Op zich lijkt 8Mbyte erg weinig voor alle lokale variabelen en parameters, maar in heel wat gevallen worden op de stack verwijzingen bijgehouden naar de eigenlijke data die terug te vinden is in het nog te bespreken heap segment.

Het memory mapped segment wordt veelal gebruikt om files in het geheugen in te laden om snelle access te verkrijgen tot deze files. Dit segment wordt onder andere gebruikt voor snelle access te krijgen tot *dynamic libraries* (bv. .dll files in Windows of .so files in Linux). Deze files vormen een bibliotheek met functies, die door meerdere applicaties gebruikt kunnen worden. Het heap segment wordt gebruikt voor geheugen dat wordt gealloceerd tijdens de uitvoering van een programma (bv. via de C-functie malloc). Het beheren van het geheugen dat deel uitmaakt van de heap is vaak erg complex. De resterende 3 segmenten worden gebruikt voor het inladen van de code en de globale variabelen van het process, waarbij het data segment de waardes bevat van de globale variabelen waaraan reeds een waarde is toegekend bij de start van het process en het BSS segment voorziet het nodige geheugen voor de overige globale variabelen.

De random offsets die we zien in Figuur 28 bevatten geen data en zijn enkel aanwezig om het leven van de *hacker* wat te bemoeilijken. Deze velden zorgen er namelijk voor dat de exacte startpositie van de verschillende segmenten niet precies gekend is, waardoor het iets moeilijker is om een user programma te *hacken*. Wanneer er geen gebruik wordt gemaakt van deze random offsets en er geen limiet is op de omvang van de stack, dan bekomt men de klassieke Linux geheugenlayout te zien in Figuur 29. De vaak voorkomende *segmentation fault* verwijst naar het feit dat een process iets tracht te doen met betrekking tot de bovenstaande segmenten dat niet is toegestaan, zoals een write operatie naar het text segment, een read naar het kernel segment wanneer het process in user mode is, het veroorzaken van een stack overflow, enz.

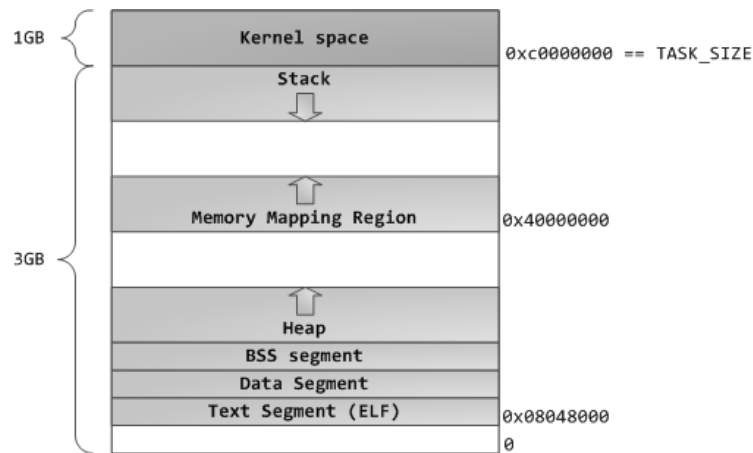


Fig. 29: Klassieke virtuele geheugenlayout in Linux

2 Page table organisatie

Het zou wenselijk zijn dat de page table van een proces steeds aanwezig is in het geheugen (wanneer het proces actief is). Tenslotte moet de processor frequent een adresvertaling doorvoeren (waarbij een virtuele page number wordt omgezet in een frame number) en is het belangrijk dat het frame number snel kan worden gevonden. Idealerweise wordt het startadres van de page table opgeslagen in een register van de processor, daarbij wordt dan het virtuele page number opgeteld om te komen tot de page table entry die het gezochte frame number bevat. Dit frame number vormt dan samen met de offset van het virtuele adres, het fysieke adres (zie Figuur 30).

Echter, zelfs wanneer de virtuele adresruimte niet heel erg groot is (bv. 4 GByte) en de pages niet al te klein (bv. 4096 bytes), bevat de page table erg veel entries (10^6 in ons voorbeeld). Men zou kunnen stellen dat heel wat processen veel kleiner zijn en daarom geen grote page table vereisen, maar zoals we eerder aangaven liggen de verschillende segmenten die deel uitmaken van de virtuele adresruimte erg verspreid over deze ruimte. Dit maakt dat ook processen die weinig geheugen vereisen niet zomaar een kleine page table kunnen hanteren.

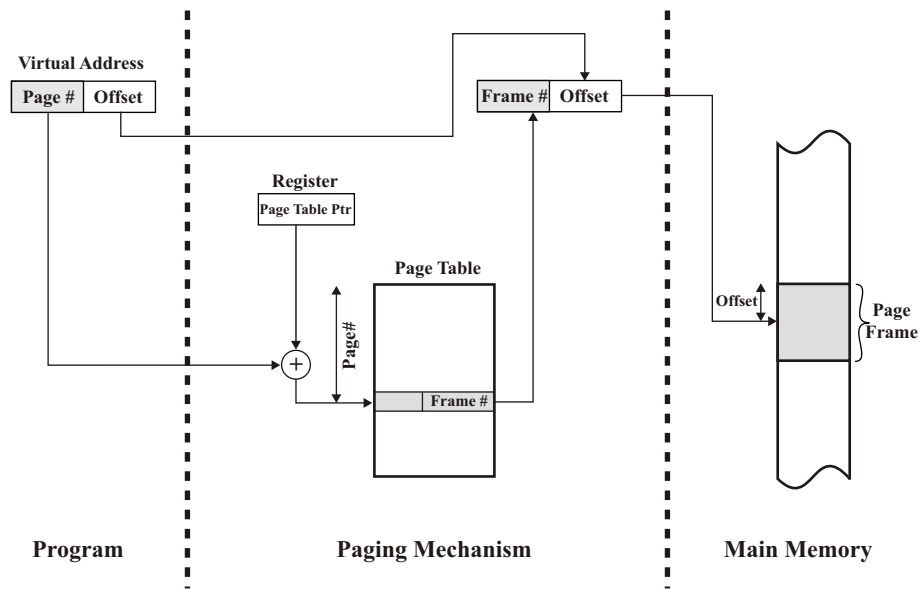


Fig. 30: Adresvertaling in een paging systeem [Stallings 03]

2.1 Multilevel page tables

Een vaak gebruikte oplossing om de grootte van de page table te reduceren, vormen de *multilevel* page tables. In geval van twee levels worden de eerste n bits van het virtuele page number gebruikt als index tot de level 1 page tabel (de zogenaamde *page directory*). Elke entry in deze table verwijst op zijn beurt wederom naar een page table die 2^m page entries bevat en waar we het frame number terug vinden. Met deze organisatie zal men de level 1 page table steeds in het geheugen opslaan (daar deze veel kleiner is) en slechts enkel de gebruikte level 2 tables inladen. Hierdoor zullen ongebruikte geheugengebieden geen geheugen meer opeisen.

Een logische en vaak gehanteerde keuze voor het aantal entries in een level 2 page table is deze zo te kiezen dat een level 2 page table precies één page groot is en gemapt wordt op een enkele frame (bv. x86 architecturen). In dit geval kunnen volgende scenario's zich voordoen bij het vertalen van een virtueel adres, afhankelijk van het feit of de vereiste level 2 page table al dan niet aanwezig is in het geheugen.

1. Het zou kunnen dat de vereiste level 2 page table aanwezig is in het geheugen, wat inhoudt dat we een frame number zullen aantreffen in

de level 1 page table entry. Wanneer we vervolgens de laatste m bits van het virtuele page number gebruiken als index tot deze level 2 page table, dan vinden we hierin het frame number terug dat samen met de offset van het virtuele adres het fysieke adres vormt. Indien het gezochte geheugenwoord niet in het geheugen aanwezig is, dan treedt er een page fault op en zal de virtuele page in het geheugen worden ingeladen en wordt de level 2 page table entry aangepast.

2. Wanneer de level 2 page table niet aanwezig is in het geheugen dan treedt er eveneens een page fault op en moeten we deze eerst in het geheugen inladen. Daarna kan er eventueel nog een tweede page fault optreden indien ook het geheugenwoord zelf niet aanwezig is in het fysieke geheugen.

Een nadeel van multilevel page tables is dat een enkele opzoeking van een frame number meerdere geheugenoproepen vereist. Dit is met name een nadeel in systemen die meer dan 4 Gbyte geheugen ondersteunen omdat deze vaak gebruik maken van een page table met 3 levels. De 32 bit x86 architecturen maken gebruik van 2 levels, waarbij het 20 bit page nummer is opgedeeld in twee stukken van 10 bits. Het nadeel van de meerdere geheugenoproepen wordt voor het grootste deel opgevangen door de TLB cache die we in Sectie 2.3 bespreken.

2.2 Inverted page tables

Het bijhouden van een page entry per mogelijke virtuele page (per proces) lijkt op zich wat omslachtig daar er veel meer virtuele dan fysieke pagina's zijn. Het voordeel van deze aanpak is natuurlijk wel dat we de frame numbers kunnen opzoeken in de table via *direct* access. Wanneer we enkel een table bijhouden met daarin een entry voor elke fysieke pagina (of frame), dan verkrijgen we een veel kleinere tabel.

Het opzoeken van een frame number is nu wel een stuk minder eenvoudig: we moeten alle frames doorlopen om te kijken of één ervan overeenstemt met de gevraagde pagina van een bepaald proces. Hiervoor zouden we, in principe, *associative* access kunnen gebruiken, maar dat is een erg dure oplossing. Sequentiële access is dan weer goedkoop maar te traag. Een binaire zoekmethode is wel efficiënt, maar vergt veel moeite bij het toevoegen van entries in de tabel (daar deze gesorteerd moet zijn).

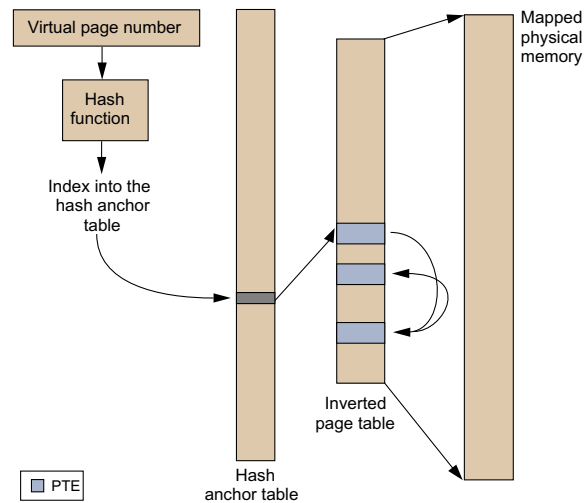


Fig. 31: Inverted page tables [Jacob 98]

De oplossing tot deze moeilijkheden is *hashing*, een methode die in heel wat systemen wordt gebruikt. De page/frame tabel wordt dan uitgebreid met een *hash anchor* table (zie Figuur 31). Wanneer we het frame number van een bepaalde pagina van een proces wensen op te zoeken, zullen we eerst een hash functie op dit nummer (en de proces id) loslaten. Dit kan eenvoudig door een deel van de bits te gebruiken als index tot de hash anchor tabel. Deze table zal op zijn beurt verwijzen naar een bepaalde entry in de page/frame tabel, waarvan de *index*, met wat geluk, het frame number bevat (indien de page in het geheugen aanwezig is). In Figuur 32 zien we dit in meer detail. De 2 bits in stap twee zijn afkomstig van het feit dat elke byte geadresseerd is en een page table entry 4 bytes groot is. De functie van de TLB cache wordt later duidelijk.

We moeten wat geluk hebben daar de hash functie meerdere proces id en virtuele page number paren afbeeldt op dezelfde table entry. Wanneer er meerdere van deze page numbers aanwezig zijn in het geheugen zullen ze een gelinkte lijst vormen, die we een keten of *chain* noemen (dit kan in de page table zoals op de figuur of via een conflict resolution table). Wanneer het fysieke geheugen is opgebouwd uit N frames, zal het aantal mogelijke uitkomsten M van de hash functie doorgaans iets groter zijn. Door deze waarde iets groter te nemen verkleinen we de kans dat er pages op dezelfde entry worden afgebeeld en maken we de ketens dus wat korter. Voor bepaalde

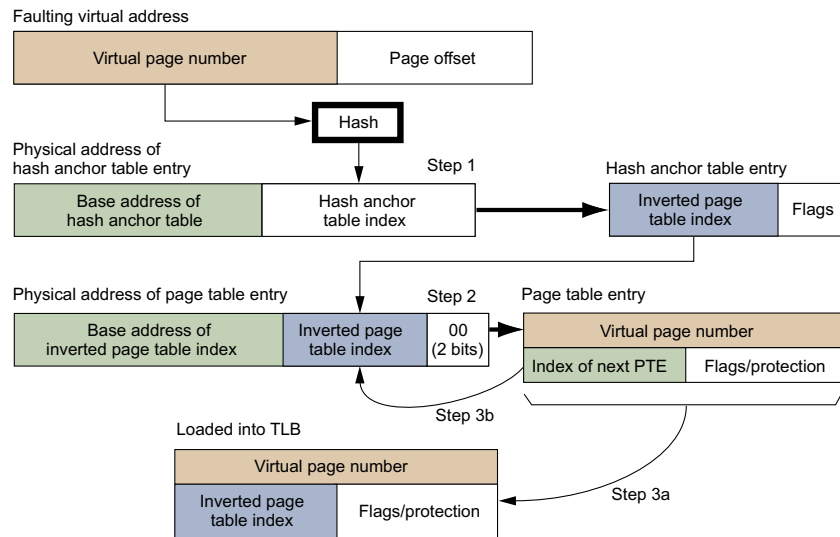


Fig. 32: Lookup algoritme voor inverted page tables [Jacob 98]

tables kan men aantonen dat deze aanpak doorgaans een resultaat oplevert na 1.5 entries in de page tabel (gegeven dat de page in het geheugen aanwezig is en met de nodige onderstellingen). Merk ook op dat de hash anchor table toelaat dat we $M > N$ kiezen zonder dat we de frame number expliciet hoeven op te slaan in de page table entry (de index of offset geeft namelijk het nummer aan).

Het sharen van pages is wel een moeilijkheid wanneer men inverted page tables hanteert. Elke frame wordt namelijk maar op één (of geen) page afgebeeld.

2.3 Translation lookaside buffer

Ondanks de voorgestelde datastructuren, blijft het vertalen van virtuele adressen in fysieke adressen een betrekkelijk trage operatie. Om ervoor te zorgen dat deze niet telkenmale moet plaatsvinden wanneer er een virtueel adres opduikt, zijn de meeste systemen uitgerust met een *translation lookaside buffer* (TLB) cache. Deze cache bevat echter geen user code of data, maar een (klein) deel van de page table entries. Steunend op het reference of locality principle, zal deze cache de meest recent voorkomende page table entries op een snelle manier beschikbaar stellen aan de processor. Wanneer

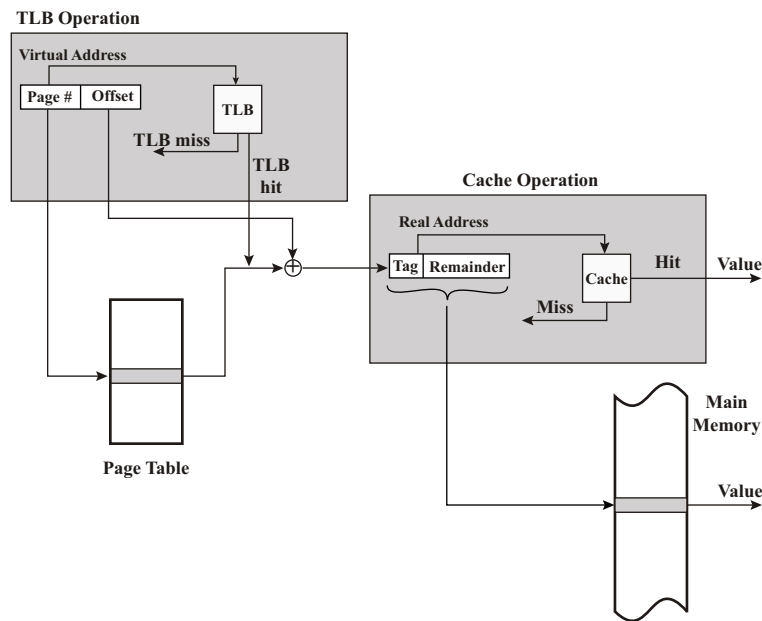


Fig. 33: TLB en cache operatie [Stallings 03]

dezelfde vertaling meermaals kort na elkaar plaatsvindt, zal er de eerste maal een vertaling plaats vinden via de page table en zal het resultaat opgeslagen worden in de TLB cache. Bij de volgende vertalingen wordt de cache succesvol geraadpleegd.

Het opzoeken van virtuele page numbers in de cache kan, zoals bij elke cache, gebeuren op een aantal manieren: via direct, associative of set-associative access. De grootte van de tag is afhankelijk van de gekozen methode. Wanneer er een cache miss optreedt, zal de processor terugvallen op de page table in het geheugen om de vertaling uit te voeren. De TLB cache bestaat meestal uit een beperkt aantal registers die deel uitmaken van de processor. Bijvoorbeeld, de 80486 architectuur maakt gebruik van een associatieve cache die bestaat uit 32 registers. De TLB cache bevat ook steeds page table entries die behoren tot het proces dat momenteel in uitvoering is. Dit impliceert dat elke proces wissel ook de TLB cache moet leegmaken (men spreekt dan van een *TLB flush*).

Het opvragen van een geheugenwoord in een computersysteem dat is uitgerust met een data en TLB cache, omvat een aantal stappen (zie Figuur 33). Eerst zal de virtuele page number van het woord in de TLB cache wor-

den opgezocht. Indien dit resulteert in een TLB miss, wordt de page table in het geheugen (of virtuele geheugen) aangesproken. Dit kan op zijn beurt mogelijk in een page fault resulteren, wat inhoudt dat het operating systeem de page, die het gevraagde woord bevat, eerst moet inladen. Wanneer de processor uiteindelijk over het fysieke adres kan beschikken, wordt dit opgezocht in de data cache. Resulteert dit in een cache miss, dan wordt het woord uit het geheugen opgevraagd. Indien er een miss optreedt in één van beide caches, zal ook deze aangepast worden, gebruikmakend van het cache replacement algoritme.

Net zoals andere caches wordt ook de TLB cache soms opgedeeld in een data (D-TLB) en instruction (I-TLB) cache. De page table entries naar pages met data en instructies worden dan respectievelijk in de D-TLB en I-TLB cache opgeslagen.

3 Virtueel geheugen: software support

Bij het vertalen van virtuele adressen wordt alles hardwarematig opgelost, zolang er geen page fault optreedt. Wanneer een gevraagde page zich niet in het fysieke geheugen bevindt, moet het operating systeem ervoor zorgen dat de page in kwestie in het geheugen geladen wordt. Hierbij moet het enkele belangrijke beslissingen nemen.

Ten eerste stelt zich de vraag of we enkel de gevraagde page, d.i. *demand paging*, of ineens enkele extra pages inladen, wat men aanduidt met de term *prepaging*. De meeste systemen gebruiken demand paging. Prepaging kan een interessant alternatief vormen wanneer we de interne werking van harddisks en andere storage media in gedachte houden.

Een tweede meer voorname beslissing is in welk page frame we de in te laden page zullen plaatsen. Wat neerkomt op het bepalen van de page die we tijdelijk in het secundaire geheugen opslaan, daar placement in een paging systeem geen issue is. Dit probleem kunnen we opdelen in een aantal subproblemen die we elk op zich zullen bespreken: (i) hoe is het aantal pages per proces vastgelegd, (ii) kan een page ook een frame van een ander proces innemen en (iii) eens we de set van kandidaat page frames hebben geïdentificeerd, hoe kiezen we hieruit de te vervangen page? De policy die antwoord biedt op de eerste twee vragen, duidt men aan met de term *resident set management*. Het page replacement algoritme gaat in op de derde vraag, dewelke we eerst van naderbij bekijken.

3.1 Page replacement algoritmen

Bemerkt dat het page replacement algoritme enkele overeenkomsten vertoont met het cache replacement algoritme. In beide gevallen hebben we dezelfde objectieven voor ogen: we willen de page/cache line vervangen zodat er na-dien zo weinig mogelijk page faults/cache misses optreden. We zien dan ook dat een aantal cache replacement algoritmen wederkeren in deze sectie. Beide problemen zijn in de praktijk weliswaar niet equivalent. Zo zal het cache replacement algoritme, in geval van een set-associatieve mapping, slechts moeten kiezen uit een erg kleine set van kandidaat cache lijnen (slechts 2, 4 of 8), terwijl het page replacement probleem mogelijk uit een grotere set een selectie zal maken.

Optimale algoritme: We starten met het bespreken van het optimale page (en ook cache) replacement algoritme. Dit algoritme zal gegarandeerd resulteren in het minimaal aantal page faults. Het vereist wel perfecte kennis van alle toekomstige gebeurtenissen, waardoor het niet implementeerbaar is. Dit algoritme wordt daardoor veelal gehanteerd als vergelijkings- of referentiepunt. In de laatste sectie van dit hoofdstuk zullen we de optimaliteit van dit algoritme ook formeel bewijzen. Het optimale algoritme, vaak het MIN algoritme genoemd, werd (net zoals virtueel geheugen) in de jaren 60 door Belady ontwikkeld en bestaat erin dat we van alle kandidaat pages, de page vervangen die het verste in de toekomst zal worden heropgevraagd.

Least Recently Used (LRU) algoritme: Het LRU algoritme, dat reeds kort besproken werd bij cache replacement, zal de minst recent gebruikte page vervangen. Het tracht het gedrag van het MIN algoritme na te doen, zonder over de nodige toekomstige informatie te beschikken. Terugdenkend aan het locality of reference principe, lijkt dit inderdaad een nuttige aanpak: recent opgevraagde pages worden met een grotere waarschijnlijkheid terug opgevraagd in de nabije toekomst. Voor de implementatie kunnen we een eenvoudige stack hanteren. Hoewel de performantie doorgaans erg goed is, zijn er een aantal ernstige nadelen aan verbonden:

- Telkens dat er een page hit optreedt, moet de gevraagde page naar de meest recent gebruikte (MRU) positie verplaatst worden. Mutual exclusion voor deze positie is dan ook noodzakelijk wanneer er vele threads/processen trachten de MRU positie aan te passen. Hiervoor

zal er vaak een lock bit gehanteerd worden, die aanleiding kan geven tot een sterke degradatie van de performantie daar alle processen om de beurt slechts tot de kritische zone toegang krijgen.

- Voor virtueel geheugen vergt de aanpassing van de MRU positie bij elke cache hit erg veel processing power.
- LRU zal pages die over korte tijdspannes veelvuldig worden opgevraagd en vervolgens een tijd ongebruikt zijn, makkelijk vervangen. De frequentie waarmee een page wordt opgevraagd, speelt dus een ondergeschikte rol.
- De LRU performantie is ook gevoelig voor (lange) sequenties van pages die slechts een enkele maal worden opgevraagd (dit noemt men een *scan*).

First In First Out (FIFO) algoritme: Het FIFO algoritme vervangt de page die reeds het langste aanwezig was in het geheugen. De gedachte hierbij is dat deze page mogelijk ook lang niet meer gebruikt is. Dit is natuurlijk vaak niet het geval, daar we frequent opgevraagde pagina's best zo lang mogelijk in het geheugen houden. FIFO heeft wel het voordeel dat de implementatie slechts uit een enkele pointer bestaat en dat cache hits geen update van de pointer vereisen. Een eigenaardig neveneffect van FIFO is wel dat, voor bepaalde page sequenties, de aanwezigheid van meer geheugen aanleiding kan geven tot meer page faults. Dit effect is gekend onder de naam van Belady's anomalie (zie Figuur 34).

Clock algoritme Het Clock algoritme werd in 1968 geïntroduceerd door Frank Corbató. Als algoritme situeert het zich tussen FIFO en LRU. Naast een enkele *next frame* pointer, wordt er een enkele bit, de *use bit*, bijgehouden per page frame. Alle frames worden in een circulaire buffer geplaatst. De use bit wordt bij het inladen van een nieuwe page op 1 gezet, alsook telkenmale wanneer er een page hit optreedt. Wanneer er een page fault plaatsvindt, zal de next frame pointer de buffer aflopen tot er een frame gevonden wordt waarvan de use bit op 0 staat. Wanneer de pointer wijst naar een frame waarvan de bit 1 is, wordt deze op 0 gezet en gaat de pointer over naar de volgende frame.

Het clock algoritme vereist ook betrekkelijk weinig hardware ondersteuning. Het volstaat dat de hardware een set van reference bits ondersteunt

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	0	0	0	4	4	4	4	4	4
Frame 2		2	2	2	3	3	3	3	3	1	1	1
Frame 3			1	1	1	2	2	2	2	2	0	0
Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	3	3	3	4	4	4	4	0	0
Frame 2		2	2	2	2	2	2	3	3	3	3	4
Frame 3			1	1	1	1	1	1	2	2	2	2
Frame 4				0	0	0	0	0	0	1	1	1

(red indicates page fault)

Fig. 34: Belady's anomalie: bij FIFO kan meer geheugen tot meer page faults leiden

die aangeven of een bepaalde page is opgevraagd. Bij elke page hit zal de hardware de overeenkomstige bit op één zetten (indien deze nog niet gelijk aan één was). Bij een page miss, zal het clock algoritme vervolgens gebruik maken van deze reference bits.

Pages die veelvuldig worden opgevraagd kunnen op deze wijze lang in het geheugen blijven zonder te moeten terugvallen op het LRU algoritme. Een interessante variant van het Clock algoritme maakt gebruik van de *modified* of *update* bit die wordt bijgehouden voor elke page die aanwezig is in het geheugen. Eerst wordt de circulaire buffer doorlopen om een frame te vinden met beide bits gelijk aan 0. Vinden we zulk een frame, dan wordt deze gekozen. Anders wordt de buffer nogmaals doorlopen op zoek naar een frame met de use bit op 0, maar met de modified bit op 1. Bij deze tweede rotatie wordt de use bit op 0 gezet wanneer deze op 1 stond. Is er na de tweede rotatie nog geen frame gevonden, dan worden de eerste twee stappen/rotaties nogmaals uitgevoerd. Doordat alle use bits ondertussen op nul staan zal dit steeds resulteren in de selectie van een frame. Deze variant tracht eerst niet gewijzigde frames te vervangen, wat de efficiëntie vaak ten goede komt, aangezien het frame niet eerst moet worden weggeschreven.

Ondertussen zijn er tal van varianten op het Clock algoritme ontwikkeld, zoals het gebruik van een tweede wijzer (zie slides), en deze worden gebruikt in tal van (hedendaagse) systemen. In Figuur 35 wordt de evolutie van elk van de vier voorgaande algoritmen nogmaals gedemonstreerd aan de hand van een voorbeeld.

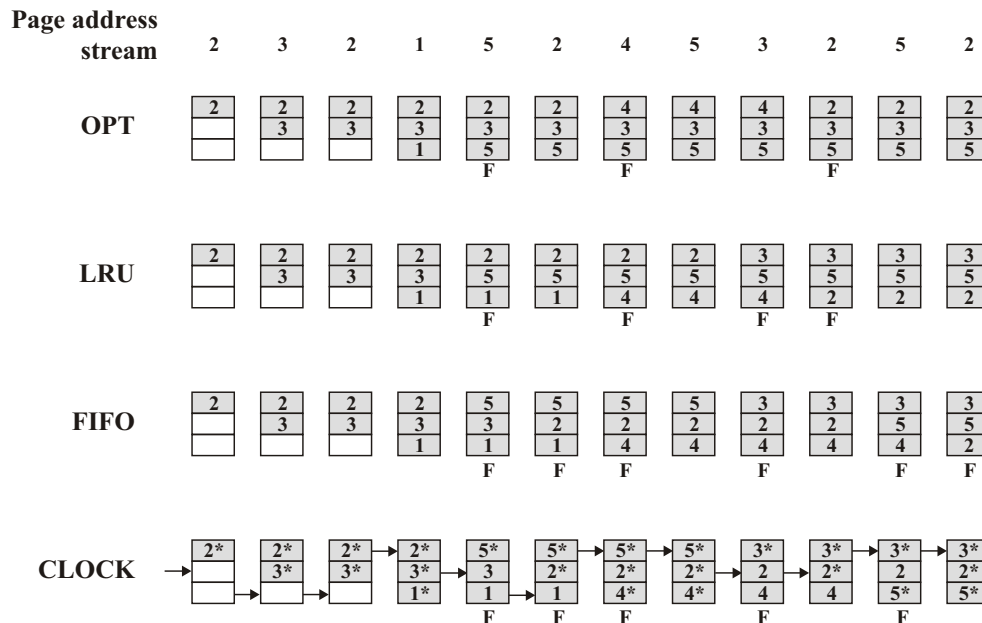


Fig. 35: Gedrag van enkele page replacement algoritmen [Stallings 03]

Page buffering and segmented FIFO: Page buffering is eveneens een replacement algoritme dat een tussenoplossing aanbiedt voor FIFO en LRU. We leggen dit uit aan de hand van het Segmented FIFO (SFIFO) algoritme gepubliceerd door Turner en Levy in 1981. SFIFO zal twee lijsten $L1$ en $L2$ van frames bijhouden. Wanneer een nieuwe page in het geheugen wordt gelezen, zal deze vooraan in de $L1$ buffer worden geplaatst. Deze buffer volgt een eenvoudig FIFO regime. Het frame achteraan in de $L1$ buffer wordt vervolgens vooraan de $L2$ buffer toegevoegd. Het frame dat achteraan in de $L2$ buffer stond, zal de nieuwe page bevatten. Wanneer er een page hit optreedt in de $L1$ buffer, gebeurt er niets. Echter, heeft de hit plaats in de $L2$ buffer, dan wordt deze frame vooraan in de $L1$ buffer toegevoegd. Zonder $L2$ verkrijgen we het FIFO systeem, indien de $L1$ buffer slechts één frame bevat dan is het algoritme equivalent aan LRU.

De kracht van dit algoritme is dat de performantie (voor een goede keuze voor de omvang van $L1$ en $L2$) dicht aanleunt bij LRU, terwijl de hoeveelheid werk bij een page hit laag gehouden wordt. Pages die regelmatig worden opgevraagd worden van tijd tot tijd wel naar de $L2$ lijst verwezen, maar indien ze snel genoeg nogmaals gerefereerd worden, belanden ze wederom in

de *L1* lijst zonder ooit uit het geheugen te zijn verwijderd. De *L2* buffer vangt in zekere zin foutieve beslissingen van het FIFO algoritme op.

Men kan dit idee ook uitwerken in combinatie met andere replacement algoritmen. Men spreekt dan van *page buffering*. In deze context wordt de *L2* lijst regelmatig aangeduid met de term de *free page list*. Het idee hierachter is dat de *L2* lijst vaak ook enkele frames bevat die wel behoren tot het *resident memory* van een proces, dat zijn de page frames die behoren tot een proces, maar die nog niet ingenomen zijn door een page. Een meer geavanceerde versie van page buffering bestaat erin ook een *L3* lijst bij te houden. Pages die uit de *L1* lijst verwijderd worden, zullen dan plaatsnemen in de *L2* of *L3* buffer afhankelijk van het feit of ze een update hebben ondergaan of niet. Page buffering wordt ook als vangnet gehanteerd in de context van resident set management met variabele allocatie (zie verder).

3.2 Resident set management

Wanneer een page fault optreedt, wordt het replacement algoritme geactiveerd om een page frame te selecteren uit de kandidaat frames. Welke frames in aanmerking komen, bepaalt de *resident set* policy. Vooreerst moeten we bemerken dat een deel van de frames in elk geval nooit in aanmerking komen. Deze frames, die *geloocked* zijn, behoren bijvoorbeeld tot het kernel proces. Voor de page frames die niet in het geheugen vergrendeld zijn, bestaan er een aantal mogelijkheden.

Een eerste onderscheid bestaat er tussen systemen die een vast of een variabel aantal page frames alloceren per proces (d.i. *fixed* of *variable* allocation). Daarnaast zullen sommige systemen toelaten dat een page van het ene proces een frame van een ander proces inneemt. Wanneer pages enkel eigen page frames kunnen opeisen, spreken we van een lokale replacement policy, anders van een globale policy. Een operating systeem dat een lokale policy combineert met een variabele allocatie, zal regelmatig een evaluatie uitvoeren die het aantal page frames van een proces mogelijk verhoogt of verlaagt. Wanneer het gaat om extra frames, zullen deze vrije frames gaandeweg ingenomen worden. Zijn er geen van deze frames meer beschikbaar bij het optreden van een page fault, dan selecteert het replacement algoritme een page uit alle frames behorende tot het proces dat de fault veroorzaakte.

Variabele allocatie is ook erg nuttig wanneer er weinig processen actief zijn. Tenslotte is er dan geen reden om de resident set van een proces te beperken. Indien er echter veel processen actief worden, stelt zich de vraag

hoe we het aantal page frames dynamisch gaan bepalen. Een sleutelrol bij het beantwoorden van deze vraag, speelt de *active working set* $W(t, \Delta)$ van een proces. Deze set bevat, op tijd t , alle pages die door een proces zijn opgevraagd gedurende de laatste Δ seconden dat het actief was (men hanteert hier soms het begrip virtuele tijd). De evolutie van de omvang van deze set wordt gekenmerkt door periodes van stabiliteit die afgewisseld worden met overgangperiodes. Gedurende een overgang zal de active set vaak sterk toenemen.

Indien we de actieve set van alle processen monitoren, dan kunnen we regelmatig alle pages die niet meer behoren tot de actieve set van een proces, verwijderen. De meeste systemen verwijderen deze pages niet echt, maar hanteren een page buffering mechanisme om eventuele foutieve beslissingen op te vangen. Deze aanpak lijkt aanlokkelijk, maar behelst enkele moeilijkheden. Net zoals LRU geeft ook hier het observeren van het verleden geen garanties voor wat er komen gaat. Daarnaast vergt het tracken van de set $W(t, \Delta)$ heel wat moeite. In de praktijk zal men daarom eerder de page fault frequentie (of rate) observeren. Veel page faults geven aan dat de active working set groter is dan het aantal beschikbare frames.

Een mogelijke aanpak is om een *use bit* te definiëren (zoals bij het Clock algoritme) die we op 1 zetten wanneer een actieve page wordt opgevraagd. Bij elke page fault wordt er vervolgens gekeken wanneer de vorige fault optrad. Was dit erg recent, dan wijst dit op een onderschatting van de actieve set en wordt de nieuwe page toegevoegd aan de set van beschikbare pages. Vond de laatste fault langer geleden plaats, dan lijkt de working set voldoende groot ingeschat en zullen alle pages waarvoor de use bit nog op 0 staat eerst verwijderd worden. In dit geval zetten we ook alle use bits terug op 0.

Deze aanpak lijkt effectief, er is echter een groot nadeel: in geval van een overgangperiode zal het aantal gebruikte pages sterk groeien. Dit terwijl er ook vele pages in de set zijn die niet meer worden gebruikt. Dit komt de stabiliteit van het systeem niet ten goede. Om dit te voorkomen kan men de use bits eveneens evalueren na elke Q page faults (in combinatie met twee thresholds om een minimale en maximale tijd tussen de evaluaties te garanderen).

Zoals aangegeven zal dit mechanisme doorgaans ook page buffering hanteren. Wijzigingen in de pages die uit de actieve set verwijderd worden, moeten ook, voor ze effectief uit het main geheugen worden gewist, worden doorgevoerd in het secundaire geheugen. Dit kan page per page gebeuren, net voor ze verwijderd worden (d.i. *demand cleaning*) of in batches (d.i. *pre-*

cleaning). De eerstgenoemde methode is minder efficiënt, maar schrijft enkel de noodzakelijke data naar disk. Een populaire methode bestaat erin de inhoud van de page buffer regelmatig naar disk te schrijven.

4 Optimaliteit van het MIN algoritme: een bewijs

In deze sectie geven we een elementair bewijs van de optimaliteit van het MIN algoritme. Dit bewijs werd onlangs, in april 2007, door Benjamin Van Roy in het tijdschrift *Information Processing Letters* gepubliceerd. We beschouwen een reeks van T page requests, $\omega_0, \dots, \omega_{T-1}$ en tonen aan dat het MIN algoritme het aantal page faults minimaliseert. Laat Ω de set zijn van alle pages en $S_t \subseteq \Omega$ de set van pages in het geheugen op tijd t , dus net voor request ω_t wordt geplaatst. Indien $\omega_t \in S_t$, dan zal $S_{t+1} = S_t$, anders wordt ω_t aan S_t toegevoegd en moet er een andere page uit het geheugen verwijderd worden. De betrachting is om $\sum_{t=0}^{T-1} \mathbf{1}(\omega_t \in S_t)$ te maximaliseren, waarbij $\mathbf{1}(A) = 1$ indien A waar is, anders is $\mathbf{1}(A) = 0$.

We starten met $J_t(S_t)$ te definiëren als het maximaal aantal hits dat nog kan optreden vanaf tijdstip t . Bijgevolg,

$$J_t(S_t) = \mathbf{1}(\omega_t \in S_t) + \max_{S_{t+1} \in U_t(S_t)} J_{t+1}(S_{t+1}), \quad (2)$$

voor $t < T$ en $S_t \subseteq \Omega$, waarbij $U_t(S_t)$ alle mogelijke deelverzamelingen van Ω bevat die op tijd $t+1$ in het geheugen kunnen zitten (gegeven dat S_t op tijd t in het geheugen zat). Kortom,

$$U_t(S_t) = \{S \subseteq S_t \cup \omega_t : \omega_t \in S, |S| = |S_t|\}.$$

Verder noteren we het volgende tijdstip waarop page ω opnieuw opgevraagd wordt als $\tau_t(\omega) = \min\{t \leq z < T : \omega_z = \omega\}$. Wanneer er een page fault optreedt op tijd t , zal het MIN algoritme de page ω vervangen waarvoor $\tau_t(\omega)$ het grootste is (van alle aanwezige pages in het geheugen). Indien een page ω niet meer opgevraagd wordt na tijd t dan is $\tau_t(\omega) = \infty$.

Om de optimaliteit van het MIN algoritme te bewijzen beschouwen we alle bijectieve afbeeldingen h tussen twee geheugentoestanden S en S' (met $|S| = |S'|$). We definiëren vervolgens d_t als de afstand tussen S en S' :

$$d_t(S, S') = \min_{h \text{ bijectie}} |\{\omega \in S \mid \tau_t(\omega) > \tau_t(h(\omega))\}|.$$

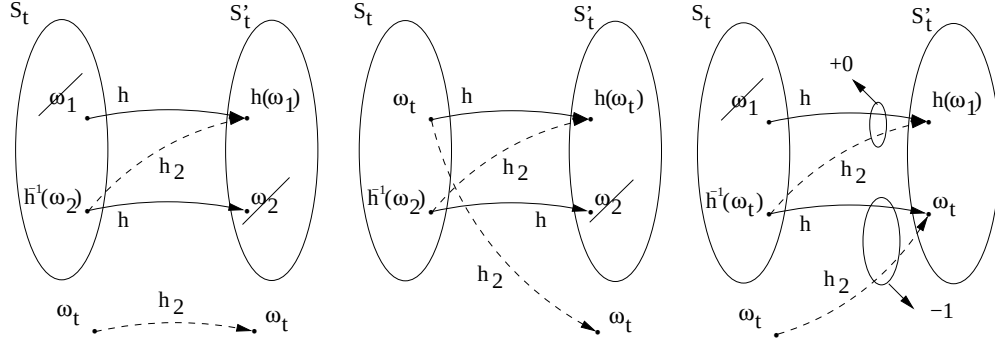


Fig. 36: Grafische weergave van de ongelijkheden

Laten we volgend voorbeeld bekijken: $S = \{\omega_a, \omega_b, \omega_c\}$ en $S' = \{\omega_a, \omega_d, \omega_e\}$ met $\tau_t(\omega_a) = 8$, $\tau_t(\omega_b) = 13$, $\tau_t(\omega_c) = 6$, $\tau_t(\omega_d) = 11$, $\tau_t(\omega_e) = 4$. Dan zal volgende bijectie h :

$$h(\omega) = \begin{cases} \omega_a & \text{voor } \omega = \omega_a, \\ \omega_d & \text{voor } \omega = \omega_c, \\ \omega_e & \text{voor } \omega = \omega_b, \end{cases}$$

ervoor zorgen dat $d_t(S, S')$ gelijk is aan 1.

We bewijzen nu volgend lemma: $J_t(S') - J_t(S) \leq d_t(S, S')$ en dit voor $t \leq T$ en $S, S' \subseteq \Omega$ met $|S| = |S'|$. We hanteren achterwaartse inductie op t . $J_T(S) = J_T(S') = d_T(S, S') = 0$. Laat $S_t = S$, $S'_t = S'$ en laat $S_{t+1} \in U_t(S_t)$ gekozen door het MIN algoritme en $S'_{t+1} \in U_t(S'_t)$ door een optimaal algoritme. Stel dat het resultaat geldt voor tijd $t+1$: $J_{t+1}(S'_{t+1}) - J_{t+1}(S_{t+1}) \leq d_{t+1}(S_{t+1}, S'_{t+1})$. Dan vinden we volgende ongelijkheden

$$d_{t+1}(S_{t+1}, S'_{t+1}) = \begin{cases} = d_t(S_t, S'_t) & \text{voor } \omega_t \in S_t \cap S'_t, \\ \leq d_t(S_t, S'_t) & \text{voor } \omega_t \notin S_t \cup S'_t, \\ \leq d_t(S_t, S'_t) + 1 & \text{voor } \omega_t \in S_t \setminus S'_t, \\ \leq d_t(S_t, S'_t) - 1 & \text{voor } \omega_t \in S'_t \setminus S_t, \end{cases}$$

De gelijkheid is duidelijk daar $S_t = S_{t+1}$ en $S'_t = S'_{t+1}$ (en de h die $d_t(S_t, S'_t)$ minimaliseert kan steeds zo gekozen worden dat $h(\omega_t) = \omega_t$, door $h^{-1}(\omega_t)$ op $h(\omega_t)$ af te beelden mocht dit niet het geval zijn). Voor de overige drie gevallen verwijzen we eveneens naar Figuur 36. In het tweede geval maakt ω_t geen deel uit van beide geheugentoestanden. Stel dat $S_{t+1} = (S_t \cup \omega_t) \setminus \omega_1$ en $S'_{t+1} = (S'_t \cup \omega_t) \setminus \omega_2$. Laat h de bijectie zijn die een minimum realiseert

voor $d_t(S_t, S'_t)$. Dan definiëren we h_2 identiek aan h behalve dat $h_2(\omega_t) = \omega_t$ en $h_2(h^{-1}(\omega_2)) = h(\omega_1)$, dan is $d_t(S_t, S'_t)$ maximaal met één toegenomen, namelijk wanneer

$$\begin{aligned}\tau_t(\omega_1) &\leq \tau_t(h(\omega_1)), \\ \tau_t(h^{-1}(\omega_2)) &\leq \tau_t(\omega_2), \\ \tau_{t+1}(h^{-1}(\omega_2)) &> \tau_{t+1}(h(\omega_1)).\end{aligned}$$

Echter, gezien het MIN algoritme ω_1 kiest, moet $\tau_t(\omega_1) \geq \tau_t(h^{-1}(\omega_2))$, wat samen met de derde vergelijking een tegenspraak oplevert op de eerste vergelijking. Dit bewijst de eerste ongelijkheid.

In het derde geval maakt ω_t deel uit van S_t maar niet van S'_t . Als we h_2 dan definiëren zoals h , maar met $h_2(\omega_t) = \omega_t$ en $h_2(h^{-1}(\omega_2)) = h(\omega_t)$ dan neemt $d_t(S_t, S'_t)$ hoogstens met één toe. In het laatste geval maakt ω_t wel deel uit van S'_t , maar niet van S_t . Bijgevolg moet $\tau_t(h^{-1}(\omega_t)) > \tau_t(\omega_t) = t$. We kiezen nu h_2 identiek aan h , behalve dat $h_2(\omega_t) = \omega_t$ en $h_2(h^{-1}(\omega_t)) = h(\omega_1)$. Omwille van het MIN algoritme moet $\tau_t(\omega_1) \geq \tau_t(h^{-1}(\omega_t))$, dus indien $\tau_t(\omega_1) \leq \tau_t(h(\omega_1))$ dan geldt eveneens dat $\tau_t(h^{-1}(\omega_t)) \leq \tau_t(h(\omega_1))$, waardoor $d_t(S_t, S'_t)$ in totaal met minstens één afneemt.

Hieruit volgt:

$$d_{t+1}(S_{t+1}, S'_{t+1}) \leq d_t(S_t, S'_t) + \mathbf{1}(\omega_t \in S_t) - \mathbf{1}(\omega_t \in S'_t).$$

Met behulp van deze ongelijkheid, de inductie hypothese en vergelijking (2) verkrijgen we dan

$$\begin{aligned}J_t(S'_t) &= \mathbf{1}(\omega_t \in S'_t) + J_{t+1}(S'_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S'_t) + d_{t+1}(S_{t+1}, S'_{t+1}) + J_{t+1}(S_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S_t) + d_t(S_t, S'_t) + J_{t+1}(S_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S_t) + \max_{\tilde{S} \in U_t(S_t)} J_{t+1}(\tilde{S}) + d_t(S_t, S'_t) \\ &= J_t(S_t) + d_t(S_t, S'_t),\end{aligned}$$

waarbij de eerste gelijkheid volgt uit de optimaliteit.

Gegeven S_t , laat S_{t+1} de toestand zijn die we verkrijgen door het MIN algoritme te hanteren en S'_{t+1} de toestand bereikt door een optimaal algoritme (eveneens vanuit S_t !). Aangezien S_{t+1} de afstand $d_{t+1}(\tilde{S}_{t+1}, S'_{t+1})$ minimaliseert over alle $\tilde{S}_{t+1} \in U_t(S_t)$, waartoe ook S'_{t+1} behoort is $d_{t+1}(S_{t+1}, S'_{t+1}) = 0$. Aan de andere kant hebben we $0 \leq J_{t+1}(S'_{t+1}) - J_{t+1}(S_{t+1}) \leq d_{t+1}(S_{t+1}, S'_{t+1})$, of nog, $J_{t+1}(S_{t+1}) = J_{t+1}(S'_{t+1})$, wat de optimaliteit bewijst.

UNIPROCESSOR SCHEDULING

In dit hoofdstuk gaan we wat dieper in op de schedulingbeslissingen die het operating systeem in een multiprogrammeer omgeving moet nemen. Zoals reeds eerder aangegeven, bevindt elk proces zich in een bepaalde toestand: running, ready, blocked, suspended-ready, etc. Het scheduling mechanisme zal in sterke mate bepalen hoe de toestand van een bepaald proces evolueert. Vaak worden deze beslissingen opgedeeld in drie deelproblemen: *long-term*, *medium-term* en *short-term* scheduling. De nadruk van dit hoofdstuk ligt op de derde vorm van scheduling.

Long-term en medium-term scheduling

De invloed van long-term scheduling situeert zich voornamelijk in het bepalen van het aantal actieve processen. Wanneer een nieuw proces, waarvan de toestand *new* is, zich aandient, zal het inactief blijven tot de long-term scheduler besluit het aan de lijst van actieve processen toe te voegen. Wanneer de activatie plaatsgrijpt, verandert de toestand van het proces in *ready* of *ready-suspend*.

Alle nieuwe processen vormen samen een wachtlijn en worden bediend door de long-term scheduler. Deze zal twee types van beslissingen moeten nemen:

1. Wanneer kan een nieuw proces worden geactiveerd?
2. Welk proces kan het best worden geactiveerd uit alle nieuwe, wachtende processen?

De gewenste mate van multiprogrammering speelt in het eerst geval een grote rol. Meer actieve processen geeft ook aanleiding tot meer processen die ready zijn, wat de efficiëntie van de processor ten goede komt. Anderzijds, meer actieve processen verhoogt de uitvoertijd van de actieve processen (en vergt meer virtueel geheugen). De long-term scheduler zal de idle time van de processor nauwlettend volgen en processen activeren wanneer de idle time te hoog oploopt.

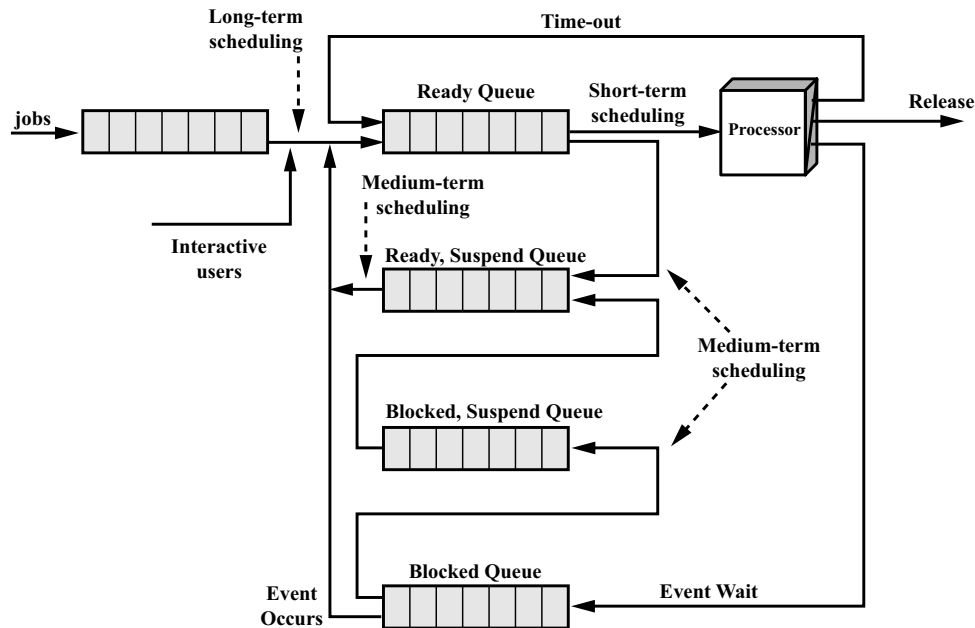


Fig. 37: Invloed van scheduling op de processtoestand [Stallings 03]

Voor de keuze van het te activeren proces kan ook de aard van het proces een doorslaggevende rol spelen. Processen worden namelijk vaak opgedeeld in *processor-bound* en *I/O-bound* processen. Deze laatste set zijn diegene die veelvuldig gebruik maken van I/O operaties, terwijl *processor-bound* processen veel rekenwerk vergen en minder nood hebben aan I/O operaties. Te veel I/O-bound processen kan leiden tot een situatie waarin alle processen geblokkeerd zijn. Enkel *processor-bound* processen selecteren leidt tot een inefficiënt gebruik van de I/O devices.

Verder zullen interactieve processen vaak onmiddellijk tot de lijst van de actieve processen mogen toetreden, tot een bepaalde threshold bereikt wordt. Dienen er zich nog interactieve processen aan, dan worden deze afgebroken en wordt de gebruiker ingelicht.

Medium-term scheduling treedt iets vaker op en bepaalt welke processen (gedeeltelijk) in het fysieke geheugen aanwezig zijn. Deze vorm van scheduling leunt dus nauw aan bij het virtueel geheugenmanagement, of meer precies, bij het resident set management.

1 Non-real-time short-term scheduling

De short-term scheduling functie geeft aan welke van de ready processen over de processor mag beschikken, of nog, running is. De functionaliteit van de verschillende vormen van scheduling zien we ook terug in Figuur 37. Voor we verscheidene scheduling algoritmen kunnen vergelijken, is het belangrijk om even na te denken hoe we bepalen of zulk een algoritme al dan niet goed presteert.

1.1 Performantie maten

Een eerste criterium is de (gemiddelde) *turnaround time* van een proces. Deze geeft aan hoelang het duurt eer een proces volledig is uitgevoerd, startende vanaf het moment dat het werd opgestart. Deze tijd bevat dus eveneens de tijd dat een proces blocked is of moet wachten in de ready toestand. De *response time* is hiermee sterk gerelateerd en geeft de tijd aan tot een gebruiker feedback ontvangt dat zijn proces lopende is. Deze laatste maat is voornamelijk van belang vanuit het oogpunt van de gebruikers in interactieve systemen en in mindere mate voor het systeem zelf.

Naast een lage response en turnaround time, is het ook wenselijk dat de processor zo goed als altijd aan het werk is, zodat er geen capaciteit verloren gaat. Men hanteert hierbij de term *processor utilization*. Een hoge utilization geeft vaak aanleiding tot een grotere turnaround time (of wachttijd). Een enkel algoritme kan dan ook nooit alle performantiematen gelijktijdig optimaliseren. Verder is ook de *fairness* en *predictability* van enige waarde. De eerste geeft aan in welke mate de capaciteit van de resources eerlijk verdeeld is over de verschillende processen. Over de definitie van wat eerlijk is, zijn er in het verleden al heel wat discussies geweest en dat zal niet veranderen in de toekomst. Een vaak gehanteerde waarde met betrekking tot fairness is de genormaliseerde turnaround time. Dit is de turnaround time van een proces gedeeld door de tijd dat het proces effectief heeft kunnen beschikken over de processor. De predictability geeft tenslotte aan hoe gevoelig de uitvoertijd van een proces is in functie van de aanwezigheid van andere processen. Een grote variatie van de uitvoertijd is niet wenselijk.

1.2 Notatie van scheduling modellen

Voor we de eigenlijke bespreking van de scheduling algoritmen aanvangen, voeren we een universele notatie in voor een scheduling probleem: $\alpha|\beta|\gamma$, waarbij α de machine omgeving voorstelt, β de randvoorwaarden en γ de functie die we wensen te minimaliseren. Wanneer een algoritme optimaal is voor een bepaald scheduling model, dan zullen we het exacte model waarvoor de optimaliteit opgaat op deze manier compact kunnen noteren.

In dit hoofdstuk is $\alpha = 1$, wat aangeeft dat de machine over precies één processor beschikt. Wanneer we $\alpha = Pm$ noteren, dan zijn er m identieke processoren aanwezig. Elke taak kan dus op elk van deze m processoren worden uitgevoerd en de looptijd is dezelfde op elke machine. De mogelijkheden voor β die van belang zijn in deze cursus zijn: r_j , $pmtn$ en $prec$. r_j geeft aan dat de taken aankomsttijden hebben die verschillen van nul. In dit geval geeft r_j de aankomsttijd van taak j aan. Een taak kan nooit gestart worden voor ze is aangekomen. $pmtn$ geeft aan of taken onderbroken mogen worden, terwijl de $prec$ aangeeft dat sommige taken pas uitgevoerd kunnen worden indien bepaalde andere taken eerst zijn uitgevoerd. Indien, we r_j , $pmtn$ of $prec$ niet vermelden dan wordt er veronderstelt dat deze niet van toepassing zijn.

Noteer C_j als de completion time van taak j (onder een bepaalde schedulingdiscipline). Bij onze bespreking van non-real-time algoritmen, zijn er voor γ , de functie die we wensen te minimaliseren, twee mogelijkheden: $\gamma = \sum C_j$ of $\gamma = \sum w_j C_j$. In het eerste geval wensen we de som van de completion times zo klein mogelijk te hebben, in het tweede geval de gewogen som.

Bijvoorbeeld, $1||\sum C_j$ geeft aan dat de taken uitgevoerd worden op een machine met één processor, dat we de som van de completion times willen minimaliseren en dat de taken allemaal uitgevoerd kunnen worden vanaf tijd nul. Verder is er geen taak onderbreking toegelaten en zijn er geen taken die voor bepaalde andere taken moeten worden uitgevoerd. Mogen we een taak toch onderbreken dan noteren we dit als $1|pmtn|\sum C_j$, enz.

1.3 Scheduling algoritmen

We starten met een aantal van de meest gekende en bestudeerde scheduling algoritmen. We maken gedurende dit overzicht ook onderscheid tussen systemen waarin een proces/taak kan onderbroken worden, genaamd *preemptive* systemen, en deze waarin een taak onafgebroken wordt uitgevoerd (zoals in

Proces	Arrival time	Service time (T_s)	Start time	Completion time	Turnaround time (T_q)	T_q/T_s
A	0	1	0	1	1	1
B	1	100	1	101	100	1
C	2	1	101	102	100	100
D	3	100	102	202	199	1.99
Mean					100	26

Fig. 38: FIFO discipline voorbeeld [Fink 88]

sommige productiesystemen), of nog niet-preemptive systemen.

First-Come-First-Served (FCFS): De eenvoudigste discipline bestaat erin de processen uit de ready queue te bedienen in de volgorde waarin ze in de queue geplaatst zijn. Het proces dat reeds het langst ready was, wordt in dit geval eerst gekozen.

Het grote nadeel van deze aanpak is dat kleine taken, die erg snel uitgevoerd kunnen worden, vaak belanden achter grote processen die langdurig van de processor gebruik maken (zie Figuur 38). Met andere woorden, de gemiddelde turnaround tijd is vaak ver van optimaal, terwijl het nog erger gesteld is met de genormaliseerde turnaround time.

Als we dan ook nog in rekening brengen dat we processen kunnen opdelen in processor- en I/O-bound processen, dan merken we dat I/O-bound processen sterk benadeeld worden. Dit komt doordat ze telkens na het uitvoeren van een I/O operatie opnieuw achteraan moeten plaatsnemen in de ready queue.

FCFS, ook wel eens First-In-First-Out (FIFO) genoemd, is voornamelijk interessant in combinatie met prioriteitsmechanismen. Een onderwerp waar we verder op terugkomen.

Round Robin (RR): Een manier om het dominerende gedrag van langdurige taken tegen te gaan in een FIFO systeem, bestaat erin om een maximale tijd in te stellen dat een enkel proces onafgebroken kan beschikken over de processor. Na deze tijd, die vaak aangeduid wordt met de term *quantum*, wordt de taak in uitvoering onderbroken en moet het proces opnieuw achteraan plaatsnemen in de ready queue. Wanneer we in Figuur 38 een RR discipline hanteren met een quantum van 5, dan zien we dat proces C plots

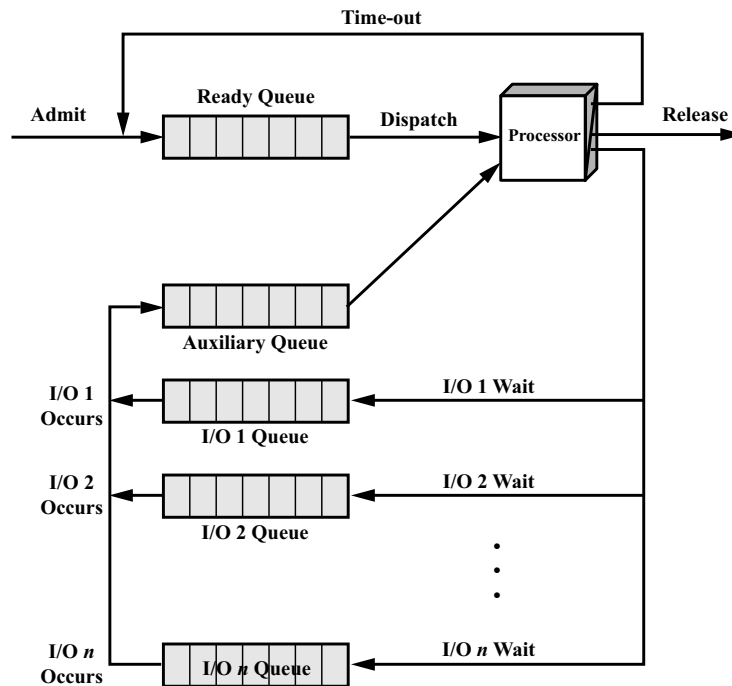


Fig. 39: Fairness tussen I/O- en processor-bound processen bij RR [Stallings 03]

een veel lagere wachttijd realiseert.

Een belangrijke keuze bij de RR discipline is de lengte van het quantum q . Een zeer kort quantum zal, in theorie, steeds aanleiding geven tot een lagere gemiddelde uitvoertijd. Echter, in de praktijk moet men ook de tijd nodig om twee processen te wisselen in rekening brengen. Een te kleine waarde q voor het quantum, impliceert dat de processor meer bezig is met het wisselen van processen, dan met het uitvoeren ervan. Het limietgeval waarbij het quantum q oneindig klein wordt, is gekend onder de naam *processor sharing*. Dit model is niet praktisch realiseerbaar, maar een evaluatie ervan kan belangrijke implicaties hebben voor systemen met betrekkelijk kleine quanta en is wiskundig vaak minder complex.

Hoewel RR het probleem van de grote genormaliseerde turnaround tijden deels oplost, worden de I/O-bound processen nog steeds nadelig behandeld. Vaak zal er namelijk al een I/O operatie plaatsvinden voordat het quantum

van het proces is afgelopen. Processor-bound processen van hun kant gebruiken hun quantum wel telkens volledig. Een mogelijke aanpak om deze oneerlijkheid tegen te gaan, zien we in Figuur 39. Processen die opnieuw ready worden na op een I/O event te wachten, nemen plaats in een aparte queue. Wanneer deze queue processen bevat, worden deze eerst bediend. De tijd dat ze over de processor mogen beschikken is gelijk aan het quantum q min de tijd dat ze reeds over de processor konden beschikken sinds de laatste maal dat ze uit de ready queue werden gekozen. Er is wel wat boekhouding vereist om dit schema te realiseren.

Shortest Process Next (SPN): Met deze discipline wordt steeds het proces uit de ready queue gekozen dat de minste processortijd vereist (dat is, het kortste proces). Wanneer we een aantal taken moeten schedulen, die allen op tijd 0 aanwezig zijn, dan kunnen we eenvoudig tonen dat deze volgorde aanleiding geeft tot de laagst mogelijke gemiddelde turnaround time (onder de aanname dat het proces in één keer kan worden uitgevoerd). Kortom, gebruik makend van onze notatie stellen we dat SPN is optimaal voor

$$1 || \sum C_j.$$

We kunnen dit eenvoudig bewijzen (voor niet-preemptieve systemen, het resultaat voor preemptieve systemen volgt uit het SRTN resultaat). Stel dat de processing time van taak j gelijk is aan p_j , voor $j = 1, \dots, N$. Stel verder dat we eerst taak 1 uitvoeren, dan taak 2, enz. Dan zal de totale turnaround time (T_q) gelijk zijn aan

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i. \quad (3)$$

Wanneer we de taken zo ordenen dat $p_1 \leq p_2 \leq \dots \leq p_N$, dan is deze uitdrukking minimaal. Kortom, als we taken niet mogen onderbreken en alle taken zijn aanwezig op tijd 0 dan is SPN optimaal voor de gemiddelde wachttijd.

Echter, wanneer alle taken nog niet in het systeem zijn op tijd nul, dan is dit niet meer het geval, of nog SPN is niet optimaal voor

$$1 |r_j| \sum C_j,$$

waarbij we r_j noteerde als de aankomsttijd van taak j . We zien dit aan de hand van volgend eenvoudig voorbeeld. Stel dat we twee taken hebben met $p_1 = 100$, $p_2 = 1$, $r_1 = 0$ en $r_2 = 1$. SPN resulteert in een gemiddelde turnaround time van $(100 + 100)/2 = 100$. Indien de processor echter leeg blijft tot tijd 1 en dan eerst taak 2 uitvoert, is de gemiddelde turnaround time gelijk aan $(102 + 1)/2 = 51.5$. Bij het bespreken van de SRTN discipline gaan we dieper op dit probleem in.

Hoewel SPN optimaal is voor het scheduleren van een batch van taken, is deze discipline moeilijk realiseerbaar, daar het voorkennis vereist van de looptijd van alle processen. In een productieomgeving gebeurt het wel eens dat een set van dezelfde taken veelvuldig wordt uitgevoerd. In dat geval kan men de processing time van de taken schatten en gebruiken bij het scheduleren van toekomstige taken. Merk op dat deze techniek nadelig is voor grote taken.

Weighted Shortest Process Next (WSPN): Dit algoritme, dat eveneens onder de naam Smith's rule gekend is, verwerkt de taken niet volgens oplopende lengte p_j , maar volgens p_j/w_j waarbij $w_j \geq 0$ een gewicht is. Stel dat alle taken op tijd 0 aanwezig zijn. Herinner dat we de completion time van taak j als C_j hebben genoteerd. Dan zal WSPN de gemiddelde gewogen completion time minimaliseren, of nog WSPN is optimaal voor

$$1 \parallel \sum_{j=1}^N w_j C_j.$$

We bewijzen dit even voor niet-preemptieve systemen.

Stel dat een andere orde S' beter is. Nummer de taken zodat ze in volgorde afgewerkt worden. Dan bestaat er een i met $p_{i+1}/w_{i+1} < p_i/w_i$, d.i. $p_{i+1}w_i < p_iw_{i+1}$. We kunnen $\sum_{j=1}^N w_j C_j$ herschrijven als

$$w_1 p_1 + w_2(p_1 + p_2) + \dots + w_N(p_1 + \dots + p_N) = \sum_{k=1}^N p_k \left(\sum_{l=k}^N w_l \right). \quad (4)$$

Wisselen we taak i en $i + 1$ dan wijzigt enkel de i -de en $i + 1$ -ste term van

$$p_i(w_i + \dots + w_N) + p_{i+1}(w_{i+1} + \dots + w_N)$$

naar

$$p_{i+1}(w_i + \dots + w_N) + p_i(w_i + w_{i+2} + \dots + w_N)$$

Het verschil tussen beide is $p_i w_{i+1} - p_{i+1} w_i > 0$, wat aangeeft dat de gewogen completion time is afgenomen. Dit is in tegenspraak met de minimaliteit van S' .

Shortest Remaining Time Next (SRTN): We kunnen het SPN algoritme verfijnen wanneer taken onderbroken kunnen worden (we veronderstellen dat het onderbreken zelf geen tijdsverlies oplevert). SRTN zal steeds bij het ready worden van een nieuw proces controleren of dit nieuwe proces minder looptijd behoeft dan de resterende processing time van het huidige proces. Indien dit het geval is, wordt het lopende proces onderbroken en vervangen door het nieuwe proces. Op deze manier zal het proces dat nog het minste processortijd vereist steeds aanwezig zijn in de processor.

SRTN is optimaal wanneer het gaat om het minimaliseren van de turnaround time. Ongeacht of de taken reeds allemaal vanaf tijd 0 aanwezig zijn (in tegenstelling tot SPN) en wanneer we onderbreking toelaten:

$$1|r_j, pmtn| \sum C_j,$$

aangezien de gemiddelde turnaround time dan gelijk is aan $\sum_{j=1}^N (C_j - r_j)/N$ en r_j een vaste waarde heeft, volstaat het $\sum_j C_j$ te minimaliseren.

Stel dat een andere discipline S' beter is. Dan bestaat er een tijdstip t waarop taak j wordt uitgevoerd terwijl er een andere taak i aanwezig is (d.i., $t \geq r_i$) die een kleinere resterende looptijd heeft. We noteren de resterende looptijd als p_j^r en p_i^r , respectievelijk. Vervolgens wisselen we taak i en j zodat gedurende de tijd dat taak i of j actief was, we eerst i gedurende de p_i^r eenheden volledig uitvoeren. Vervolgens laten we taak j uitvoeren in de resterende p_j^r tijd. Als we C'_j en C'_i noteren als de nieuwe completion times van beide taken dan is $C'_j = \max\{C_i, C_j\}$ en $C'_i < \min\{C_i, C_j\}$. Bijgevolg zien we dat

$$\sum_k C_k = \sum_{k \neq i,j} C_k + C_i + C_j > \sum_{k \neq i,j} C_k + C'_i + C'_j. \quad (5)$$

Aangezien de completion times van de andere taken niet wijzigen, was $\sum_k C_k$ niet minimaal en verkrijgen we een tegenspraak op de optimaliteit van S' .

Hoewel SRTN optimaal is, vergt het wel perfecte kennis van de lengte van alle taken en moet ook de resterende processing time van alle taken bijgehouden worden. Daarnaast treden er ook starvation effecten op voor de langere taken, doordat deze continu worden voorbij gestoken door korte taken.

Nonpreemptive SRTN (nSRTN): We kunnen met het resultaat van het SRTN algoritme ook een nieuwe scheduling discipline opstellen die tracht om de looptijd te minimaliseren wanneer taken niet onderbroken worden en waarbij alle taken niet aanwezig hoeven te zijn op tijd 0. Het gaat dus om volgend model:

$$1|r_j| \sum C_j.$$

Men heeft aangetoond dat het vinden van de optimale oplossing NP-hard is. Echter, met behulp van het SRTN resultaat kunnen we een volgorde opstellen zodat de som van de completion times hoogstens twee maal zo groot is als de optimale, men noemt zulk een algoritme een *2-approximation*.

We gaan als volgt te werk. Stel dat we de taken hernummeren zodat de SRTN discipline ze afwerkt in volgorde: $C_1 \leq C_2 \leq \dots \leq C_N$. Dan construeren we een nieuwe discipline met completion times $\bar{C}_1 \leq \bar{C}_2 \leq \dots \leq \bar{C}_N$, door de taken in dezelfde volgorde af te werken, waarbij we eventueel de processor een tijdje leeg laten tot de taak in kwestie is aangekomen (denk terug aan het SPN voorbeeld met de twee processen). We noemen deze discipline nonpreemptive SRTN.

Deze nieuwe discipline zal aanleiding geven tot een gemiddelde turn-around time die maximaal twee maal zo groot is als de optimale. Om dit te bewijzen volstaat het te tonen dat $\sum_j \bar{C}_j \leq 2 \sum_j C_j$. Want het beste algoritme dat geen onderbrekingen mag hanteren, kan nooit beter presteren dan het optimale algoritme dat wel gebruik kan maken van onderbrekingen.

Wanneer we kijken naar de completion time van de j -de taak $\bar{C}_j$, dan is deze maximaal indien de eerste taak als laatste van de eerste j taken aankomt (want een idle periode eindigt steeds met een aankomst van een nieuwe taak):

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k. \quad (6)$$

Verder is $r_k \leq C_k \leq C_j$ voor $k \leq j$, bijgevolg is $\max_{k \leq j} r_k \leq C_j$. Aangezien de taken in volgorde worden afgehandeld, is verder $\sum_{k=1}^j p_k \leq C_j$. Hieruit volgt $\bar{C}_j \leq 2C_j$, wat volstaat om het gestelde te bewijzen.

We kunnen deze bovengrens niet meer aanscherpen, want er zijn voorbeelden waarin de gemiddelde looptijd tot 2 maal zo groot is. Neem bijvoorbeeld 2 taken met $p_1 = N$, $p_2 = 1$, $r_1 = 0$ en $r_2 = N - 2$. Dan zal de gemiddelde looptijd gelijk zijn aan $(N + 3)/2$ in het geval van SPN. SRTN doet iets

Algoritme	Eigenschap	Systeem
SPN	optimaal	$1 \sum C_j$
WSPN	optimaal	$1 \sum w_j c_j$
SRTN	optimaal	$1 r_j, pmtn \sum C_j$
nSRTN	2-approximatie	$1 r_j \sum C_j$

Fig. 40: Theoretische resultaten voor Uniprocessor Scheduling

beter met $(N + 1 + 1)/2$. Echter, bij nonpreemptive SRTN blijft de processor leeg gedurende de eerste $N - 2$ eenheden en is de gemiddelde looptijd bijgevolg gelijk aan $(2N - 1 + 1)/2 = N$. De limiet voor N naar oneindig van $2N/(N + 2)$ is gelijk aan 2. Een overzicht van deze theoretische resultaten vinden we terug in Figuur 40.

Highest Response Ratio Next (HRRN): Deze discipline maakt gebruik van de gewogen turnaround time (T_q/p_s) bij het selecteren van een proces. In het bijzonder heeft elk proces een voorlopige gewogen turnaround time gelijk aan $RR = (w + p_s)/p_s$, met w gelijk aan de tijd dat het proces reeds wachtte op de processor. Bij het opstarten is $RR = 1$, daarna neemt de waarde toe met de wachttijd. De toename is wel proportioneel met de processing time p_s van het proces. Hierdoor zal de RR van kortere processen sneller toenemen.

De HRRN discipline zal, wanneer een proces de processor vrijgeeft, het proces selecteren met de grootste RR waarde. We merken dat korte processen voordeel krijgen, omdat hun RR sneller stijgt. Anderzijds wordt het starvation effect van de langere processen tegengegaan doordat deze uiteindelijk toch de grootste RR waarde verkrijgen. Deze discipline vergt wel, net als alle voorgaande, kennis van de processing time van de processen (of tenminste een schatting hiervan).

Multilevel Feedback Scheduling: Feedback scheduling tracht het gedrag van de voorgaande schema's te mimicken, zonder echter over de (geschatte) processing times te beschikken. Het idee is om niet naar de tijd te kijken dat een proces nog nodig heeft, informatie die we niet hebben, maar te kijken naar de tijd dat een proces reeds over de processor kon beschikken. Naarmate deze tijd toeneemt, wensen we een proces minder frequent toe te laten tot de processor.

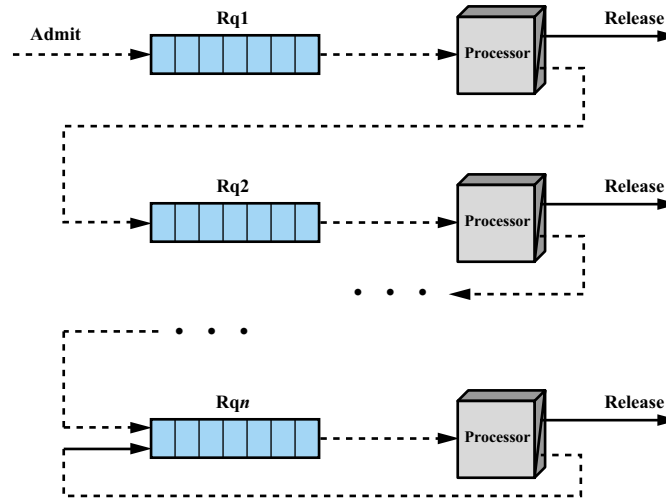


Fig. 41: Multilevel Feedback Scheduling [Stallings 03]

Een mogelijke implementatie bestaat erin om n ready queues te hanteren, die met een prioriteitsmechanisme worden bediend (zie Figuur 41). Dit houdt in dat processen in de i -de queue pas kunnen beschikken over de processor, als de $i - 1$ voorgaande queues leeg zijn. Een proces dat ready wordt, zal plaatsnemen in de eerste queue. Wanneer de processor een proces uit queue i selecteert, kan het gedurende één quantum over de processor beschikken. Vervolgens neemt het achteraan plaats in de $i + 1$ -ste queue (tenzij $i = n$, want dan neemt het wederom plaats in queue n). Indien een proces afgewerkt wordt, verdwijnt het uit het systeem. Elk van de queues wordt op een FCFS wijze bediend, behalve de laatste die een RR discipline hanteert.

We merken op dat korte taken op deze manier bevoordeeld worden, daar zij minder ver in de rij van queues afdalen. Om starvation van langdurige processen tegen te gaan, zal het quantum q vaak toenemen met de index i van de queue, bijvoorbeeld $q = 2^{i-1}$. Verder hebben I/O-bound processen het voordeel dat ze steeds opnieuw plaats nemen in queue 1, nadat ze een I/O operatie hebben uitgevoerd. Figuur 42 geeft tenslotte een overzicht van de verscheidene disciplines aan de hand van een voorbeeld.

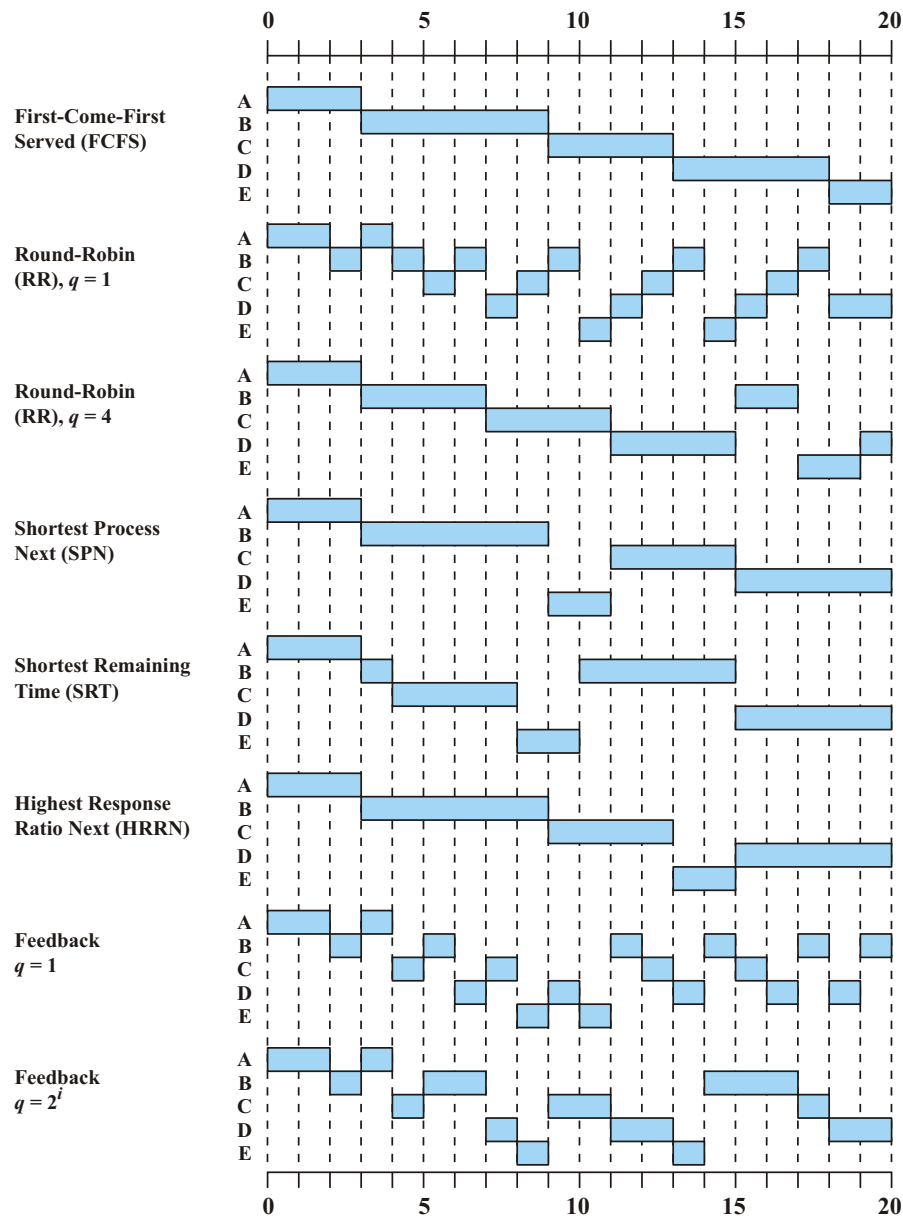


Fig. 42: Schedulingvoorbeeld met 5 processen met $r_i = 2i - 2$ [Stallings 03]

1.4 Traditionele UNIX scheduler

In deze sectie kijken we even in meer detail naar de werking van een traditionele UNIX scheduler. Deze scheduler werkt vrij gelijkaardig aan een

multilevel feedback queue. De tijd wordt door de scheduler opgedeeld in intervallen van gelijke lengte (typisch 1 seconde) en elk proces j heeft een prioriteit $P_j(i)$ gedurende interval i . Hoe lager deze waarde, hoe hoger de prioriteit. Verder heeft de scheduler ook een *scheduler resolution*, die aangeeft hoe vaak er een clock interrupt wordt gegenereerd per seconde, doorgaans zijn dit een 60 tal interrupts per seconde.

Voor elke proces j wordt er een counter $U_j(i)$ bijgehouden die aangeeft hoe frequent het proces actief was tijdens het huidige interval i , bij elke interrupt zal de counter van het actieve proces met één verhoogd worden. Met behulp van deze counter wordt er, op het einde van elk interval, een exponentieel gewogen gemiddelde van het CPU gebruik berekend, als volgt

$$CPU_j(i) = U_j(i)/2 + CPU_j(i-1)/2.$$

Vervolgens wordt de prioriteit van alle processen aangepast met behulp van onderstaande formule:

$$P_j(i) = Base_j + CPU_j(i-1)/2 + nice_j,$$

waarbij het gebruik van $Base_j$ en $nice_j$ later wordt toegelicht. Bij elke clock interrupt verkrijgt het proces met de hoogste prioriteit, d.i., de laagste $P_j(i)$ waarde, de CPU. Meestal zijn er meerdere processen met dezelfde prioriteit en wordt round-robin gehanteerd. Net zoals bij de multilevel feedback queue zien we dus dat de prioriteit van een proces dat over de CPU kon beschikken, zal afnemen. Echter, in tegenstelling tot de klassieke multilevel feedback queue, zal de prioriteit van niet-actieve processen vanzelf toenemen, omdat de variabele $CPU_j(i)$ dan kleiner is dan $CPU_j(i-1)$. Op deze manier zal starvation van grote processen automatisch worden tegengegaan.

Merk ook op dat er zich steeds nieuwe processen kunnen aandienen tijdens een interval i , die mogelijk een hogere prioriteit hebben dan het proces dat momenteel actief is. In dit geval zal het proces in uitvoering de CPU bij de volgende clock interrupt moeten afstaan aan het nieuwe proces.

De $Base_j$ parameter zorgt ervoor dat de processen in verschillende klassen worden opgedeeld. Zo zullen swap processen en enkele I/O ondersteunende processen steeds een hogere prioriteit krijgen dan de user processen. Door de $Base_j$ van de user processen voldoende groot te nemen, zal het swap proces steeds een kleinere $P_j(i)$ waarde hebben, zelfs wanneer het zeer vaak actief was gedurende de laatste intervallen.

Time	Process A		Process B		Process C	
	Priority	CPU Count	Priority	CPU Count	Priority	CPU Count
0	60	0 1 2 • • 60	60	0	60	0
1	75	30	60 0 1 2 • • 60	0	60	0
2	67	15	75	30	60 0 1 2 • • 60	0
3	63 7 8 9 • • 67	7	67	15	75	30
4	76	33	63 7 8 9 • • 67	7	67	15
5	68	16	76	33	63	7

Fig. 43: Voorbeeld van een traditionele UNIX scheduler [Stallings 03]

De $nice_j$ parameter kan door de gebruiker zelf worden ingesteld. Default is deze gelijk aan nul. Echter, de gebruiker kan deze verhogen tot 19 om zo andere processen vaker voorrang te verlenen.

1.5 Fair-share scheduler

Hoewel heel wat scheduling disciplines voor fairness zorgen tussen verschillende processen, bestaat er vaak ook een vraag naar fairness tussen groepen van processen. Zo zou men alle processen per gebruiker kunnen groeperen en fairness tussen gebruikers realiseren. Of indien meerdere organisaties samen een enkele (dure) machine delen, dan is het wenselijk de resources evenredig aan de gemaakte investeringen te verdelen tussen deze organisaties.

Een mogelijke oplossing in het geval van de klassieke UNIX schedulers bestaat erin om nog een counter $GU_k(i)$ in te voeren per groep k . Deze zal met één verhogen bij elke clock interrupt, indien een proces uit groep k actief was. Deze counter wordt analoog aan de variabele $U_j(i)$ gebruikt om

de variabele $GCPU_k(i)$ aan te passen op het einde van elk interval:

$$GCPU_k(i) = GU_k(i)/2 + GCPU_k(i-1)/2.$$

Deze variabele zal op zijn beurt dan de prioriteit van een proces j uit groep k als volgt beïnvloeden:

$$P_j(i) = Base_j + CPU_j(i-1)/2 + GCPU_k(i-1)/4W_k + nice_j,$$

waarbij W_k een groepsgewicht is, met $\sum_k W_k = 1$, dat aangeeft hoeveel procent van de CPU aan groep k toekomt. Een dubbel zo grote waarde voor W_k zou moeten resulteren in een dubbel zo frequent gebruik van de CPU. Indien er veel processen in een enkele groep zitten, dan zal de term $GCPU_k(i-1)/4W_k$ vaak doorslaggevend zijn voor de prioriteit $P_j(i)$ te bepalen. Deze term zal bepalen welke groep(en) er over de CPU kan(kunnen) beschikken. De term $CPU_j(i-1)/2$ geeft vervolgens aan welk van de processen behorende tot deze groep(en) effectief over de CPU kan(kunnen) beschikken.

2 Real-time short-term scheduling

In deze sectie maken we kennis met enkele real-time scheduling algoritmen. Het grote verschil met hun non-real-time tegenhangers, is dat er met een taak i ook een deadline d_i is geassocieerd. Deze deadline geeft aan wanneer de taak ten laatste uitgevoerd moet zijn. In sommige gevallen kan het ook zijn dat er een deadline rust op de starttijd in plaats van de completion tijd, dit is voornamelijk het geval voor niet-preemptieve systemen.

Real-time scheduling oplossingen worden vaak opgedeeld in statische en dynamische oplossingen. Statische oplossingen worden off-line uitgevoerd, deze algoritmen worden dus uitgevoerd voordat het systeem operationeel is. We onderscheiden twee types:

1. Statische *tabel-gedreven* oplossingen: In dit geval wordt er op voorhand een tabel opgesteld die perfect weergeeft welke taken wanneer uitgevoerd worden. Indien niet alle taken hun deadline kunnen halen, dan zullen er enkele taken niet uitgevoerd worden.
2. Statische *prioriteit-gedreven* oplossingen: In tegenstelling tot tabel-gedreven oplossingen wordt er geen tabel geproduceerd als output, maar krijgt elke taak een prioriteit als resultaat van het scheduling algoritme. Een klassieke, op prioriteiten gebaseerde scheduler zorgt voor de eigenlijke uitvoering van de taken.

Wanneer we opteren voor een dynamische oplossing dan zal er on-line een beslissing genomen worden. Met andere woorden, terwijl het systeem operationeel is, worden er beslissingen genomen in verband met het aanvaarden van nieuwe taken. Dit kan op een *best-effort* wijze, waarbij alle taken initieel aanvaard worden en deze die hun deadline niet halen worden stopgezet. Daarnaast kan het ook zijn dat een taak enkel wordt toegelaten indien er zekerheid is dat ze kan worden uitgevoerd binnen de vooropgestelde deadline. Dit heeft als voordeel dat er geen CPU tijd verloren gaat aan de gedeeltelijke uitvoering van taken. Het nadeel is natuurlijk de complexiteit die gepaard gaat met zulk een implementatie.

Belangrijk is ook op te merken dat niet alle taken deadlines hoeven te hebben in een real-time systeem. Verder kan het ook zijn dat sommige deadlines *hard* zijn, wat inhoudt dat ze nooit overschreden mogen worden, of *soft*, wat dan weer aangeeft dat een kleine overschrijding nog oplosbaar is. In tegenstelling tot een klassieke UNIX scheduler, zullen sommige real-time schedulers ook in staat zijn een proces van lagere prioriteit sneller te onderbreken dan de eerstvolgende clock interrupt. Dit kan van belang zijn daar clock interrupts maar elke paar milliseconden optreden, terwijl bijvoorbeeld militaire vluchtsimulators interrupts wensen te ondersteunen die binnen de 100 microseconden effect hebben.

We zullen twee groepen van statische scheduling oplossingen bespreken: deadline scheduling, dat een tabel-gedreven oplossing aanreikt, en rate monotonic scheduling, wat een aanpak is die behoort tot de groep van prioriteit-gedreven oplossingen.

2.1 Deadline scheduling

We veronderstellen dat we alle uit te voeren taken kennen, weten wat hun looptijd p_i en deadline d_i is. In een real-time setting is de veronderstelling dat we de uitvoertijd kennen meer realistisch, tenslotte heeft deze ook een invloed op de deadline. Zoals voorheen, noteren we C_j als de completion time van taak j . We stellen $U_j = 1$ indien $C_j > d_j$, wat aangeeft dat taak j niet tijdig is uitgevoerd, anders stellen we $U_j = 0$. We zijn nu niet zozeer geïnteresseerd in de gemiddelde turnaround tijd $1/n \sum_{j=1}^n C_j$, maar in het aantal taken $\sum_{j=1}^n U_j$ dat zijn deadline niet haalt of in de maximale mate waarin een taak te laat is, zijnde $L_{max} = \max_{j=1}^n (C_j - d_j)$. Wat betreft onze scheduling model notatie hebben we dus twee extra mogelijkheden voor γ , de functie die we wensen te minimaliseren. Merk op, indien $\gamma = L_{max}$ of

$\sum U_j$, dan geeft dit impliciet aan dat de taken over een deadline beschikken, we hoeven d_j dus niet als randvoorwaarde in β te specificeren.

Earliest Deadline First (EDF) – Earliest Due Date (EDD): Deze discipline, die ook gekend is onder de naam Jackson's rule, tracht de maximale mate waarin een taak te laat is te minimaliseren, of nog, L_{max} zo klein mogelijk te maken. Ze stelt dat we de taken uitvoeren in volgorde van oplopende deadline d_i en dit zonder onderbreking, we ordenen de n taken dus zodanig dat $d_1 \leq d_2 \leq \dots \leq d_n$. Merk op, er is geen kennis van de looptijd p_i vereist. We zullen bewijzen dat deze eenvoudige discipline $\max_{j=1}^n (C_j - d_j)$ minimaliseert, indien alle taken vanaf tijd 0 uitgevoerd kunnen worden (dat is, de aankomsttijden r_i zijn gelijk aan nul). Het gaat dus om volgend systeem:

$$1 || L_{max}.$$

Stel dat S een andere orde is. Dan bestaan er twee taken j en k die vlak na elkaar uitgevoerd worden met $d_k < d_j$, zoniet dan is S de EDF orde. Door deze in volgorde te wisselen wijzigen we enkel de termen $C_j - d_j$ en $C_k - d_k$ in $\max_{i=1}^n (C_i - d_i)$ en verkrijgen we een nieuwe discipline S' . Stel dat taak j op tijd t startte in geval van de orde S , dan $C_j = t + p_j$ en $C_k = t + p_j + p_k$. Door de verwisseling zien we dat $C'_j = t + p_k + p_j = C_k$ en $C'_k = t + p_k$. Bijgevolg,

$$C'_j - d_j = C_k - d_j < C_k - d_k,$$

en

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k.$$

Kortom, $\max_{i=1}^n (C'_i - d_i) \leq \max_{i=1}^n (C_i - d_i)$ en de orde S' lijkt meer op de EDF orde dan S . Door dit argument te herhalen zien we dat dit maximum voor de EDF orde kleiner is of gelijk aan het maximum voor de orde S , dewelke willekeurig gekozen was.

Een zeer belangrijk gevolg van dit resultaat is, dat een reeks taken met bijhorende deadlines enkel kan gerealiseerd worden indien ook de EDF orde alle deadlines haalt, of nog, dat $L_{max} = \max_{j=1}^n (C_j - d_j) \leq 0$ voor EDF. We kunnen zelfs eenvoudig¹ aantonen dat EDF alle deadlines haalt als en slechts

¹ Dat deze voorwaarde noodzakelijk is, dat is duidelijk. Het omgekeerde volgt onmiddellijk door te stellen dat indien EDF de taken niet kan schedulen, er minstens één van deze voorwaarden moet falen.

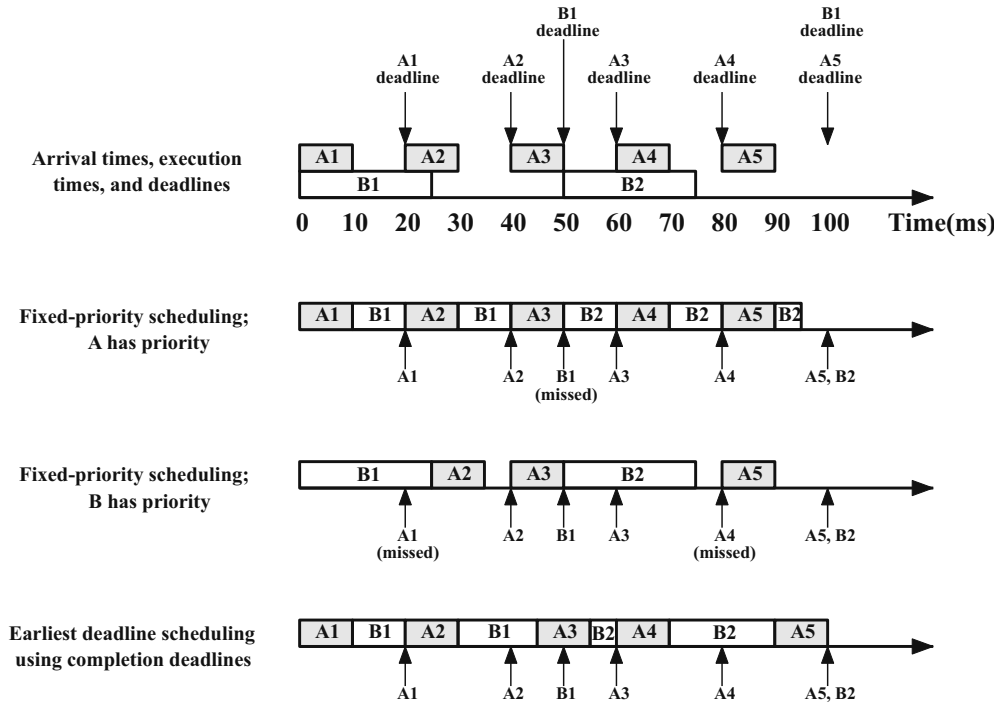


Fig. 44: Voorbeeld van EDF scheduling ten opzichte van priority scheduling [Stallings 03]

als voor elke deadline d_i geldt

$$u_i = \frac{1}{d_i} \sum_{k: d_k \leq d_i} p_k \leq 1.$$

$u = \max_i u_i$ is noemen we de loading factor of load. Kortom, als de load minder dan 1 is dan zal EDF de taken kunnen schedulen zonder dat er een deadline wordt overschreden. EDF heeft echter geen goede performantie wat betreft het aantal taken dat zijn deadline niet haalt wanneer $u > 1$. Hiervoor kunnen we beter het algoritme van Moore en Hodgson hanteren. Een voorbeeld van EDF scheduling zien we in Figuur 44.

Moore-Hodgson Algoritme (MHA): Dit algoritme kijkt naar hetzelfde probleem als het EDF algoritme, maar tracht het aantal taken dat zijn deadline niet haalt $\sum_{j=1}^n U_j$ te minimaliseren. Het werkt als volgt:

- S1: Dit algoritme sorteert eerst alle taken in de EDF orde zodat $d_i \leq d_{i+1}$ voor $i = 1, \dots, n - 1$. Hierdoor bekomen we een reeks van n taken.
- S2: Vervolgens wordt er gekeken of er (nog) een taak in de reeks voorkomt die te laat uitgevoerd zou worden. Bestaat deze niet, dan is de orde optimaal, anders gaan we verder met Stap S3.
- S3: Laat J_α de eerste taak zijn in de reeks die te laat is uitgevoerd. Neem dan een job J_β met $p_\beta = \max_{i=1}^\alpha p_i$, of nog een zo lang mogelijke taak uit de eerste α taken in de reeks en verwijder deze uit de reeks. Pas de completion times aan van de taken die na J_β werden uitgevoerd en keer terug naar Stap S2.

Men kan bewijzen dat dit algoritme het kleinst mogelijke aantal taken verwijdert zodat de deadlines van alle resterende taken worden gerespecteerd. Het is dus optimaal voor volgend systeem:

$$1 || \sum U_j.$$

Hoewel dit eenvoudig te bewijzen is, laten we het bewijs voor wat het is.

Aankomsttijden en preemptie: Wanneer we de extra eis opleggen dat elke taak ook een aankomsttijd r_i heeft en we laten geen preemptie toe, dan zijn beide besproken problemen echter NP-hard. Oplossingen in polynomiale tijd zijn wel beschikbaar indien we aankomsttijden combineren met preemptie. Bijvoorbeeld, EDF is optimaal voor

$$1|r_j, pmtn|L_{max}$$

en zal steeds alle deadlines halen als de load kleiner is of gelijk aan één. In het geval dat er aankomsttijden $r_j > 0$ zijn, wordt de load gedefiniëerd als $u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)}$ met

$$u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j.$$

Ook in het niet-preemptieve geval is EDF nog optimaal als we ons beperken tot *no-idling* scheduling ordes. Deze laten niet toe dat de processor een tijdje idle is, terwijl er toch nog een taak gestart kan worden. Het bewijs is analoog aan het voorgaande door op te merken dat voor twee ordes die beide geen idling toelaten, de processor op dezelfde ogenblikken idle zal zijn.

Precedence voorwaarden: Wanneer we in de praktijk een set real-time taken wensen uit te voeren, dan kan het voorvallen dat we alle taken niet zomaar in gelijk welke volgorde kunnen uitvoeren. De uitvoering van een taak i kan soms enkel nadat een andere taak j is uitgevoerd. We zeggen dan dat er tussen taak i en j een *precedence* voorwaarde of relatie bestaat. We kunnen al deze voorwaarden karakteriseren met behulp van een gerichte graaf $G = (V, E)$ door alle taken als knopen in V voor te stellen en de precedence relaties als edges in E . Deze graaf mag geen gesloten paden bevatten, want anders zal elke mogelijke volgorde één van de precedence regels verbreken. Kortom, de graaf moet *acyclisch* zijn.

Als we veronderstellen dat we de duur p_j en de deadline d_j van alle taken kennen en dat de aankomsttijden r_j gelijk zijn aan nul, dan zal een algoritme van Lawler de maximale mate waarin de deadlines worden overschreden $L_{max} = \max_j (C_j - d_j)$ minimaliseren, het is dus optimaal voor

$$1|prec|L_{max}$$

en het werkt als volgt:

- S1: Laat $num = n$ het totaal aantal taken zijn en G de graaf zijn met de precedence voorwaarden. Stel verder dat $U(G)$ de knopen in G bevat waaruit er geen edges vertrekken. Stel dat T gelijk is aan de uitvoertijd van alle taken samen, i.e., $T = \sum_{j \in V} p_j$.
- S2: Kies een taak j in $U(G)$ waarvoor $T - d_j$ minimaal is (of nog, waarvoor d_j maximaal is). Deze taak krijgt nummer num . Verlaag num met één, stel $T = T - p_j$ en verwijder de knoop j uit de graaf G . Herbepaal de verzameling $U(G)$ en herhaal stap S2 tot $num = 0$.

De taken worden uitgevoerd in de volgorde aangegeven door de num waarde die verkregen werd tijdens stap S2. Kortom, de volgorde van uitvoering wordt in omgekeerde volgorde vastgelegd. Een grafische weergave van de werking van het algoritme van Lawler wordt weergegeven in Figuur 45.

Wanneer we ook hier aankomsttijden $r_j > 0$ in het spel brengen en we laten geen preemtie toe, dan hebben we opnieuw met een NP-hard probleem te maken (daar geen precedence een bijzonder geval is van precedence met $G = (V, E)$ zodat E leeg is). Wanneer we preemtie wel ondersteunen dan bestaat er opnieuw een optimale oplossing die in polynomiale tijd gevonden kan worden (zijnde met behulp van het algoritme van Baker).

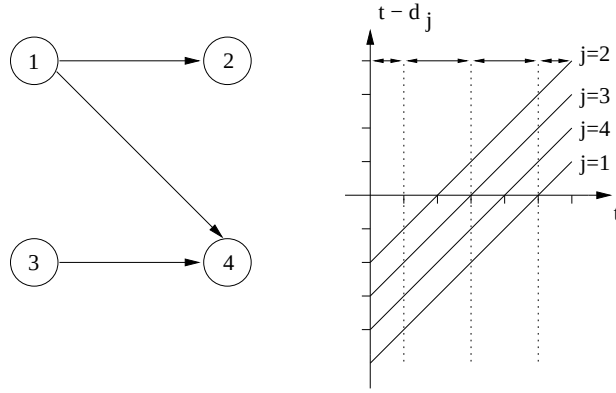


Fig. 45: Voorbeeld van precedence scheduling met behulp van Lawler's algoritme: $p_1 = 1$, $p_2 = 2$, $p_3 = 2$, $p_4 = 1$ en $d_1 = 5$, $d_2 = 2$, $d_3 = 3$, $d_4 = 4$. De optimale volgorde is 1, 2, 3, 4 en $L_{max} = 2$.

2.2 Rate monotonic scheduling (RMS)

In deze sectie leggen we ons toe op het probleem van het schedulen van periodische taken. We stellen dat we n periodische taken hebben. Alle instanties van taak i hebben een looptijd p_i en de periode van de taak i instanties is gelijk aan T_i , wat inhoudt dat we de taak één maal per elke T_i tijdseenheden wensen uit te voeren. Met andere woorden, voor elke taak wordt er een oneindige stroom van instanties gegenereerd met aankomsttijden $r_j = jT_i$, voor $j \geq 0$. Als *relatieve* deadline voor taak i nemen we T_i , wat impliceert dat een instantie van taak i uitgevoerd moet zijn voordat een nieuwe instantie van dezelfde taak zich aandient.

Het percentage van de tijd dat een taak de processor belast, of nog, de *utilization*, is duidelijk gelijk aan p_i/T_i . Voor we rate monotonic schedulen bespreken, tonen we aan dat een set van n periodische taken kan gerealiseerd worden met EDF als en slechts als

$$U = \sum_{j=1}^n p_j/T_j \leq 1.$$

We tonen eerst aan dat de load $u \leq U$. Namelijk, voor alle $t_1 < t_2$ hebben we

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^n \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j \leq (t_2 - t_1)U,$$

want er zijn ten hoogste $\left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor$ instanties van taak j die in het interval $[t_1, t_2)$ aankomen en die tevens een deadline hebben die kleiner is dan t_2 .

Omgekeerd moeten we tonen dat $u \geq U$, waaruit dan volgt dat $u = U$. We doen dit door $t_1 = 0$ en t_2 aan het kleinste gemeenschappelijke veelvoud H van $T_1, \dots, T_n$ toe te kennen. Dan

$$u_{[0, H)} H = \sum_{j: 0 \leq r_j, d_j \leq H} p_j = \sum_{j=1}^n \frac{H}{T_j} p_j = HU.$$

Aangezien u het supremum is over alle intervallen is $u \geq U$. Hiermee is het resultaat bewezen, daar EDF alles kan schedulen zonder een deadline te overschrijden als en slechts als de load $u = U \leq 1$.

Rate Monotonic Scheduling (RMS): Hoewel EDF optimaal is voor het schedulen van periodische real-time taken, in de zin dat het de maximale capaciteit van de processor kan benutten, is het vaak zo dat niet alle taken real-time zijn of dat niet alle taken harde deadlines hebben. Een interessant alternatief kan in dit geval geboden worden door aan elke instantie van taak i een prioriteit mee te geven en deze te integreren in een eerder klassieke UNIX scheduler. In dit geval zullen de prioriteiten van de real-time taken hoger zijn dan deze van alle andere processen en worden deze ook niet aangepast tijdens de uitvoering.

RMS, ontwikkeld door Liu en Layland, stelt dat we de taken ordenen zodat $T_1 \leq T_2 \leq \dots \leq T_n$ en de instanties van taak i de i -de hoogste prioriteit te geven. Men kan dan aantonen dat RMS steeds de deadlines zal respecteren indien

$$U = \sum_{j=1}^n p_j / T_j \leq n(2^{1/n} - 1).$$

Voor $n = 1$ is deze waarde gelijk aan 1, naarmate n groter wordt, daalt deze naar $\ln 2$ (dit zien we via L'Hopitals regel aangezien de afgeleide van $(2^{1/n} - 1)$ gelijk is aan $-2^{1/n} \ln 2 / n^2$). Dit is echter enkel een voldoende voorwaarde en geen noodzakelijke voorwaarde. RMS is optimaal binnen de klasse van de statische prioriteit-gedreven oplossingen (indien de relatieve deadlines gelijk zijn aan de periode). Verder zal RMS ook steeds alle deadlines respecteren indien alle periodes veelvouden van elkaar zijn en $U \leq 1$.

DISK MANAGEMENT

In dit hoofdstuk gaan we wat dieper in op de ontwerpbeslissingen die men moet treffen bij het management van (hard)disks. We starten met een bespreking van de werking van een (magnetische) disk. Daarna bekijken we enkele disk scheduling algoritmen. We eindigen met wat uit te wijden over een meer recente evolutie in disk management, zijnde RAID systemen.

1 Magnetische Disks

Een harddisk bestaat uit één (of meerdere) cirkelvormige (metalen) schijf waarop informatie wordt opgeslagen door middel van magnetisering. Typisch wordt deze informatie door een *disk head*, die aan een arm bevestigd is, geschreven en gelezen. De schijf roteert tegen een constante snelheid, meestal 3600, 5400, 7200 of 10000 rounds per minute (RPM). Dit cijfer is doorgaans correct tot op een halve procent of minder. In de praktijk zal de rotatiesnelheid lichtjes variëren rond het gemiddelde.

De informatie op de disk wordt georganiseerd in *tracks* en *sectoren*. Tracks zijn concentrische cirkels die elk bestaan uit een aantal sectoren (zie Figuur 46). Elke sector bevat een vast aantal bytes, dat bij heel wat disks default gelijk is aan 512 bytes. Wanneer data gelezen of geschreven wordt, gebeurt dit steeds in veelvouden van sectoren. Elke sector bevat ook wat overhead zoals error correction codes, ID informatie en een synchronisatie veld. Wanneer de harddisk uit meerdere schijven (platters) bestaat, dan spreken we over een cilinder als we verwijzen naar de set van tracks die op dezelfde afstand van de buitenkant van de disk gelegen zijn.

Zone bit recording: In het verleden bevatte elke track een vast aantal sectoren. Dit houdt in dat op de binnenste tracks de data erg compact bij elkaar zit, terwijl er op de buitenkant een veel lagere data densiteit is. Dit impliceerde ook dat de lees- of schrijfsnelheid dezelfde was over de hele disk. De capaciteit van de disks werd hierdoor veelal bepaald door de hoeveelheid info die men op de binnenste tracks kon plaatsen.

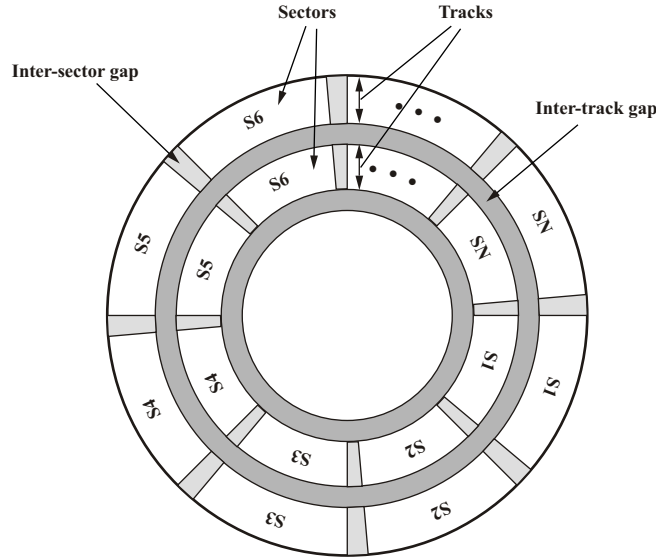


Fig. 46: Disk data layout [Stallings 03]

Vandaag maken veel disks gebruik van *zone bit recording*, wat inhoudt dat de disk in een aantal zones wordt opgedeeld en binnen elke zone bestaan alle tracks uit evenveel sectoren. Een track aan de buitenkant zal tot twee keer meer sectoren bevatten dan deze aan de binnenkant (zijnde enkele honderden). Door de constante rotatiesnelheid betekent dit dus dat de schrijven leessnelheid op een buitenste track een stuk hoger ligt in vergelijking met een track op de binnenste zone. Data wordt dan ook van buiten naar binnen op de disk geplaatst. I/O operaties op recent geschreven data, wanneer de disk bijna vol is, kunnen dus beduidend trager verlopen.

Positionering: Wanneer er een read of write optreedt, dan zal de head eerst bewegen tot deze zich boven de juiste track heeft gepositioneerd. Daarna wordt er gewacht tot de correcte sector (of sectoren) zich aandienen. De tijd nodig voor het positioneren van de head is de *seek time*, terwijl men het wachten op de sector de *rotational delay* noemt. De tijd nodig voor het lezen of schrijven zelf is de *transfer time*. Laat r het aantal revoluties per seconde zijn, T_s de seek time, N het aantal bytes op de track en b het aantal te lezen/schrijven bytes, dan is de access time T_a gemiddeld gezien gelijk aan

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}. \quad (7)$$

De gemiddelde rotational delay is gelijk aan 8.33 of 4.16 ms voor een disk met respectievelijk 3600 of 7200 rpm. De seek time ligt meestal rond de 8 tot 10 ms (met 15 tot 20 ms voor een *full strike*). De snelheid waarmee de arm voortbeweegt is echter niet uniform en is afhankelijk van het aantal tracks waarover we de arm bewegen.

Een seek is doorgaans opgebouwd uit vier fases: *speed-up*, *coast*, *slowdown* en *settle*. Hij versnelt dus eerst (*speed-up*), houdt vervolgens de constante maximale snelheid aan (*coast*), vertraagt tot nul (*slowdown*) en vervolgens wordt de head perfect boven de track gepositioneerd (*settle*). Heel korte seeks (van maximaal twee tot vier cilinders) bestaan veelal uit een *settle* (ongeveer 1 ms), korte seeks van enkele honderd tracks hebben geen *coast* fase, terwijl erg lange seeks voor een significant deel bestaan uit de *coast* fase (waar de tijd lineair toeneemt met het aantal af te lopen tracks). Voor de HP C2200A disk (335 MB) met 1449 cilinders kan de seek time als volgt goed worden benaderd, waarbij d het aantal af te leggen tracks weergeeft:

$$\text{seek time (ms)} = \begin{cases} 3.45 + 0.597\sqrt{d} & d \leq 616, \\ 10.8 + 0.012d & d > 616. \end{cases}$$

De seek time en rotational delay geven vaak veruit de grootste bijdrage aan de access time.

Wanneer er meerdere schijven aanwezig zijn, dan zullen de disk heads zich allemaal boven dezelfde track bevinden (omdat ze allemaal aan dezelfde arm verbonden zijn). Er zal echter maar één head data lezen of schrijven op elk gegeven ogenblik. Wanneer een leesoperatie wel een wissel van schijf vereist, maar geen trackwissel, dan zal er hierbij vaak toch een *settle* tijd verlopen, omdat de verschillende schijven niet perfect identiek gemaakt kunnen worden.

De beweging van de disk arm wordt gestuurd door de disk controller, deze moet de exacte kracht aangeven die de motor, die de armbeweging aandrijft, moet hanteren voor een bepaalde seek. Deze data wordt gedeeltelijk opgeslagen in een tabel. Om te vermijden dat alle mogelijke waarden opgeslagen moeten worden, kan de disk controller ook interpolatie methodes hanteren. Door temperatuurswisselingen en andere factoren moet deze tabel regelmatig gehercalibreerd worden, wat een kleine seconde duurt. Dit gebeurt automatisch om de zoveel tijd (bv. 1530 minuten) wanneer de machine actief blijft.

Track skewing en sparing: Om de leessnelheid van de disk hoog te houden bij track wissels, zal de data van de binnenste van twee aangrenzende tracks

verschoven worden met een hoeveelheid die overeenstemt met de (worst-case) tijd nodig om van track te wisselen (track skewing). Verder zullen een heel beperkt aantal sectoren reeds van bij de aanmaak van de disk fouten bevatten. Deze worden door veelvuldige tests opgespoord in de fabriek en door middel van een lijst aan de controller meegegeven. De controller kan de foutieve sectoren dan mappen op een alternatieve lokatie (sparing).

De disk controller: Net zoals alle andere I/O apparatuur, beschikt ook de harddisk over een controller die verbonden is met de bus (die toelaat data uit te wisselen met de processor, het main geheugen, etc). Deze controller beschikt over een aantal eigen registers en een buffer die minstens één sector kan opslaan (tenslotte is de leessnelheid van de disk meestal trager dan de snelheid van de bus). Het is de device driver software die de nodige requests via de processor in de registers van de controller plaatst. Wanneer de harddisk is uitgerust met *command queueing*, zullen er meerdere openstaande requests aanwezig kunnen zijn in de harddisk en kan de controller deze zelfstandig afhandelen in een mogelijk meer optimale volgorde. Het is dus belangrijk indien de operating systeem software zelf een volgorde wenst op te leggen, goed te weten hoe de controller hiermee precies omgaat. Indien de requests voldoende gespreid worden doorgestuurd naar de controller, dan worden deze natuurlijk wel in volgorde afgehandeld.

Disk caching: Vandaag de dag kan een disk eveneens over een eigen cache beschikken. Deze cache zal bij het lezen voornamelijk een *read-ahead* strategy volgen, waarbij ook de sectoren volgend op de gevraagde sector(en) in de cache worden ingelezen om mogelijke toekomstige requests onmiddellijk af te handelen. In tegenstelling tot andere caches, zal data die uit deze cache gelezen wordt, vaak direct verwijderd worden (het lezen van files is dan ook veel meer sequentieel van aard dan het doorlopen van code). Wanneer de read-ahead operaties een track grens overschreidt, dan spreekt men van *aggressive* read-ahead. Dit laatste kan nadelig zijn omdat een track wissel tijd vergt en niet onderbroken kan worden.

Oude harddisks hanteerden ook wel eens *on-arrival* read-ahead. Hierbij werd de track helemaal gelezen en de leesoperatie startte van zodra de disk head boven de track gepositioneerd werd. Voor operaties waarbij gevraagd werd een groot deel van de track te lezen, kon dit heel wat tijdswinst opleveren, want er werd niet gewacht op de start van het te lezen gebied. Echter, daar er

steeds meer sectoren op een enkele track staan en de gemiddelde size van een leesoperatie amper toeneemt, loont het niet meer echt de moeite on-arrival read-ahead te ondersteunen.

Voor het schrijven is caching ook erg nuttig daar dezelfde data overschreven kan worden in de cache zonder dat we tussentijds een schrijfoperatie op de disk moeten uitvoeren. Verder kan de controller zelf bepalen in welke volgorde de data ook effectief wordt weggeschreven op de disk. Enige voorzichtigheid is wel noodzakelijk, daar het cache geheugen mogelijk volatiel is, wat aangeeft dat de inhoud verloren gaat wanneer de cache niet van stroom voorzien wordt.

2 Disk Scheduling

De bovenstaande beschouwingen met betrekking tot de gemiddelde access time T_a zijn voornamelijk van toepassing voor *random requests*, wat wil zeggen dat de opeenvolgende locaties die we op de disk bezoeken zich op willekeurige posities bevinden. We kunnen de verloren tijd door de armbeweging en rotational delay reduceren door de reads en writes in een bepaalde volgorde af te handelen.

First-In-First-Out (FIFO): De meest voor de hand liggende en eerlijke volgorde is FIFO. FIFO kan aanleiding geven tot een goede performantie als vele requests geclusterd zijn op een beperkt aantal naburige tracks. Echter, in vele gevallen zal FIFO aanleiding geven tot een lage performantie die niet veel beter is dan random scheduling. In Figuur 47(a) zien we een voorbeeld waarin 8 requests in FIFO volgorde worden afgehandeld. In totaal moet de arm 640 tracks aflopen, wat aangeeft dat de FIFO volgorde inderdaad weinig efficiënt is.

Shortest Seek Time First (SSTF): Dit algoritme zal de huidige positie van de disk head in rekeningen nemen. De volgende request die afgehandeld wordt, is deze die de kleinste seek time vereist. Omdat de snelheid van de head om van track te wisselen niet lineair is in functie van het aantal tracks, is er wel een schatting vereist om de seek time te bepalen. Hoewel deze techniek een hoge utilization kan realiseren (en locality of reference eigenschappen uitbuit), kan hij aanleiding geven tot starvation. De disk blijft mogelijk erg

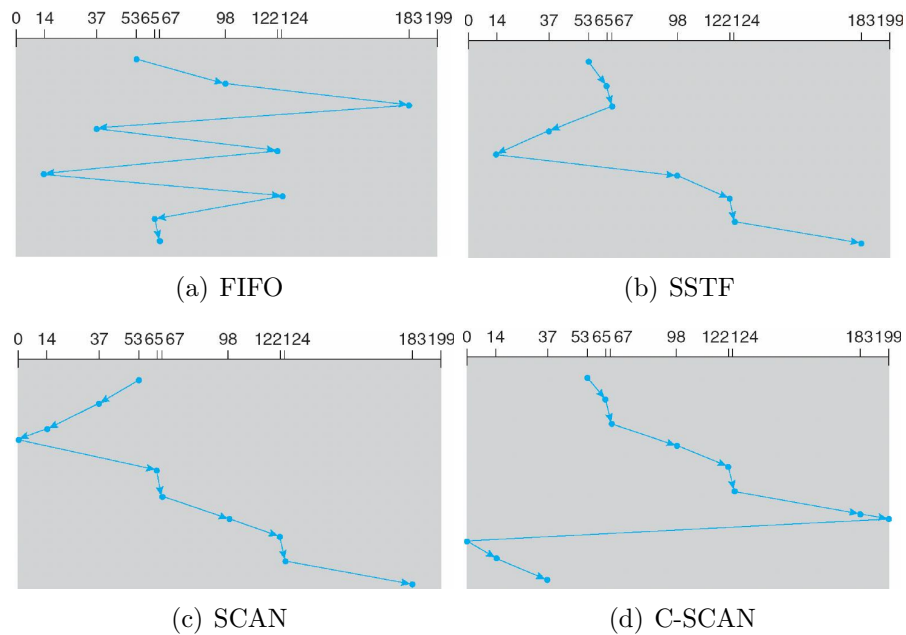


Fig. 47: Disk scheduling met queue = 98, 183, 37, 122, 14, 124, 65, 67 en met de arm op positie 53 bij de start

lang rond dezelfde sectoren hangen, waardoor andere requests onaanvaardbaar lang moeten wachten. Dit probleem stelt zich voornamelijk in systemen waarbij de disk zwaar belast wordt met lees- en schrijfp opdrachten.

In Figuur 47(b) zien we een voorbeeld van SSTF scheduling, waarbij we voor de eenvoud onderstellen dat de arm tegen een constante snelheid beweegt. In totaal worden er slechts 236 tracks afgelopen, in tegenstelling tot de 640 tracks van de FIFO volgorde. Hoewel de SSTF volgorde veel weg heeft van de shortest process next (SPN) scheduling orde, is deze echter niet optimaal, omdat het afhandelen van een eerste request de tijd beïnvloedt die nodig is om de overige requests af te handelen. Het LOOK algoritme, dat we verder bespreken, zal de 8 requests in 208 track wissels afwerken.

SCAN en Cyclical SCAN (C-SCAN): De SCAN policy pakt het starvation effect van SSTF aan door de mogelijke armbeweging te beperken. Namelijk, de arm mag enkel van richting veranderen bij het bereiken van de buiten- of binnenkant van de disk. Studies met random workloads hebben aangetoond

dat SCAN veel lagere response time varianties realiseert, maar wel een iets hogere gemiddelde tijd nodig heeft. De SCAN techniek heeft wel als nadeel dat de tracks in het midden van de disk regelmatig bezocht worden dan deze aan de buiten- of binnenzijde. Dit impliceert dat de performantie kenmerken verschillen op basis van de positie van de data, wat niet wenselijk is. Om de ongelijkheid die SCAN creëert teniet te doen, kan men ook eisen dat de arm wanneer hij de binnenkant van de disk bereikt, onmiddellijk volledig terug moet gaan naar de buitenkant voordat er nieuwe requests worden afgehandeld. Dit is de wijze waarop het C-SCAN algoritme te werk gaat.

LOOK en Cyclical LOOK (C-LOOK): Het LOOK algoritme is een variant van SCAN, waarbij de disk head zal wisselen van richting wanneer er geen requests meer uitstaande zijn in de richting waar de head zich momenteel naar beweegt. Hoewel iets complexer, kan dit aanleiding geven tot een kleine performantie verbetering. C-LOOK is, zoals je kan verwachten, de variant van LOOK die net als C-SCAN de ongelijkheid ten gevolge van de track locatie wegwerkt.

VSCAN(R): Dit algoritme, dat $0 \leq R \leq 1$ als input parameter heeft, zal de request kiezen die het dichtste bij de huidige positie gelegen is. De metriek voor het meten van de afstand is echter niet langer de fysische afstand (zoals bij SSTF). Afhankelijk van het feit of de request een verandering van de richting waarin de arm aan het bewegen was vereist of niet, wordt er bij de fysieke afstand R keer de volledige breedte van de disk opgeteld, zodat de requests die een verandering van richting vereisen als verder weg worden beschouwd. VSCAN(0) is dus equivalent aan SSTF, VSCAN(1) aan LOOK (en niet aan SCAN). Experimenten hebben aangetoond dat VSCAN(0.2) een goed evenwicht geeft tussen lage gemiddelde seek times en de mate waarin starvation optreedt.

FSCAN: Een ander effect dat bij de SCAN methode kan optreden, is *arm stickiness*. Wanneer er veel requests binnen komen voor dezelfde track, dan kan de arm hier lang blijven hangen. Zolang de arm boven deze track gepositioneerd is, zullen alle nieuwe requests voor deze track ook voorrang krijgen op de andere uitstaande requests. Men kan dit deels oplossen door een *two stage* buffer te gebruiken. Alle requests worden hierbij in een eerste buffer verzamelt. Wanneer er een nieuwe set van requests wordt gescheduled, dan

worden alle requests naar een eerste buffer doorgeschoven. Vervolgens wordt de SCAN procedure toegepast op deze set. De volgende te scheduleren set van requests, wordt ondertussen tijdelijk tegengehouden en opgeslagen in de tweede buffer.

Tenslotte moet er ook opgemerkt worden dat hedendaagse computersystemen ook over een disk cache beschikken. Hierdoor zullen opeenvolgende leesoperaties veel sneller verlopen. Het locality of reference principe wordt dan ook voornamelijk door de cache uitgebuit en in mindere mate door de scheduling strategie. Verder kan ook de file allocatie methode de performantie van de disk sterk beïnvloeden.

3 RAID: redundant arrays of independent discs

Een belangrijke ontwikkeling in disk management, die ondertussen is uitgegroeid tot een miljarden business, is de introductie van Redundant Arrays of Independent Discs (RAID) door Patterson, Gibson en Katz in hun 1988 SIGMOD paper. Het onderliggende idee is om een performant en hoge capaciteit disk systeem op te zetten door gebruik te maken van vele *inexpensive* disks. Belangrijk hierbij is wel dat de betrouwbaarheid van zulk een systeem even groot is dan bij een klassieke enkele harddisk. Tenslotte zal in het geval van N disks de gemiddelde tijd tot een failure met een factor N verkleinen.

RAID systemen worden, net zoals in het oorspronkelijke paper, opgedeeld in verschillende *levels*, gaande van RAID 0 tot RAID 7. Verder bestaan er ook multilevel systemen. We starten met de eenvoudigste RAID level, die eigenlijk een AID is (en geen RAID) daar deze geen redundancy bevat.

Wanneer we het over de performantie van een RAID systeem hebben, kijken we voornamelijk naar de lees- en schrijfsnelheid, waarbij we een onderscheid maken tussen *random* reads en *sequential* reads. Random reads bestaan uit vele kleine leesoperaties op verschillende locaties op de (logische) disk. Sequentiële reads zijn langdurige leesoperaties waarbij de seek en rotation time een minder belangrijke rol spelen. Dezelfde opdeling kunnen we ook maken voor writes. Verder wordt ook gekeken naar de robuustheid. Kan het systeem met crashes omgaan, wat is in dit geval de overblijvende performantie en hoe eenvoudig kan het systeem hersteld worden?

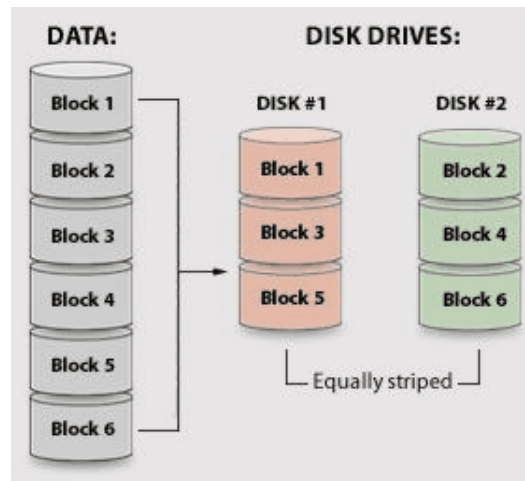


Fig. 48: RAID level 0

3.1 RAID 0

Wanneer we RAID level 0 hanteren, delen we alle data op in een aantal blokken. Deze blokken spreiden we vervolgens over N disks, door op disk i blokken i , $N + i$, $2N + i$, enzovoort te schrijven (zie Figuur 48). De blokken 1 tot N noemt men een *stripe*, evenals blokken $N + 1$ tot $2N$, $2N + 1$ tot $3N$, etc. Vandaar dat men vaak naar RAID 0 verwijst met de term *striping*.

Een belangrijke parameter van striping is de stripe lengte (die gelijk is aan N keer de blok grootte). Grote stripes zijn veelal nuttig om random reads te ondersteunen, omdat er dan, wanneer de controller dit ondersteunt, meerdere (korte) leesoperaties gelijktijdig kunnen plaatshebben (één op elke disk). Kortere stripes, waarbij data van een enkele read over meerdere disks is gespreid, zijn dan weer nuttig voor de sequentiële leesoperatie, doordat een enkele read simultaan kan gebeuren. Merk op dat de seek en rotation time in dit geval iets groter is, omdat we de tijd van de meest verwijderde head moeten rekenen. Ook writes gebeuren in een RAID 0 systeem erg efficiënt, waar dezelfde opmerkingen gelden voor de stripe lengte.

De robuustheid is natuurlijk erg slecht. Als één disk faalt, dan ligt het hele systeem onderuit (nog meer bij korte stripe lengtes). Vandaar dat deze oplossing enkel verantwoord is voor systemen die een zeer hoge performantie vereisen (zonder al te duur te zijn) en waarvan de data niet zo belangrijk is of regelmatig gebackupt wordt. Verder is het ook best dat beide (alle) disks

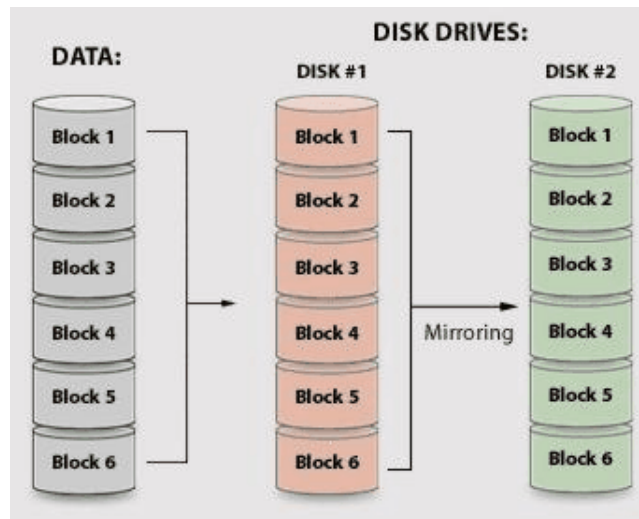


Fig. 49: RAID level 1

dezelfde eigenschappen, bv. opslagcapaciteit, hebben.

3.2 RAID 1

RAID level 1 verschilt erg van level 0 en blinkt vooral uit in robuustheid en in mindere mate in performantie. Het grootste nadeel van deze oplossing is de prijs. Bij RAID 1 wordt alle data op elke disk geplaatst, vandaar dat men hier spreekt van *mirroring* (zie Figuur 49). In de praktijk hanteert men bijna nooit meer dan 2 disks in deze configuratie.

Het is onmiddellijk duidelijk dat dit systeem een goede betrouwbaarheid heeft. Indien een disk faalt, dan blijft het systeem operationeel en werkt als een single disk systeem. Ook de restauratie is vrij eenvoudig, er moet enkel een nieuwe kopij gemaakt worden.

Random leesoperaties in RAID 1 verlopen iets sneller, doordat de load over beide disks kan gespreid worden. Voor sequential reads is er meestal weinig winst tov een enkele disk, doordat een enkele grote leesoperatie op één disk zal gebeuren. Writes moeten op beide disks gebeuren, hierdoor is de write operatie iets trager dan bij single disk systemen. Namelijk, de grootste seek en rotation time van beide disks bepaalt de uitvoertijd. Bij sequentiële writes weegt dit minder door. Wanneer een disk faalt zal de schrijfperformantie dus lichtjes toenemen, in tegenstelling tot de reads.

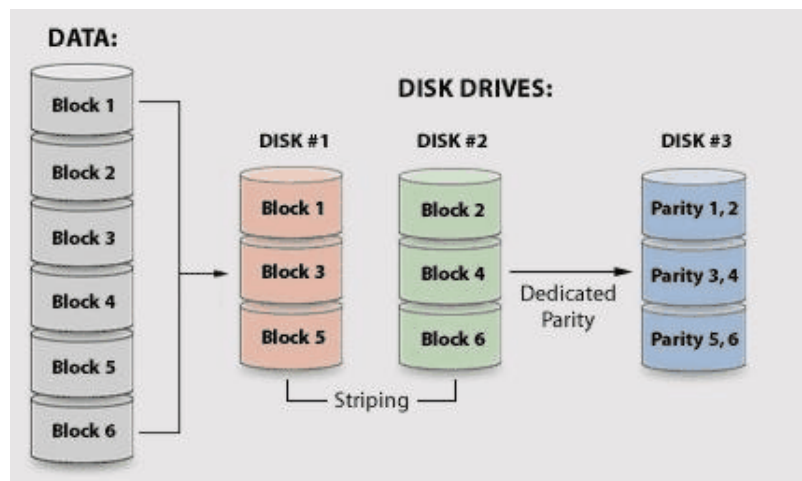


Fig. 50: RAID level 3

De kosten voor RAID 1 systemen zijn erg hoog, waardoor dit vooral gebruikt wordt wanneer de data zeer belangrijk is (bv. financiële data) en performantie een mindere rol speelt.

3.3 RAID 3 en 4

RAID 3 en 4 werken analoog aan RAID 0, wat wil zeggen dat de data over twee (of meer) disks wordt gestripet. Echter, er wordt ook een extra disk voorzien die *parity* data bevat. Parity data is één van de meest eenvoudige vormen van error correction coding. Om de waarde van de parity bits te bepalen, tellen we alle blokken die deel uitmaken van dezelfde stripe bij elkaar op (waarbij $1 + 1 = 0$). De i -de bit van het parity blok is dus 0 (1) indien de som van de i -de bits van elk van de blokken even (oneven) is.

In geval van RAID 3 zijn de stripes zeer kort (minder dan 1024 bytes in een stripe), waardoor elke lees (of schrijfoperatie) alle disks simultaan zal gebruiken. De heads van de disks worden in RAID 3 ook gesynchroniseerd (en bevinden zich dus allemaal op dezelfde track en sector). Sequentiële read operaties verlopen erg snel. Ook writes zijn vrij efficiënt doordat we meestal een hele stripe ineens schrijven, waardoor we de parity data op basis van de te schrijven input kunnen berekenen. Random reads en writes zijn iets minder efficiënt door de zeer korte stripes.

RAID 4 is analoog aan RAID 3, maar hanteert grote stripes en zal de

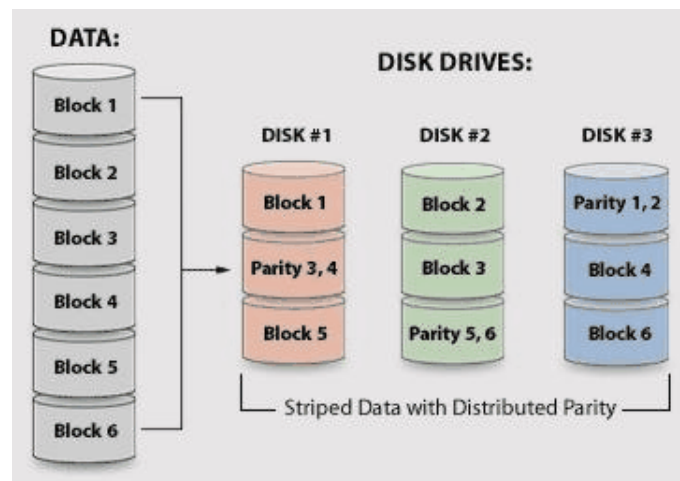


Fig. 51: RAID level 5

heads van de disks niet synchroniseren. Hierdoor verbetert de performantie van de random leesoperaties. Echter, bij een random schrijfoperatie zal vaak maar één blok uit een stripe wijzigen. Dit houdt in dat we eerst de oude en nieuwe data moeten vergelijken en op de plaatsen waar er verschillen zijn, deze bij de oude parity moeten optellen om de nieuwe parity data te bekomen. Een enkele write vergt dus fysisch 2 write en 2 read operaties, wat erg duur is voor systemen met veel writes. Sequentiële reads en writes verlopen iets minder vlot dan bij RAID 3.

De robuustheid van RAID 3 en 4 systemen is vergelijkbaar. Het systeem blijft operationeel wanneer er een disk uitvalt, want de informatie op de disk die faalde, kan gereconstrueerd worden uit de overige data (door de som te nemen van de overige blokken). Echter, wanneer we data lezen moet de ontbrekende data vaak bepaald worden. Dit houdt in dat het opvragen van een enkel blok aanleiding kan geven tot vele leesoperaties (één op elke overblijvende disk). In RAID 3 is dit minder dramatisch, omdat de kleine stripes meestal toch volledig gelezen worden (en de heads gesynchroniseerd zijn). Bij RAID 4 kan het zijn dat de data van een andere disk komt dan deze die faalde. Echter, staat de opgevraagde info wel op deze disk, dan vergt dit veel extra werk. De gevolgen van een RAID 3 falings zijn dus iets minder erg dan bij een RAID 4 falings.

Beide systemen hebben als groot nadeel dat alle parity data zich op dezelfde disk bevindt, waardoor deze bij elke write operatie betrokken wordt.

Indien er veel writes optreden, vormt deze disk dus een potentiële bottleneck. RAID 5 lost dit probleem op. Tenslotte merken we op dat RAID level 2 verschilt van level 3 door gebruik te maken van Hamming codes (ipv parity checks) om de data te beschermen. Hoewel deze oplossing robuuster is, zijn er meer extra disks nodig en vergt de berekening van de controle data veel meer werk. Omwille van deze redenen bestaan er zo goed als geen commerciële RAID 2 oplossingen.

3.4 RAID 5 en 6

RAID level 5 verwijdt de bottleneck aanwezig door de parity disk in RAID 4 systemen door de parity data te spreiden over alle disks (zie Figuur 51). Hierdoor zal de performantie van random writes licht verbeteren. Echter, in geval een disk faalt, is er nog steeds een duidelijke performantie degradatie.

RAID 5 is wellicht de populairste RAID level die we vandaag de dag in computersystemen aantreffen. Hierdoor wordt deze vorm ook financieel aantrekkelijker. Door het spreiden van de parity data kunnen random reads zelfs efficiënter verlopen dan bij een level 0 configuratie (omdat er een disk extra is). De extra kost van een write blijft echter bestaan, maar er is nu ook robuustheid tov level 0. Softwarematige oplossingen van RAID 5 systemen kunnen de performantie ernstig bedreigen door de hoge hoeveelheid parity berekeningen.

RAID level 6 is een uitbreiding van level 5, waarbij nog een extra disk wordt toegevoegd. Naast de parity check, wordt er nog een extra blok aan controle informatie opgesteld per stripe. Hierdoor kan het systeem een dubbele disk falend overleven.

3.5 RAID $X + Y$

Niet lang na de introductie van RAID levels 0 tot 6, werden er configuraties opgesteld die meerdere levels combineerden, om zo de voordelen van beide levels uit te buiten. Dit heeft aanleiding gegeven tot een aantal succesvolle combinaties, die natuurlijk iets complexer zijn in operatie en implementatie.

Wanneer we level X en Y combineren (voor zover dit mogelijk is), dan delen we de data eerst op in sets door gebruik te maken van RAID level Y . In elke set passen we dan RAID level X toe en we noteren dit als RAID $X + Y$. Bijvoorbeeld passen we RAID 0 + 1 toe, dan gaan we eerst *mirroring* toepassen en vervolgens in elke mirror *striping* uitvoeren (zie Figuur 52). De

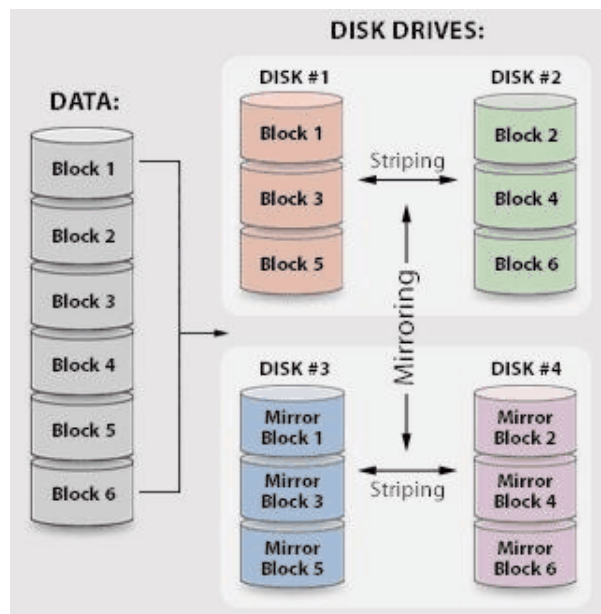


Fig. 52: RAID level 0+1

alternatieve volgorde, zijnde RAID 1 + 0 met eerst striping en dan mirroring, geeft aanleiding tot dezelfde fysieke dataverdeling. Echter, de werking van beide schema's is verschillend (zie verder).

De naamgeving van multiple RAID levels laat zeer te wensen over in de literatuur. Zo wordt RAID $X + Y$ ook genoteerd als XY en X/Y , maar ook als YX , Y/X of zelfs $Y + X$. Het is dus van groot belang bij aankoop na te kijken welke combinatie er werkelijk gehanteerd wordt. De volgorde verschilt zelfs onder producten van dezelfde producent. Daarnaast wordt het $0 + 3$ schema soms ook als 53 aangeduid, wat helemaal tot verwarring leidt.

RAID 0 + 1 en 1 + 0

Wanneer we RAID 0 en 1 combineren, dan doen we dit veelal om de prestatie van RAID 0 te koppelen aan de betrouwbaarheid van RAID 1. Stel dat we bijvoorbeeld 10 disks hebben. Bij RAID 0 + 1 nemen we eerst een mirror van alle data en verdelen vervolgens elke mirror apart over 5 disks door striping toe te passen. Wanneer er een lees- of schrijfoperatie plaatsheeft, zal de controller een mirror kiezen en uit deze mirror door striping de data lezen of schrijven.

Wanneer we RAID 1 + 0 toepassen, verlopen reads en writes iets anders. De data is opnieuw verdeeld over 5 disks en gedupliceerd, maar nu zal de controller eerst striping gebruiken en in tweede instantie voor alle betrokken blokken uit de stripe een mirror selecteren.

Wat de performantie betreft, is er weinig verschil tussen beide volgor-des. Leesoperaties zijn zeer efficiënt, schrijfoperaties iets minder doordat elke write op twee plaatsen moet gebeuren. Het grote verschil zit echter in de robuustheid. Wanneer we $2N$ disks hebben en 1 disk faalt bij RAID 0 + 1 dan zal de controller alle requests afhandelen via de mirror waarin geen fout is opgetreden. De performantie neemt dus sterk af want alle andere disks die eveneens deel uitmaken van de mirror waarin de falende plaatsvond, worden niet meer gebruikt. Wanneer er vervolgens nog een disk faalt in de overgebleven mirror, dan is het systeem onbruikbaar geworden (maar vaak nog wel herstelbaar).

RAID 1 + 0 is robuuster, want bij het uitvallen van een disk zullen alle overige $2N - 1$ disks nog wel in gebruik blijven (omdat de mirror in tweede instantie wordt gekozen voor elk blok). Meer zelfs, zolang er geen twee disks falen die dezelfde data bevatten, kan het systeem operationeel blijven. RAID 1 + 0 is dus veel robuuster, omdat de RAID 0 + 1 controller niet voldoende intelligent is om de overige disks in de gefaalde mirror te gebruiken.

RAID 0 + 5 en 5 + 0

Wanneer we beschikken over $N * M$ disks, met $N \geq 2$ en $M \geq 3$, dan kunnen we een RAID 0 + 5 of 5 + 0 opstelling maken. Bij RAID 0 + 5 hebben we M RAID 0 sets van elk N disks, die samen een RAID level 5 vormen. Of nog, we maken eerst M logische disks waarop we RAID 5 toepassen en verdelen vervolgens de inhoud van elke logische disk over N RAID 0 disks. Voor RAID 5 + 0 daarentegen vormen we N RAID 5 sets met M disks elk.

Write operaties zijn door de parity ietwat trager dan bij RAID 0 + 1 of 1 + 0, maar de disks worden wel efficiënter gebruikt. Namelijk, bij RAID 0 + 1 of 1 + 0 wordt de opslagcapaciteit slechts voor de helft benut, terwijl RAID 0 + 5 en 5 + 0 een gebruik heeft van $(M - 1)/M$. Het aantal disks kan vaak wel op meerdere manieren geschreven worden als $N * M$. Stel dat we over $21 = 3 * 7 = 7 * 3$ disks kunnen beschikken. Dan kunnen we bij RAID 5 + 0 ofwel 7 RAID 5 sets vormen van elk 3 disks of 3 RAID 5 sets met elk 7 disks. In het eerste geval is de capaciteit voor 67% benut, in geval twee voor 86%. De keuze met meer overhead is natuurlijk wel robuuster.

Er zijn nog heel wat andere nuttige combinaties zoals $0 + 3$, $3 + 0$, $1 + 5$ of $5 + 1$, die elk door hun eigen voor- en nadelen gekenmerkt worden.

CONCURRENCY

1 Mutual exclusion, deadlocks en starvation

Zoals al eerder aangegeven zal een hedendaags computersysteem zich niet beperken tot het gelijktijdig uitvoeren van een enkel proces, maar zal het een hele reeks processen en threads beheren. In de context van een enkele processor spreken we dan van *multiprogramming*, terwijl we in het geval van meerdere processoren de term *multiprocessing* hanteren. In beide gevallen moeten deze processen dan ook tal van algemene *resources* delen, zoals I/O devices, geheugen, processortijd, etc. Hierbij is het vaak ook van belang dat meerdere processen niet gelijktijdig over een resource kunnen beschikken, om een correct verloop van de programma's te garanderen. Bekijk volgend eenvoudig voorbeeld:

```
procedure echo;  
var outp, inp:  character;  
begin  
    input (inp, keyboard);  
    outp := inp;  
    output (outp,display);  
end
```

We veronderstellen dat meerdere processen gebruik zullen maken van deze eenvoudige procedure en we plaatsen ze dan ook in het geheugen dat gedeeld wordt door alle processen, zodat we slechts een enkele kopij van de procedure in het geheugen moeten opslaan. Dit betekent ook dat de variabelen **inp** en **outp** eveneens gedeeld worden door alle processen. Stel nu dat een eerste proces P0 de *echo* procedure oproept en vlak na het inlezen van de variabele **inp** onderbroken wordt. Wanneer even later een tweede proces P1 ook de procedure *echo* oproept, zal de variabele **inp** overschreven worden. Indien nog later proces P0 hervat wordt, zal het verkeerde karakter op het scherm

verschijnen. Merk op dat hetzelfde probleem ook kan voorvallen indien P0 niet onderbroken wordt, maar er een tweede processor aanwezig is waarop proces P1 actief is.

In dit voorbeeldje zien we dat het *interleaven* van beide processen aanleiding geeft tot een foutief verloop, namelijk, het eerste proces P0 wordt op een ongelukkig moment onderbroken. Om een correcte werking te garanderen mag een volgend proces pas gebruik maken van de procedure *echo*, indien de voorgaande oproep volledig afgelopen is. Deze procedure noemen we daarom een *kritische sectie* van het proces. Het voorkomen dat meerdere processen gelijktijdig in de kritische sectie aanwezig zijn, wordt doorgaans aangeduid met de term *mutual exclusion* (wederzijdse uitsluiting). Naast de mogelijke problemen die kunnen ontstaan doordat meerdere processen een kritische sectie gelijktijdig betreden, kunnen er zich ook andere problemen stellen.

Bijvoorbeeld, wanneer twee processen P0 en P1 beide resources R0 en R1 nodig hebben voor een bepaalde operatie. Dan kan het voorvallen dat P0 over R0 kan beschikken, terwijl P1 resource R1 reeds ter zijner beschikking heeft. Beide processen zijn dus geblokkeerd en wachten eigenlijk op elkaar, echter geen van beide zal ooit verder geraken tenzij de andere zijn resource weer vrijgeeft. In dit geval spreken we van een *deadlock*. Tenslotte kan het ook gebeuren dat drie (of meer) processen een resource delen en dat twee van hen om de beurt kunnen beschikken over de resource. In dit geval komt het derde proces dus nooit tot uitvoering en spreken we van *starvation*. Het is de taak van het operating systeem om ervoor te zorgen dat alle processen tot een correcte uitvoering komen, ongeacht de snelheid waarmee andere processen worden uitgevoerd.

Er bestaan verschillende technieken om mutual exclusion te realiseren. Men kan het probleem softwarematig aanpakken door te stellen dat de processen zelf maar moeten instaan voor het realiseren van mutual exclusion. Daarnaast kan men ook hardwarematige oplossingen ontwikkelen. Tenslotte kan men een oplossing aanbieden via het operating systeem of de programmeertaal die de gebruiker hanteert. We zullen starten met enkele softwarematige oplossingen te bekijken, daar deze historisch ook het eerst verschenen.

2 Softwarematige oplossingen

De eerste technieken voor het realiseren van mutual exclusion tussen *twee* processen werden in de jaren 60 van de vorige eeuw ontwikkeld door Dekker. In deze sectie behandelen we het algoritme van Dekker door stapsgewijs tot een oplossing te komen. Daarna kijken we naar een eenvoudigere oplossing bedacht door Peterson begin jaren 80.

2.1 Algoritme van Dekker

Wanneer we nadenken over hoe we mutual exclusion moeten realiseren, dan lijkt de oplossing erg voor de hand te liggen. Voor elke kritische sectie houden we een globale variabele bij die aangeeft of er momenteel een proces aanwezig is in de kritische sectie. Wanneer deze variabele aangeeft dat er een proces aanwezig is, moeten alle overige processen wachten. Is de kritische sectie vrij, dan kan het proces de sectie binnentreden nadat het de globale variabele heeft aangepast. Tenslotte zal het proces bij het verlaten van de sectie wederom de variabele aanpassen om aan te geven dat de sectie weer vrij is.

Dit geeft echter aanleiding tot problemen: om de sectie binnen te treden moet het proces eerst testen of de sectie vrij is en indien dit het geval is zal het de variabele aanpassen en binnentreden. Echter wanneer het proces precies onderbroken wordt nadat het zag dat de sectie vrij is, dan zou een ander proces eveneens kunnen starten met dezelfde procedure en vaststellen dat de sectie vrij is. Daar het eerste proces de variabele nog niet heeft aangepast, kunnen beide processen in de kritische sectie verzeild geraken. Kortom, omdat een proces tussen de test en de schrijfactie onderbroken kan worden, is mutual exclusion niet gegarandeerd.

Daarnaast stelt zich nog een tweede nadeel: als de sectie bezet is, moeten de wachtende processen regelmatig nagaan of de sectie ondertussen weer vrij is. Dit herhaaldelijk controleren vergt processortijd en noemt men *busy waiting*.

Eerste poging: Een eerste idee is om een globale variabele `turn` te hanteren die aangeeft welk van de twee processen als volgende de kritische sectie mag betreden. Dus zolang `turn` aangeeft dat het andere proces aan de beurt is, moeten we busy wachten tot de sectie vrijkomt. Wanneer een proces de sectie verlaat zal de beurt overgaan naar het andere proces (de variabele `turn`

wordt dus aangepast):

PROCESS 0

```
:  
while turn  $\neq$  0 do {nothing};  
<critical section>  
turn := 1;  
:
```

PROCESS 1

```
:  
while turn  $\neq$  1 do {nothing};  
<critical section>  
turn := 0;  
:
```

Hoewel deze oplossing correct is, behelst zij enkele serieuze nadelen. Ten eerste moeten beide processen om de beurt de sectie betreden, wat maakt dat het snellere proces, dat veel in de sectie moet zijn, vertraagd wordt. Daarnaast moet de variabele **turn** ook worden geïnitieerd, waardoor één van beide processen bevoordeeld wordt.

Tweede en derde poging: In plaats van gebruik te maken van een enkele globale variabele kunnen we ook trachten een variabele bij te houden voor elk van de twee processen (**flag[0]** en **flag[1]**). Om de kritische zone te betreden, controleert het proces eerst of het andere proces ook deze intentie heeft door naar zijn variabele te kijken. Is dit niet het geval dan zetten we de **flag** van dit proces op *true* en gaan we de kritische sectie binnen. Bij het verlaten van de sectie zet het proces zijn **flag** weer op *false*. De globale variabelen worden op *false* geïnitieerd:

PROCESS 0

```
:  
while flag[1] do {nothing};  
flag[0]:=true;  
<critical section>  
flag[0]:=false;  
:
```

PROCESS 1

```
:  
while flag[0] do {nothing};  
flag[1]:=true;  
<critical section>  
flag[1]:=false;  
:
```

Deze oplossing schiet ernstig tekort: wanneer een proces vlak na de **while** lus onderbroken is, kan ook het andere proces de **while** lus vervolledigen, daar de **flag** van het eerste proces nog steeds *false* aangeeft. Er is dus geen sprake van mutual exclusion. Ook het omwisselen van de **while** lus en de assignment brengt geen soelaas:

PROCESS 0

```
:
flag[0]:=true;
while flag[1] do {nothing};
<critical section>
flag[0]:=false;
:
```

PROCESS 1

```
:
flag[1]:=true;
while flag[0] do {nothing};
<critical section>
flag[1]:=false;
:
```

In dit geval geven we dus eerst aan dat we de kritische sectie willen betreden, voor we controleren of de andere ook deze intentie heeft. In dit geval kan er een deadlock optreden wanneer een proces wordt onderbroken vlak nadat het zijn vlag op *true* heeft gezet, waardoor het andere proces hetzelfde kan doen en beide processen vast zitten in de while lus.

Vierde poging: De oplossing lijkt zich nu aan te dienen: wanneer we merken dat het andere proces eveneens zijn vlag heeft gezet, dan zetten we onze vlag even terug op *false* en wachten een tijdje voor we ze weer op *true* zetten. Zo geven we het andere proces even de kans om de kritische sectie te betreden:

PROCESS 0

```
:
flag[0]:=true;
while flag[1] do
    flag[0]:=false;
    <delay a short time>
    flag[0]:=true;
end;
<critical section>
flag[0]:=false;
:
```

PROCESS 1

```
:
flag[1]:=true;
while flag[0] do
    flag[1]:=false;
    <delay a short time>
    flag[1]:=true;
end;
<critical section>
flag[1]:=false;
:
```

Deze oplossing is nog niet ideaal, want er is geen bovengrens op de wachttijd van beide processen. Het kan namelijk gebeuren dat door een erg ongelukkige samenloop van omstandigheden, beide processen aan elkaar voorrang willen geven: ze zetten hun vlag vlak na elkaar op *false*, wachten beide even om de andere te laten voorgaan en zetten dan weer vlak na elkaar hun vlag op *true*, om aan te geven dat ze beide de kritische sectie willen betreden.

Finale oplossing: De oplossing bestaat erin om nog een enkele variabele `turn` toe te voegen die aangeeft welk van de twee processen hoffelijk zal zijn indien blijkt dat beide hun vlag op *true* hebben gezet (`turn = 1` geeft aan dat proces 0 hoffelijk is).

Laten we de correctheid van deze code eens nagaan op een vrij formele wijze. Als preconditionie tot de kritische sectie van proces P0 geldt dan duidelijk dat

$$\text{flag}[0] \wedge (\neg \text{flag}[1] \vee (\text{turn}=0)),$$

terwijl de preconditionie voor P1 luidt dat

$$\text{flag}[1] \wedge (\neg \text{flag}[0] \vee (\text{turn}=1)).$$

Deze beide condities kunnen niet gelijktijdig opgaan, waardoor mutual exclusion gegarandeerd wordt.

PROCESS 0

```

:
flag[0]:=true;
while flag[1] do
  if turn = 1 then
    flag[0]:=false;
    while turn = 1 do {nothing};
    flag[0]:=true;
  end;
end;
<critical section>
turn:=1;
flag[0]:=false;
:

```

PROCESS 1

```

:
flag[1]:=true;
while flag[0] do
  if turn = 0 then
    flag[1]:=false;
    while turn = 0 do {nothing};
    flag[1]:=true;
  end;
end;
<critical section>
turn:=0;
flag[1]:=false;
:

```

Deadlocks kunnen ook niet optreden, namelijk, stel dat proces P0 in de binnenste while loop verzeild geraakt, dan moet P1 zijn vlag *true* zijn en `turn = 1` zijn. Kortom, P1 is of heeft de intentie om de kritische sectie te betreden. Als P1 zich reeds in deze zone bevindt, dan zal hij bij het verlaten `turn` op nul zetten en zo P0 uiteindelijk bevrijden uit de lus. Indien P1 de kritische zone nog moet betreden, dan zal hij dit zonder probleem kunnen doen daar `turn` gelijk is aan 1 en de vlag van P0 gelijk is aan *false*. Ook nu

zal P0 de lus kunnen verlaten nadat P1 de kritische sectie heeft verlaten en turn op nul heeft gezet. Merk op dat wanneer de twee processen beide de kritische sectie wensen te betreden, het proces dat het minst recent nog tot de sectie toegang had, voorrang zal krijgen.

2.2 Algoritme van Peterson

In 1981 heeft Peterson een elegantere oplossing voor het probleem van de twee processen naar voor gebracht. Het idee is dat een proces eerst kenbaar maakt dat het de kritische zone wenst te betreden door zijn vlag op *true* te zetten en vervolgens de *turn* gelijk stelt aan het andere proces. Daarna wordt getest of het andere proces de kritische zone wil betreden. Zo niet dan kan het proces zelf de sectie ingaan, anders wacht het tot de *turn* variabele gelijk is aan zijn nummer. Dus, wanneer de twee processen beide de kritische zone in wensen te gaan, zal het proces dat als laatste de *turn* variabele wijzigde voorrang geven aan het andere proces. Het proces dat dus eerst de while lus bereikt zal winnen, wat ook garandeert dat daarna het andere proces toegang zal verkrijgen voordat hetzelfde proces wederom tot de sectie kan overgaan. Natuurlijk, als slechts één proces wenst gebruik te maken van de kritische sectie, kan het dit veelvuldig doen zonder dat het andere proces aan de beurt komt.

PROCESS 0

```
:
flag[0]:=true;
turn = 1;
while flag[1] and turn = 1 do {nothing };
<critical section>
flag[0]:=false;
:
```

Het algoritme van Peterson laat zich ook eenvoudiger veralgemenen naar n processen in vergelijking met het algoritme van Dekker. Deze veralgemening bestaat erin dat elk proces $n - 1$ fases moet doorlopen voor het de kritische sectie mag betreden. Voor $n = 2$ hebben we dus slechts één fase die precies overeenstemt met de drie lijnen uit de code. De code voor n processen maakt gebruik van een array *flag*, waar *flag[i]* aangeeft in welke fase

proces i zich bevindt (deze is nul als het proces de kritische zone niet wenst te betreden). Daarnaast is er ook een array van $n - 1$ elementen genaamd **turn**, waarvan **turn[j]** aangeeft welk proces als laatste de tweede lijn uit fase j heeft uitgevoerd. Een proces zal overgaan naar de volgende fase als blijkt dat geen enkel ander proces zich in deze fase bevindt of wanneer een ander proces later de fase betrad (merk op dat er geen problemen zijn wanneer het proces wordt onderbroken midden in de wait test):

```

PROCESS i
:
for j = 1 to n-1 do
    flag[i] := j;
    turn[j] := i;
    wait until ( $\forall k \neq i, \text{flag}[k] < j$ ) or (turn[j]  $\neq i$ );
end;
<critical section>
flag[i] := 0;
:
```

In geval van $n > 2$ processen kan een proces wel een onbegrensd aantal keer voorbij gestoken worden door andere processen. Hoewel dit algoritme eenvoudig is, wordt er nog wel steeds gebruik gemaakt van busy waiting, namelijk, de processen moeten herhaaldelijk controleren of ze een fase verder kunnen gaan. Dit vergt heel wat inspanning van de processor die verloren gaat. Vandaar dat men ook hardwarematige oplossingen heeft ontwikkeld.

3 Hardwarematige oplossingen

Een eerste eenvoudige hardware oplossing bestaat erin om interrupts te verhinderen wanneer een proces zich in een kritische sectie bevindt. Dit is een werkende oplossing, maar wanneer de tijd nodig om de kritische sectie af te werken groot is, of wanneer het proces op een event moet wachten tijdens zijn kritische sectie, geeft dit aanleiding tot een erg ongelijk en inefficiënt gebruik van de processor. Daarnaast lost dit enkel het probleem in het multiprogramming scenario op en niet in het geval van meerdere processoren.

Een betere optie is gebruik te maken van speciale machine instructies. Twee belangrijke voorbeelden zijn de *test and set* instructie en de *exchange* instructie. De eerste instructie laat toe een test gevolgd door een write opera-

tie uit te voeren als een enkele instructie en dus als atomaire operatie die niet onderbroken kan worden. Als we terugdenken aan onze eerste bespiegelingen over concurrency, dan wensten we een enkele globale variabele te hanteren om een kritische sectie te beschermen. Het probleem hierbij was dat een proces, nadat het keek of de sectie bezet was, onderbroken kon worden, waardoor andere processen dezelfde test konden ondergaan voordat het eerstgenoemde proces de sectie had opgeëist. Door de *test and set* instructie kunnen we deze eenvoudige aanpak nu wel hanteren en dit voor een willekeurig aantal processen. Initieel zetten we de variabele `bolt` op *false*, voor een proces de kritische sectie ingaat test het eerst of `bolt false` is, en zo ja, dan zet het deze op *true*. Bij het verlaten van de kritische zone wordt `bolt` wederom op *false* gezet. In de onderstaande code zal de `testset` instructie de waarde van de `bolt` variabele bij de start van de instructie als return waarde teruggeven:

```
PROCESS i
:
while (testset(bolt)) do {nothing};
<critical section>
bolt:=false;
:
```

Een gelijkaardige oplossing kan ook door de *exchange* instruction worden bekomen. Deze instructie laat toe de inhoud van een register en een geheugenplaats te verwisselen. Het idee voor mutual exclusion te realiseren bestaat er dan in om elk proces een lokale variabele `key` te geven, die op nul wordt geïnitieerd. Daarnaast is er een enkele globale variabele `bolt`, die we op 1 initialiseren. Wanneer een proces nu de kritische sectie binnen wenst te gaan, voert het eerst een *exchange* uit op de `key` en `bolt` variabele, totdat de `key` gelijk is aan 1. Wanneer een proces deze operatie uitvoert terwijl een ander proces in de kritische sectie is, zal het enkel twee nullen wisselen. Bij het verlaten van de sectie wordt wederom een *exchange* operatie doorgevoerd.

Beide oplossingen stemmen overeen met een zogenaamde *spinlock*, zijn erg eenvoudig, toepasbaar voor vele processen en kritische zones en functioneren zowel in een multiprogramming en multiprocessor context (hoewel een implementatie van de testset instructie in een multiprocessor omgeving niet vanzelfsprekend blijkt). Toch zijn er ook enkele ernstige nadelen aan verbonden. Ten eerste wordt er opnieuw gebruik gemaakt van busy waiting. Daarnaast is er geen garantie dat er geen starvation kan optreden, een proces

kan een willekeurige tijd worden uitgesloten doordat andere processen steeds de kritische zone opeisen. Tot slot kan er zelfs een deadlock optreden in geval dat de processor strikte prioriteiten hanteert bij het scheduleren van de processen (of threads).

4 Semaforen

Dijkstra realiseerde een grote vooruitgang op het vlak van concurrency met de ontwikkeling van semaforen. Een semafoor kan men beschouwen als een signaal dat toelaat processen te coördineren. Een proces kan zulk een signaal versturen door middel van de *signal* operatie, verder kan het ook een signaal ontvangen door de *wait* operatie. Wanneer het te ontvangen signaal nog niet verstuurd is, zal het proces geblokkeerd worden tot een ander proces het overeenkomstige signaal verstuurt. Een semafoor kan men dan ook zien als een *record* bestaande uit een teller en een lijst van processen. Op de teller kan men drie operaties uitvoeren:

- De teller kan men initialiseren op een niet negatieve waarde.
- De teller van de semafoor *s* kan men met één verlagen via de *wait(s)* operatie. Indien de teller hierdoor een negatieve waarde krijgt, zal het proces dat de operatie uitvoerde, geblokkeerd worden en toegevoegd worden aan de lijst van geblokkeerde processen.
- De teller kan men met één verhogen via de *signal(s)* operatie. Wanneer de bekomen waarde van de teller niet positief is, wordt er ook een proces uit de lijst verwijderd, of nog gedeblokkeerd.

Merk op dat de teller dus bijhoudt hoeveel processen er nog een signaal kunnen ontvangen zonder dat ze geblokkeerd geraken. Indien de teller negatief is, geeft zijn absolute waarde het aantal geblokkeerde processen aan. In Figuur 53 zien we dit alles nogmaals in pseudo code.

Naast deze semaforen, soms *counting semaphores* genaamd, bestaan er ook binaire semaforen die verschillen van de voorgaande doordat de teller slechts de waarden 0 en 1 kan aannemen. Bij een *waitB* signaal wordt nagegaan of de teller op 1 staat, zo ja dan verlagen we die naar nul en gaan we verder, anders wordt het proces geblokkeerd. Een *signalB* oproep zal, indien de *s.queue* leeg is, de teller op 1 zetten en anders een proces uit de lijst deblokken. Hoewel binaire semaforen minder algemeen lijken, kan

```
type semaphore = record
    count: integer;
    queue: list of process
end;
var s: semaphore;

wait(s);
    s.count := s.count - 1;
    if s.count < 0
    then begin
        place this process in s.queue;
        block this process
    end;

signal(s);
    s.count := s.count + 1;
    if s.count ≤ 0
    then begin
        remove a proces P from s.queue;
        place proces P on the ready list
    end;
```

Fig. 53: Definitie van een semafoor

men bewijzen dat ze even krachtig zijn dan algemene semaforen (dit zullen we mogelijk tijdens de oefeningen aantonen). Het idee is een nieuwe teller als globale variabele te creëren en de access hiertoe met binaire semaforen te beschermen. Het belang van binaire semaforen is dat deze in bepaalde hardware architecturen eenvoudiger te implementeren/ondersteunen zijn.

4.1 Mutual exclusion met semaforen

Mutual exclusion is eenvoudig te realiseren door gebruik te maken van semaforen. Voor elke kritische zone definiëren we een semafoor die we initialiseren op 1. Telkens als een proces een kritische zone wil ingaan, voert het eerst de *wait(s)* operatie uit, waarbij *s* de semafoor is die bij de kritische sectie hoort. Bij het verlaten van de sectie wordt de *signal(s)* operatie uitgevoerd. Wanneer nu meerdere processen de kritische sectie wensen te betreden, zal

er maar één proces slagen daar de andere allen geblokkeerd raken door de *wait(s)* operatie. Een noodzakelijke vereiste hierbij is wel dat de *wait* en *signal* operaties niet onderbroken kunnen worden of dat er tenminste geen twee processen gelijktijdig een *wait* of *signal* operatie kunnen uitvoeren, anders kan de mutual exclusion eigenschap verloren gaan.

4.2 Het producer/consumer probleem met semaforen

Een veel voorkomend probleem bij concurrente processen is het zogenaamde *producer/consumer* probleem. Bij dit probleem zijn er enerzijds één of meerdere processen die data genereren en wegschrijven in een buffer, terwijl één ander proces over deze data wenst te beschikken en ze bijgevolg uitleest uit de buffer. Voor de eenvoud stellen we dat de buffer oneindig groot is. De vereiste aanpassing voor een eindige buffer is vrij eenvoudig.

De producer en consumer functies zouden er als volgt kunnen uitzien:

<pre>producer: repeat produce item v; b[in] := v; in := in + 1; forever;</pre>	<pre>consumer: repeat while in ≤ out do {nothing}; w := b[out]; out := out + 1; consume item w; forever;</pre>
--	--

We zien dat we enerzijds de globale variabelen *in* en *out* moeten beschermen en ook moeten voorkomen dat de buffer door meerdere processen gelijktijdig wordt aangesproken. Tenslotte mag de buffer ook niet gelezen worden als hij leeg is. Voor de eenvoud stellen we *n* gelijk aan *in* - *out*. Het gebruik van twee semaforen ligt voor de hand: een eerste *s* die mutual exclusion van de buffer en de teller *n* moet waarborgen en een tweede *ne* die aangeeft wanneer de buffer iets bevat:

<pre> producer: repeat produce; waitB(s); append; n := n + 1; if n = 1 then signalB(ne); signalB(s); forever; </pre>	<pre> consumer: waitB(ne); repeat waitB(s); take; n := n - 1; signalB(s); if n = 0 then waitB(ne); consume; forever; </pre>
--	---

Deze oplossing lijkt sluitend te zijn, maar er is echter een probleem. De consumer moet een *waitB* operatie uitvoeren op de semafoor *ne* indien hij de teller *n* tot nul verlaagde. Echter, hij kan dit niet doen voor hij de *signalB(s)* operatie heeft uitgevoerd, anders treedt er een deadlock op. Hierdoor kan de consumer onderbroken worden voor hij de waarde van *n* kan raadplegen. Indien een producer tussendoor een element toevoegt, kan de consumer daarna twee elementen uit de buffer halen, terwijl er maar één aanwezig is. Dit kan eenvoudig opgelost worden door de waarde van *n* even op te slaan in een lokale variabele *m* zodat de producer procedure deze niet kan aanpassen.

Belangrijk is ook op te merken dat de aanwezigheid van de *ne* semafoor ervoor zorgt dat deze oplossing niet bruikbaar is indien er meerdere consumers actief zouden zijn. Bijvoorbeeld, indien er 10 items geproduceerd worden en twee consumers wensen elk vijf items uit te lezen, dan kan het zijn dat slechts één van de consumers in zijn opzet slaagt (omdat er mogelijk maar één keer een *signal(ne)* operatie wordt uitgevoerd).

De voorgaande oplossing wordt echter nog eenvoudiger met algemene semaforen. Nu kunnen we naast de *s* semafoor ook *n* als semafoor definiëren. Dit geeft aanleiding tot volgende code:

<pre> producer: repeat produce; wait(s); append; signal(s); signal(n); forever; </pre>	<pre> consumer: repeat wait(n); wait(s); take; signal(s); consume; forever; </pre>
--	--

Hierbij is de volgorde van de twee *wait* operaties van cruciaal belang. Indien hun volgorde omgewisseld was, dan zou er een deadlock kunnen op-

treden doordat een consumer in de kritische zone mogelijk wordt geblokkeerd. Kunnen we deze code ook hanteren indien er meerdere consumers actief zijn?

4.3 Semaforen implementeren

Het succes van semaforen staat of valt met het voorkomen dat meerdere processen gelijktijdig een *wait* of *signal* operatie uitvoeren. Met de *test and set* instructie kunnen we dit eenvoudig realiseren door een variabele `flag` aan de record in Figuur 53 toe te voegen die op *false* wordt geïnitieerd. Zowel de *wait* als *signal* operatie zullen dan starten met een repeat die geen instructies bevat tot `testset(s.flag)` vals is. Op het einde van beide operaties moet de variabele `s.flag` terug op *false* worden gezet (evenals wanneer een proces geblokkeerd wordt).

We hebben hier wel te maken met een vorm van busy waiting, maar de *wait* en *signal* instructie zijn vaak van zeer korte duur. Daarnaast is busy waiting soms ook sneller dan signaling (want het proces blijft in de ready queue). Dit is voornamelijk het geval wanneer er meerdere processoren zijn, want dan zal het wachtende proces soms maar enkele cycli in de processor busy blijven, omdat een ander proces via een andere processor de semafoor weer kan vrijgeven.

Om een snelle implementatie te realiseren van een semafoor, zal er vaak een enkele pointer worden toegevoegd aan het *proces control block* (zie Hoofdstuk 1) van elk proces. De lijst van geblokkeerde processen wordt dan ondersteunt door de semafoor zelf te laten wijzen naar het eerste wachtende proces. Dit proces zal via de pointer in het control block wijzen naar het tweede wachtende proces, enz. Een enkele pointer per proces is voldoende aangezien een proces nooit gelijktijdig op meerdere semaforen moet wachten. Verder zal de semafoor meestal ook over een pointer beschikken die wijst naar het laatste wachtende proces, zodat nieuwe processen die door de semafoor geblokkeerd worden ook snel aan de lijst kunnen worden toegevoegd. Bij goedgeschreven implementaties zullen er vaak maar een tiental instructies nodig zijn bij een *wait* of *signal* operatie. Dit geeft ook aan waarom het gebruik van busy waiting in deze context aanvaardbaar is.

Men zal tijdens de uitvoering van de *wait* of *signal* operatie ook tijdelijk de interrupts uitschakelen zodat deze steeds afgewerkt worden. Merk op, het tijdelijk uitschakelen van de interrupts is op zich al voldoende op een systeem uitgerust met een enkele processor. In geval van meerdere processoren is de combinatie met een hardware instructie zoals de *test and set* instructie wel

noodzakelijk.

Het tijdelijk uitschakelen van interrupts kan wel gevolgen hebben voor de clock die het operating systeem bijhoudt. Een clock wordt namelijk vaak geïmplementeerd door op regelmatige tijdstippen een interrupt te genereren die de clock met één verhoogt. Stel dat de interrupts echter zijn uitgeschakeld en dat er ondertussen meerdere interrupts binnenkomen die allen als doel hebben de clock met één te verhogen. Omdat deze interrupts allemaal van hetzelfde type zijn, zal de processor, wanneer de interrupts opnieuw toegelaten zijn, gewoon opmerken dat er een interrupt was van dit type en de bijhorende interrupt handler een enkele maal oproepen. Er wordt dus niet bijgehouden hoeveel maal een bepaalde interrupt werd gegenereerd, waardoor de clock enkele *ticks* kan verliezen.

5 Het readers/writers probleem

Eén van de meest fundamentele concurrency problemen is het zogenaamde *Readers/Writers* probleem. Dit probleem stelt dat er een aantal readers zijn die data wensen te lezen (b.v. de catalogoog van de bibliotheek), daarnaast kan de data ook aangepast worden door writers. Readers mogen probleemloos gelijktijdig de data accessen, echter wanneer een writer de data wil aanpassen, moet hij als enige toegang hebben tot de data, zodat er geen readers zijn die onzinnige resultaten te zien krijgen. We gaan twee oplossingen bekijken die gebruik maken van semaforen. In de eerste, eenvoudigste oplossing geven we voorrang aan de readers: zolang er data gelezen wordt, moeten de writers wachten.

Hoe kunnen we dit implementeren? We moeten duidelijk het aantal readers bijhouden, daar een writer moet weten of er nog readers aanwezig zijn. Dit kunnen we niet eenvoudig door de `s.count` van een semafoor doen, daar meer readers juist wil zeggen dat de writers extra moeten wachten. Een globale variabele `rcount` lijkt dus aangewezen met een bijhorende semafoor `x` om de mutual exclusion tot deze variabele te garanderen. Verder hebben we een semafoor `wsem` nodig die de writers verhindert om te schrijven zolang er lezers zijn (deze wordt zoals de overige semaforen op één geïnitieerd). Wanneer moeten we deze semafoor gebruiken? Indien een reader de teller `rcount` tot 0 verlaagt, dan geven we een `signal(wsem)`, dat aangeeft dat de writers kunnen beschikken over de data. Echter, meerdere readers kunnen even komen lezen voordat er een writer komt. Dus wanneer we enkel dit *sig-*

nal(wsem) geven krijgen we problemen. De oplossing bestaat erin om readers ook een *wait(wsem)* te laten uitvoeren wanneer ze de teller **rcount** verhogen tot één. Een reader zal hierop enkel blokkeren als er effectief een proces aan het schrijven is:

procedure reader:

```
wait(x);
rcount := rcount+1;
if rcount = 1 then wait(wsem);
signal(x);
READ
wait(x);
rcount := rcount-1;
if rcount = 0 then signal(wsem);
signal(x);
```

procedure writer:

```
wait(wsem);
WRITE;
signal(wsem);
```

Deze oplossing kan natuurlijk aanleiding geven tot starvation bij de writer processen. Zolang er nieuwe readers bij komen en **rcount** groter dan nul blijft, kan geen enkel writer proces data aanpassen. Stel dat we voorrang wensen te geven aan de writers. We hebben nog steeds nood aan de **rcount**, daar een writer wel nog moet wachten tot de readers, die al aan het lezen zijn, klaar zijn. Daarnaast kunnen we best ook een teller **wcount** bijhouden die aangeeft hoeveel writers er actief zijn en een semafoor **y** om de toegang tot deze teller te regelen. De teller **wcount** gebruiken we om na te gaan wanneer we de readers wederom kunnen toelaten. Hij wordt aangepast zoals de **rcount** teller. Een extra semafoor **rsem** geeft nu aan wanneer de readers weer mogen starten met lezen. In tegenstelling tot de **wsem**, zal de **rsem** semafoor ervoor zorgen dat er geen readers kunnen starten. Hij mag natuurlijk niet voorkomen dat lezers gelijktijdig kunnen lezen.

```
procedure reader:
  wait(rsem);
  wait(x);
  rcount := rcount+1;
  if rcount = 1 then wait(wsem);
  signal(x);
  signal(rsem);
  READ
  wait(x);
  rcount := rcount-1;
  if rcount = 0 then signal(wsem);
  signal(x);
```

```
procedure writer:
  wait(y);
  wcount := wcount+1;
  if wcount = 1 then wait(rsem);
  signal(y);
  wait(wsem);
  WRITE;
  signal(wsem);
  wait(y);
  wcount := wcount-1;
  if wcount = 0 then signal(rsem);
  signal(y);
```

Deze code heeft nog een kleine tekortkoming: stel dat er tijdens het schrijven vele readers zich aandienen. Dan zullen deze allen in de queue van de semafoor *rsem* terechtkomen. Wanneer de schrijfactie volledig afgelopen is, dan kunnen de readers starten met lezen. Indien er zich vlak daarna een writer aandient, dan moet deze de hele queue in *rsem* laten voorgaan. Kortom we hebben een prioriteitsqueue waarbij hoge prioriteitsklanten die de queue leeg aantreffen, alle reeds aanwezige lage prioriteitsklanten moet laten voorgaan, wat niet wenselijk is. We kunnen dit vermijden door nog een semafoor *z* in te voeren en de operaties *wait(z)* en *signal(z)* rond de *wait(rsem)* en *signal(rsem)* instructies te plaatsen in de reader procedure.

FILE SYSTEMEN

Eén van de belangrijkste doelstellingen van de meeste computersystemen bestaat erin om informatie of data te verwerken en het resultaat van deze verwerking aan de gebruiker kenbaar of beschikbaar te maken. Dit impliceert dat de gebruiker data moet kunnen ingeven, alsook het resultaat moet kunnen opslaan of opvragen. Dit zal gebeuren door middel van files. Een file is voor de meerderheid van de modale computer gebruikers dan ook het meest zichtbare concept van het operating systeem. In dit hoofdstuk bespreken we enerzijds enkele aspecten van files waarmee de meeste gebruikers (en programmeurs) veelvuldig in contact komen en anderzijds nemen we ook een kijkje naar de onderliggende technieken die gebruikt worden om een file systeem te implementeren zodat het een goede performantie realiseert. Aangezien files doorgaans op disks worden opgeslagen, is het nuttig om de werking van de eerder besproken disk scheduling algoritmen niet uit het oog te verliezen.

1 File attributen, operaties en types

Informatie op een computersysteem zal steeds georganiseerd worden in files, die doorgaans worden opgeslagen op een disk (of flash geheugen). Alle user data, hoe klein ook, zal bestaan uit één of meerdere files. De inhoud van een file is typisch een collectie van bits, bytes, lijnen of records. Afhankelijk van de inhoud van een file spreekt men onder andere van *text*, *source*, *object* en *executable* files.

Attributen

Hoewel de inhoud van een file erg kan verschillen, hebben alle files een aantal gelijkaardige attributen:

- **File name:** Voor het gemak van de gebruiker zal elke file een naam dragen, zodat ze eenvoudig herkenbaar en leesbaar zijn. In oudere systemen moest de naam van een file vaak aan vrij sterke eisen voldoen.

Vandaag de dag is er nog wel een maximale file name lengte. Deze is echter zo groot dat de lengte van de file name voor praktische doeleinden onbeperkt lijkt.

- **Type:** File types die door het operating systeem worden onderscheiden zijn ongerelateerd aan de gekende *file extensies* (bv. .doc, .txt). File systemen zoals Unix en Linux maken gebruik van een heel erg beperkt aantal file types (zie verder).
- **Location:** Deze informatie identificeert het I/O device waarop de file zich bevindt en geeft verder aan wat de locatie is van de file op het device.
- **Size:** De omvang van de file wordt eveneens bijgehouden (in bytes of blokken). Voor de meeste systemen is er ook een maximale file size, die beduidend kleiner kan zijn dan de disk capaciteit.
- **Protectie:** Aangezien de data van meerdere gebruikers vaak een enkele disk moet delen, is er ook nood aan protectie. Deze info geeft aan wie de file mag lezen, schrijven, uitvoeren, enz.
- **Time, date en user id:** Meestal wordt ook informatie bijgehouden over de eigenaar van de file, de tijd waarop de file is aangemaakt en waarop deze laatst werd aangepast, enz.

Al deze informatie voor elk van de files zal worden opgeslagen in de *directory structuur* van het file systeem, die eveneens op de disk wordt opgeslagen. We komen op elk van deze attributen, alsook op de mogelijke organisaties van de directory structuur, terug bij de bespreking van de verschillende onderdelen van een file systeem.

Operaties

We vermelden eerste even kort wat de verschillende operaties zijn die men op een file kan uitvoeren. Hoewel iedereen ongetwijfeld vertrouwd is met deze materie, halen we ze hier even aan om te wijzen op de onderliggende file systeem acties die het operating systeem hiervoor moet ondersteunen.

Ten eerste moet men een file kunnen **aanmaken** (of creëren). Hiervoor moet er vrije ruimte op de disk worden gevonden. Vandaar dat er een noodzaak is aan *free-space* management, dat de beschikbare vrije ruimte zal

inventariseren en toelaat snel vrije diskruimte te lokaliseren. We komen hier later op terug.

Verder moeten we informatie kunnen **lezen** of **schrijven** uit een file. Om dit te realiseren, moet de file eerst gelokaliseerd worden. Het lokaliseren gebeurt via het doorzoeken van de directory structuur. Verder zal vaak ook een lees of schrijf pointer bijgehouden worden, die aangeeft op welke positie in de file de data gelezen of geschreven moet worden. Deze positie is in dit geval dus *geen* input parameter van de lees- of schrijfoperatie. Na elke operatie wordt deze pointer aangepast. Aangezien een proces een file meestal niet gelijktijdig leest en schrijft, zal de lees en schrijf pointer meestal gecombineerd worden in de *current-file-position* pointer.

Een andere nuttige operatie is de **file seek**, die toelaat de waarde van de current-file-position te wijzigen. Ook bij deze operatie moet de file natuurlijk eerst gelokaliseerd worden. Files kan men ook **verwijderen**. Na het lokaliseren van de file wordt de nodige informatie uit de file directory verwijderd en moet de ruimte die de file nog op de disk innam, terug worden opgenomen door het free-space management mechanisme. Andere operaties zoals het toevoegen van data aan het einde van een file of een file hernoemen, maken soms ook deel uit van de basisoperaties van een file systeem. De overige operaties, zoals copy, worden meestal als combinatie van de voorgaande aangeboden.

Open-file table(s)

Opvallend aan de voorgaande bespreking is dat de meeste, zo niet alle, operaties vereisen dat we de directory structuur doorzoeken. Om te vermijden dat we dit moeten doen tijdens elke operatie wordt er gewerkt met een *open-file table*. Voor we een operatie op een bestaande file uitvoeren, zal deze file eerst geopend worden (dit kan impliciet of expliciet gebeuren). De open-file table zal informatie bijhouden van alle open files.

Bij het openen van de file zal de directory entry van de file eerst gelokaliseerd worden. Deze wordt vervolgens (indien de protectie van de file dit niet verhindert) naar de open-file table gekopieerd. De index van de open-file table die aangeeft waar deze entry geplaatst is, zal als return argument aan de open operatie worden teruggegeven (in het geval het openen van een file een expliciete operatie is). Alle I/O operaties zullen vervolgens gebruik maken van deze index en dus geen gebruik meer maken van de file naam. Hierdoor wordt het herhaaldelijk doorzoeken van de directory structuur vermeden.

Veelal is er ook een close operatie, hoewel de file meestal ook automatisch uit de open-file table verwijderd wordt indien het proces dat deze file opende eindigt.

De open en close operaties vereisen iets meer werk indien we werken in een multiuser omgeving, omdat meerdere processen soms eenzelfde file kunnen openen, lezen, enz. Vandaar dat er gewerkt wordt met twee niveaus van open-file tables. Op het hoogste niveau zijn er de *per-proces* open-file tables. Voor elk proces wordt er zulk een table bijgehouden. Deze duidt aan welke files het proces momenteel in gebruik heeft. Om het gelijktijdig lezen van een file door verschillende processen te ondersteunen, wordt er een current-file-position pointer bijgehouden per proces. De waarde hiervan wordt dan ook in de per-proces table opgeslagen.

Er wordt ook een pointer naar een entry in de *system-wide* open-file table voorzien. Alle informatie over de open file die identiek is voor de processen die de file momenteel gebruiken, vinden we hierin terug (zoals de file locatie, size, datums, protectie informatie, etc.). Daarnaast wordt er ook een *open-file counter* bijgehouden. Deze geeft aan hoeveel processen momenteel gebruik maken van de file. Bij elke open (close) operatie wordt deze counter met één verhoogd (verlaagd). Indien de counter de waarde van nul bereikt, dan wordt de entry uit de system-wide open-file table verwijderd.

I/O memory mapping

De informatie betreffende de locatie van de geopende files wordt doorgaans in het geheugen opgeslagen, zodat snelle access ondersteund kan worden. Verder worden files eveneens gemapt in het virtueel geheugen van een proces (zie hoofdstuk Virtueel Geheugen). Men spreekt dan van *memory mapped files*. Op deze manier kan er zeer snel informatie van en naar de file vloeien zonder dat er een disk access nodig is (indien de overeenkomstige pages in het geheugen aanwezig zijn). Lees- en schrijfoperaties naar de file, stemmen op deze manier overeen met het lezen en schrijven van een woord in het virtueel geheugen. Pas wanneer de file gesloten wordt, zal de data naar de disk worden geschreven en wordt het gealloceerde virtuele geheugen weer vrijgegeven.

Indien het operating systeem page tables hanteert om het virtuele geheugen te ondersteunen, dan kan men op deze wijze ook heel eenvoudig een file delen. Namelijk, de virtuele adressen die overeenstemmen met de file, zullen via de page table vertaald worden naar dezelfde page frame, die (een

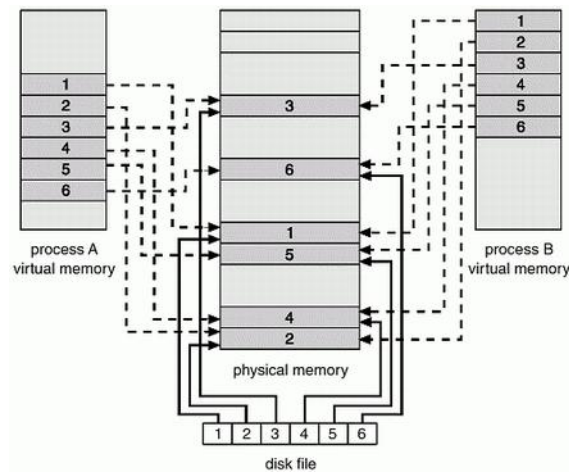


Fig. 54: Memory mapped files [Silberschatz 94]

deel van) de file bevat (zie Figuur 54). Dit is belangrijk daar verschillende processen erg vaak gebruik zullen maken van dezelfde dynamic libraries (dat is, .dll of .so files).

File types

Wanneer we spreken over het type van een file, dan denken we meteen aan de file extensie (bv. exe, .com, .bat, .obj, .o, .c, .f77, .txt, .doc, .tex, .ps, .dvi, .gif, .zip, .tar, enz.). Deze extensies zijn er enkel voor het gemak van de gebruiker en de applicaties die hiermee geassocieerd zijn en staan los van de file types erkent door het file systeem. Wanneer een file systeem gebruik maakt van meerdere file types, zal ook de interne structuur van elk file type verschillen.

In Unix of Linux zijn alle *regular* files eenvoudigweg een collectie van 8-bit bytes, zonder enige bijhorende interpretatie. Kortom, er wordt gewerkt met een enkel file type. Verder zijn er wel nog enkele extra file types voor het ondersteunen van speciale files, zoals directory files, device files, symbolische links, pipes, sockets, etc. Device files worden gebruikt om data van of naar een I/O device te schrijven en zorgen ervoor dat we op dezelfde manier data kunnen schrijven naar een file of een I/O device. Pipes en socket files laten toe om data uit te wisselen tussen processen (al dan niet op dezelfde machine). Symbolische links zullen we later bespreken. Het type van de file wordt bepaald door het eerste karakter in de output van het `ls -l` commando.

2 De directory structuur

Gezien de grote capaciteit van hedendaagse disks, zal men vaak de capaciteit van een disk opdelen in een beperkt aantal *partities* of *volumes*. Elk volume beschikt dan over zijn eigen directory structuur en kan daarom beschouwd worden als een virtuele disk. Zoals reeds eerder aangegeven bevat de directory structuur voor elke file een entry die de waardes van alle file attributen (onrechtstreeks) bijhoudt. De directory structuur moet dan ook toelaten om snel de entry horende bij een file name te lokaliseren (we keren verder op dit onderwerp terug).

2.1 Boomgestructureerde directories

Om de files van verschillende gebruikers en applicaties te organiseren wordt er veelal met een *boomstructuur* gewerkt (zie Figuur 55). Men implementeert zulk een structuur door met elk knooppunt in de boom een directory file te associëren. Een eerste file, die met de root van de boom overeenstemt, bevat onder andere een stel pointers die elk verwijzen naar één van de directory files die met de level 1 knopen van de boom overeenkomen. Deze level 1 knopen zijn de subdirectories van de root node. Analooch kan elke subdirectory op zijn beurt een eigen lijst van subdirectories bevatten. Verder bevat een directory file natuurlijk ook alle informatie over de eigenlijke data files die zich in deze directory bevinden.

Maakt een gebruiker een nieuwe file aan, dan moet de naam enkel uniek zijn binnen de directory waarin deze wordt aangemaakt. Bijgevolg, wordt een file op een unieke manier geïdentificeerd door het *path* op te geven dat we binnen de boom moeten doorlopen om de file te lokaliseren. Bijvoorbeeld, de file *exp* op level 2 van de boom heeft als pad */spell/mail/exp*. Indien er informatie over deze file nodig is, dan wordt eerst de directory file */* doorlopen om de entry die met *spell* overeenstemt te lokaliseren. Deze bevat een pointer naar de directory file */spell*, waarin we de entry *mail* opzoeken. In de directory file waar de entry *mail* naartoe wijst, vinden we tenslotte de entry *exp* die de eigenlijke file info bevat. Wanneer er vanuit de *shell* naar een bepaalde file verwezen wordt, dan zal enkel de directory file van de huidige directory doorzocht worden. Dit geeft aanleiding tot een veel sneller zoekresultaat, maar heeft ook enkele beperkingen.

Stel dat meerdere gebruikers eenzelfde applicatie (b.v. een file editor) wensen te gebruiken. We wensen in dit geval te vermijden dat de nodige

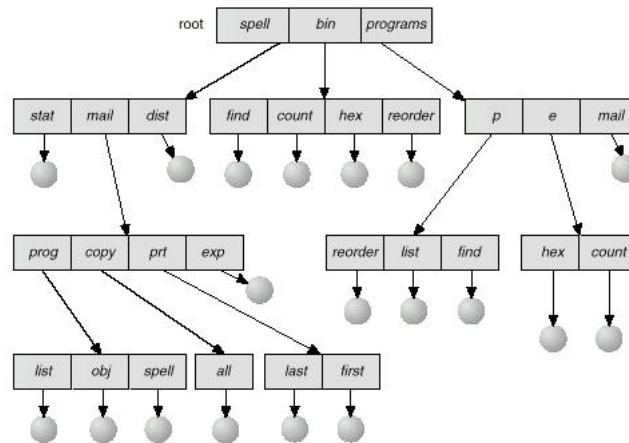


Fig. 55: Boomgestructureerde directory [Silberschatz 94]

file(s) meermaals op het systeem aanwezig moeten zijn. Anderzijds willen we ook niet dat de gebruikers steeds de volledige path name moeten opgeven. Om dit op te lossen wordt er binnen de shell gewerkt met een *search path*. In het search path staan een aantal directory namen vermeld. Wanneer een gebruiker vanuit de shell een commando oproept zal zowel de huidige directory als het search path doorzocht worden. Op deze manier kunnen alle gebruikers gewoon de korte file name hanteren, door de directory die de applicatie bevat in het search path te plaatsen. Vermits het search path er voor zorgt dat de gebruikers geen lange path namen hoeven te onthouden, laat dit toe dat we deze files ook eenvoudig kunnen herlokalisieren. Naast het opgeven van het volledige path, kan er ook met relatieve paden gewerkt worden (ten opzichte van de huidige directory).

2.2 Graafgestructureerde directories

Een boomstructuur is eenvoudig, maar is niet altijd toereikend. Stel dat alle (of enkele) gebruikers ook data in een directory van een andere gebruiker, genaamd gebruiker A, mogen lezen en dit veelvuldig doen. Dan zou het handig zijn indien zij snel access hebben tot deze directory (of files) zonder dat ze met path namen of search paths hoeven te werken. Een oplossing zou zijn om de directory van gebruiker A ook als een subdirectory bij alle overige gebruikers aan te bieden.

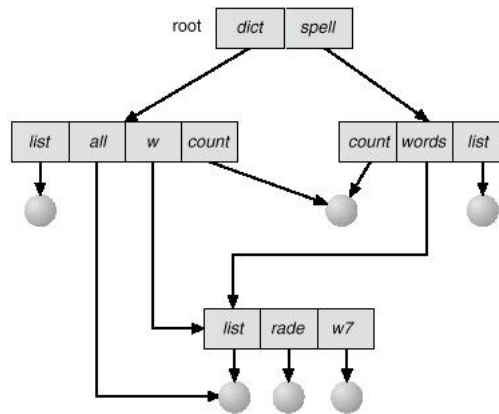


Fig. 56: Graafgestructureerde directory [Silberschatz 94]

Dit soort subdirectories kan men ondersteunen via *symbolische soft links*. Een entry in een directory file die overeenstemt met een symbolische link ziet er gelijkaardig uit als een entry voor gelijk welke andere directory, behalve dat hij ook een path naam bevat die aangeeft naar welke directory de link precies verwijst (hetzelfde gaat op voor symbolische links naar files). In UNIX kan men symbolische links aanmaken met het commando **ln -s**.

Eén van de gevolgen van het aanbieden van symbolische links, is dat een file niet langer over een unieke path naam beschikt. Dit geeft ook aan dat de directory structuur geen boom meer vormt, maar een gerichte graaf (zie Figuur 56). Deze graaf kan in principe gesloten paden bevatten, tenzij het file systeem voorkomt dat er een gesloten pad gecreëerd wordt. Dit is belangrijk daar men vaak (bijvoorbeeld voor backup doeleinden) een deel van de directory structuur wenst af te lopen.

Indien er gesloten paden kunnen voorkomen, zal de procedure die de structuur afloopt, hiermee moeten kunnen omgaan (b.v. in UNIX zal het commando **du** de structuur van de directory weergeven, waarvan de current directory de root is, voor symbolische links is de **-L** optie van belang). In veel gevallen zal er gewoon een maximum gesteld worden op het aantal (symbolische) links dat men mag nemen in een enkel path. Is er een gesloten pad aanwezig, dan wordt dit maximum overschreden en verschijnt er een warning. Procedures die directories aflopen zullen als default veelal de symbolische links niet volgen, zodat er geen gesloten paden kunnen ontstaan.

Het gebruik van symbolische links werpt ook een aantal vragen op. In-

dien we een symbolische link volgen en we gaan vervolgens naar de parent directory, waar komen we dan terecht (dit blijkt shell afhankelijk te zijn)? Ook het deleten van files werpt enkele problemen op. Kunnen we een file (of directory) zomaar fysiek deleten wanneer er nog referenties naar bestaan? Wanneer we zeker willen zijn dat er geen referenties meer naar de file bestaan, dan moeten we in elk geval een teller bijhouden die aangeeft hoeveel links er momenteel naar de file aanwezig zijn. Bij het creëren van de file wordt deze teller op één geïnitieerd. Wordt de teller door een delete verlaagd naar nul, dan kunnen we de file ook fysiek deleten (en de disk space weer vrijgeven). In UNIX zullen *hard* links deze laatste benadering hanteren, terwijl *soft* links het file systeem niet zal weerhouden om een file of directory te deleten. In geval van een hard link zal de file entry uit de directory structuur worden overgenomen. Ten slotte kan men meestal enkel hard links leggen naar een file.

2.3 Directory structuur implementatie

De hoofdtaak van de directory structuur bestaat erin de file entry horende bij een file name te lokaliseren. Deze zoekactie kan op verschillende manieren plaatsvinden afhankelijk van hoe de entries in de directory files zijn georganiseerd.

Een eerste, eenvoudige mogelijkheid is om de directory als lineaire lijst bij te houden. Dit is eenvoudig te implementeren, maar vereist wel dat elke zoekactie de (volledige) file moet afzoeken tot de gevraagde file name gevonden is. Ook bij het aanmaken van een file moet de volledige lijst worden gecontroleerd, want de nieuw gekozen file name moet uniek zijn binnen de directory. Voor het verwijderen van een file zijn er een aantal mogelijkheden. Ten eerste kan men de entry gewoon markeren als ongebruikt. Nieuwe files kunnen deze entries dan eventueel terug innemen. Verder kan men ook de laatste entry in de directory structuur in de plaats stellen van de verwijderde entry, op deze manier zal de directory ook kleiner worden na het verwijderen van een file.

Een tweede belangrijke manier om een directory te beheren, bestaat erin om een complexere data structuur te gebruiken zodat het zoeken van entries kan gebeuren in logaritmische tijd. Vandaar dat heel wat file systemen hiervoor gebruik maken van gebalanceerde bomen.

Een derde mogelijkheid is gebruik te maken van hash tables. De grootste moeilijkheid hier is het maken van een goede keuze voor het aantal mogelijke

uitkomsten voor de hash functie (wat overeenstemt met de grootte van de hash table). Grote directories hebben nood aan een grotere tabel, terwijl kleine directories te veel plaats zouden vereisen indien we de grote van de tabel sterk zouden overschatten. Een tussenliggende mogelijkheid is dat elke uitkomst verwijst naar een lineaire lijst van files.

3 Access en allocatie methodes

Het lezen en schrijven van en naar een file kan op een aantal manieren ondersteund worden. Een eerste, zeer veel voorkomende mogelijkheid is **sequential** access. Dit geeft aan dat de inhoud van een file in volgorde wordt ingelezen van de eerste logische record tot de laatste. De current-file-position pointer geeft de huidige positie in de file weer. Voor het wegschrijven van informatie wordt er ook steeds achteraan in de file geschreven. Deze methode lijkt weinig flexibel, maar is uiterst geschikt voor applicaties zoals editors, compilers, afbeeldingen, etc. Sommige file systemen laten bij sequential access ook toe om n records verder of terug te springen.

Voor applicaties die veel data manipuleren en hier en daar sporadische lees- en schrijfoperaties uitvoeren is sequential access duidelijk erg inefficiënt (denk maar aan databanken). In dit geval zal men **direct** of **relative** access hanteren. Om de vereiste data te identificeren, geeft men het nummer van het logische record mee dat men wenst te lezen of te overschrijven (als input parameter). Direct access wordt meestal gecombineerd met een zoekmethode die toelaat ook snel te bepalen in welke logische record(s) de te lezen of te schrijven data zich bevindt. Hiervoor kan men index files hanteren in combinatie met een binair zoekalgoritme of een hashfunctie. Wanneer de index file voldoende klein is, kan deze steeds in het geheugen aanwezig blijven.

3.1 Allocatie methodes

Stel dat een file in totaal n blokken (of fysieke records) groot is. Dan zal de allocatie methode aangeven hoe deze n blokken op de disk geplaatst worden. Er zijn drie belangrijke allocatie methodes die we zullen bespreken: *continuous*, *linked* en *indexed* allocatie. Later in dit hoofdstuk bekijken we ook de allocatie techniek die UNIX systemen hanteren en die eigenlijk een variant is op indexed allocatie.

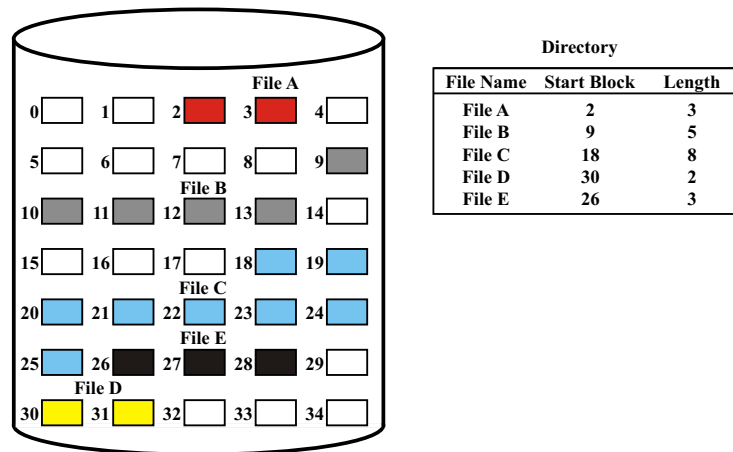


Fig. 57: Continuous file allocation [Stallings 98]

Continuous allocatie

In geval van continuous allocatie zullen de n blokken van een file allemaal achter elkaar gelokaliseerd zijn op de disk. Als we de disk blokken nummeren startende vanaf nul, dan zal de file dus blokken $b, b+1, \dots, b+n-1$ innemen voor een bepaalde b , die het start adres van de file op de disk weergeeft. De directory entry van de file bevat in dit geval het adres b , alsook het aantal blokken dat de file vereist (zie Figuur 57).

Het mag duidelijk zijn dat deze allocatie methode een zeer goede ondersteuning geeft voor zowel sequential als direct access. Aangezien opeenvolgende blokken naast elkaar op de disk gelegen zijn, er is haast geen seek time vereist van zodra het eerste blok is gevonden. Voor direct access kan het blok nummer snel gevonden worden door het vereiste relatieve bloknummer bij b , het start adres, op te tellen.

Continuous file allocation confronteert ons met gelijkaardige problemen als het dynamisch partitioneren van het geheugen. Hoe bepalen we waar we de file plaatsen? Hierbij hebben we opnieuw een aantal mogelijkheden zoals Best-fit, First-fit, Next-fit of zelfs Worst-fit voor het selecteren van de leegte.

Verder stelt zich ook het probleem dat we de grootte van de file op voorhand moeten kennen. Dit is vaak erg problematisch omdat een file over een lange periode in de tijd nog kan aangroeien of omdat het een output file is waarvan we niet onmiddellijk weten hoeveel output er zal gegenereerd worden. Zonder echter de grootte te kennen is het ook moeilijk om een keuze te

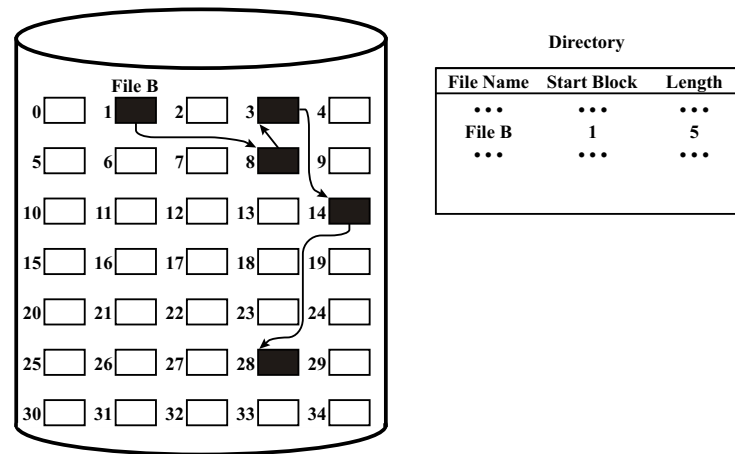


Fig. 58: Linked (of chained) file allocation [Stallings 98]

maken bij het kiezen van een leegte. Tenzij we het veelvuldig verplaatsen van de file wensen te ondersteunen, moet de gebruiker al bij het aanmaken van de file een size opgeven. Dit zal doorgaans resulteren in een overschatting, waardoor er ook sprake is van interne fragmentatie.

Soms zal het file systeem ook een routine aanbieden die toelaat om de leegtes van de disk te verwijderen (dit was bijvoorbeeld vaak het geval bij oude floppy disks die continuous allocatie hanteerden). Het veelvuldig uitvoeren van deze routine is natuurlijk wel erg belastend voor het systeem.

Tenslotte zullen sommige systemen ook toelaten om één of meer *extents* aan een file toe te voegen. In dit geval zal er een tweede, continue reeks disk blokken worden gealloceerd, indien blijkt dat de oorspronkelijke set niet meer volstaat. Het nadeel hierbij is wel dat meestal de gebruiker de omvang van de extent moet bepalen.

Linked (of chained) allocatie

Met linked allocatie kunnen we alle problemen van continuous allocatie uit de weg ruimen. Het idee is om de n blokken die samen de file vormen, als een gelinkte lijst op te slaan. De n blokken kunnen op deze manier over de hele disk verspreid liggen. De directory entry bevat een pointer naar het eerste blok, dit blok bevat een pointer naar blok nummer 2, enz (zie Figuur 58).

Bij het aanmaken van een nieuwe file, wordt er eerst een entry in de

directory aangemaakt, vervolgens wordt voor elk blok het *free-space* algoritme aangeroepen om een nieuw vrij blok te genereren. Elk blok bevat hierbij een pointer naar het volgende blok. Er is dus helemaal geen noodzaak meer om de grootte van de file op voorhand te kennen. Bijgevolg is de externe fragmentatie typerend voor een continuous allocatie niet meer van toepassing.

Het grote nadeel van deze methode is dat zowel sequentiële als directe access aanleiding geven tot betrekkelijk grote seek times, voornamelijk wanneer de verschillende blokken, geproduceerd door het free-space algoritme, ver uit elkaar gelegen zijn. Verder moet er in elk blok ook een beperkt aantal bits opgeofferd worden om de pointer naar het volgende blok te ondersteunen (b.v. 4 bytes per 512 bytes).

Men kan de nadelen van een linked allocatie schema deels vermijden, door te werken met *clusters*. Een enkele cluster bestaat in dit geval uit een vast aantal blokken (bijvoorbeeld een viertal). Hierdoor is er minder capaciteitsverlies omwille van de pointer en worden de seek times beduidend kleiner bij sequentiële en direct reads. Clusters worden in de meeste operating systemen ondersteund.

File allocation table

De file allocation table (FAT) oplossing is eigenlijk een variant van de linked allocation. Het verschil is dat er nu een aparte tabel wordt bijgehouden die per disk blok een veld bijhoudt. Elk veld bevat het nummer van het volgende blok dat deel uitmaakt van de file waartoe dit blok behoort. Het nummer van het start blok wordt als index tot deze tabel gebruikt. Indien het blok niet gebruikt wordt, dan kan men dit aanduiden met een 0-waarde. Indien we een nieuw blok moeten alloceren, dan kunnen we eenvoudigweg het eerste veld met daarin een 0-waarde lokaliseren. Een beter alternatief bestaat erin om de vrij blokken met elkaar te linken, zoals ook de blokken van de files gelinkt zijn en een pointer bij te houden naar de start van de lijst van vrije blokken.

Het voordeel van de FAT benadering is dat deze tabel zich op het initiële deel van de disk bevindt waardoor er bij direct access betrekkelijk weinig seek time wordt veroorzaakt. Dit komt omdat alle te volgen pointers in de FAT lijst vervat zijn en de mogelijkheid dus niet langer bestaat dat deze over de hele disk verspreid liggen. Verder zal vaak ook een deel van de FAT worden opgeslagen in een cache om de access tijden verder te reduceren. Voor sequentiële reads is dit voornamelijk van belang omdat de disk head anders

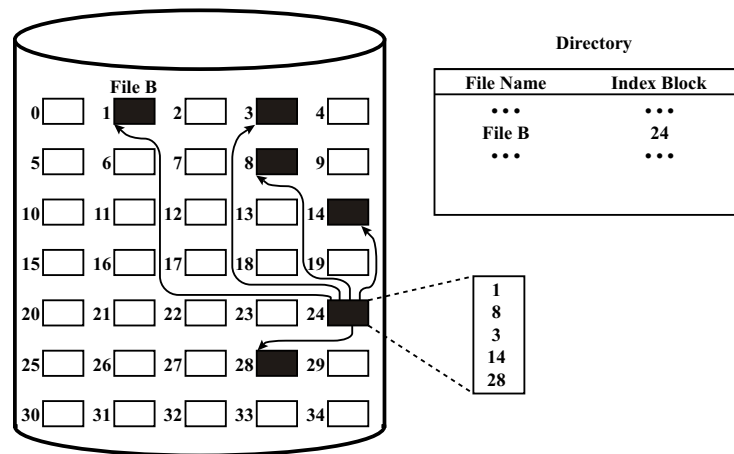


Fig. 59: Indexed file allocation [Stallings 98]

vaak heen en weer moet bewegen tussen de FAT en de eigenlijke disk blokken die de data bevatten. MS-DOS en OS/2 werkten beide met file allocation tables.

Indexed allocation

Linked allocatie lost de meeste problemen van continuous allocatie op. Echter, zoals aangegeven is direct access in het bijzonder erg traag, omdat de seek times sterk kunnen oplopen. De FAT structuur kan de ernst van dit probleem al in sterke mate doen afnemen. Een alternatieve oplossing bestaat erin een index blok te voorzien voor elke file. Dit blok bestaat dan uit een lijst van blok nummers, waarbij de i -de entry aangeeft waar op de disk blok i zich bevindt. De directory bevat in dit geval de locatie van het index blok (zie Figuur 59).

Indexed allocation laat duidelijk ook erg snelle direct access toe, want we moeten nog slechts twee pointers volgen om het vereiste disk blok te lokaliseren. De sequentiële read operaties zijn wel nog trager dan bij een continuous allocatie oplossing. Dit kan wederom deels verholpen worden door gebruik te maken van clustering. Merk ook op dat indexed allocatie in grote mate overeenstemt met het paging mechanisme dat meestal gebruikt wordt voor het beheren van het geheugen.

Het grootste nadeel van indexed allocatie is het feit dat elke file een index

blok nodig heeft, terwijl een file mogelijk maar uit één of twee disk blokken bestaat. De index tabel bevat in dit geval slechts één of twee entries die verschillen van nul. Hierdoor wordt er duidelijk meer capaciteit gebruikt door kleine files dan bij een linked allocatie.

Deze kwestie geeft ook aanleiding tot de vraag hoeveel entries een index tabel eigenlijk moet bevatten. Met andere woorden, moeten we meer dan één blok voorzien om grote files te ondersteunen? Er zijn een aantal mogelijke oplossingen denkbaar. Ten eerste zouden we, indien een enkel blok niet volstaat, een gelinkte lijst van index blokken kunnen voorzien. Het nadeel hiervan is dat het aantal te volgen pointers in geval van een referentie naar een blok achteraan in een erg grote file snel kan oplopen.

Het is dan ook verkiesbaar om (voor grote files) met een multilevel index te werken, zoals dat ook het geval was bij de multilevel page table. Op deze manier kunnen we zeer grote files ondersteunen, terwijl het aantal te volgen pointers tot twee of drie beperkt blijft. Bijvoorbeeld, wanneer een pointer 4 bytes beslaat en elk blok 4 sectoren van elk 512 bytes omvat, dan kan een 2 level index alleen al in totaal $512^2 = 262144$ blokken aanspreken, wat overeenstemt met ongeveer een halve gigabyte. Verder kan men ook een gecombineerd schema hanteren zoals we zo meteen zullen zien bij de UNIX inode.

3.2 UNIX inodes

Een interessant voorbeeld van een graafgestructureerde directory die een indexed allocatie methode hanteert, vinden we terug in UNIX systemen. Hier zal er met elke file een *inode* worden geassocieerd (waarvan we de structuur terugvinden in Figuur 60). Het woord i-node is een afkorting voor index node, waarbij het koppelteken met de tijd is verloren gegaan.

Een inode bevat alle file attributen in de eerste velden. Zo geeft de *mode* informatie weer die betrekking heeft op de protectie van de file, de *owners* geeft de ID van de eigenaar weer en de groep waartoe hij of zij behoort, de *count* telt het aantal hard links naar de file, etc. De directory file is hierdoor herleid tot een zuivere vertaler van file names naar inode nummers. Het voordeel van deze aanpak is dat het aanmaken van hard links erg eenvoudig is. Men maakt gewoon een entry aan in de directory file die eveneens wijst naar de betreffende inode en verhoogt de *count* value van deze node met één. Een directory file is eveneens een inode, waarbij de inhoud gelijk is aan een lijst van inode nummers met hun overeenkomstige file names.

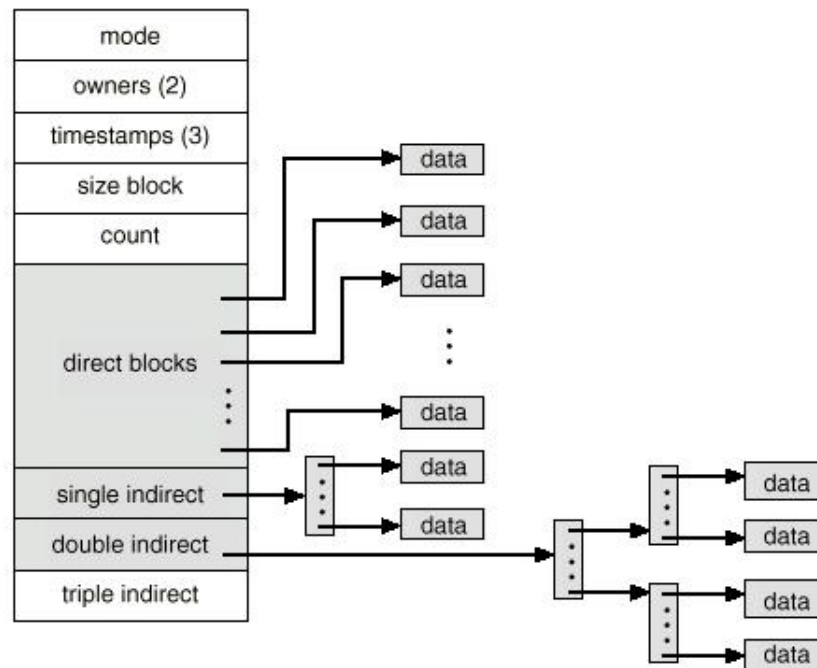


Fig. 60: De UNIX inode structuur [Silberschatz 94]

Wat de allocatie methode betreft, zien we dat de inode indexed allocatie gebruikt. Echter, enkel voor de eerste 12 (of 10) blokken is er een directe pointer van het index nummer naar het geassocieerde disk blok met de file data. Het voordeel hiervan is dat de data blokken van kleine files onmiddellijk beschikbaar zijn. Echter, direct pointers voorzien voor alle blokken zou resulteren in een erg grote inode (waarbij het onduidelijk is hoeveel velden we best moeten ondersteunen).

Daarom wordt er ook met indirecte pointers gewerkt. Na de 12 (of 10) directe pointers is er één indirecte pointer voorzien, die we de *single indirect* pointer noemen en die wijst naar een blok dat enkel bestaat uit pointers naar disk blokken. De volgende pointer in de inode, de *double indirect* pointer, verwijst dan weer naar een index blok waarvan de entries op hun beurt naar andere index blokken verwijzen. Tenslotte is er ook nog een *triple indirect* pointer, waarbij er in totaal vier pointers gevolgd moeten worden voor we over de eigenlijke data van de file kunnen beschikken.

Op deze manier is de hoeveelheid disk space die een file behoeft aan

indexering op een goede manier gerelateerd aan de file size. Bijvoorbeeld, indien er 10 directe pointers zijn, elke pointer 8 bytes groot is en een blok 4Kbyte, dan kunnen we 40Kbyte direct adresseren, 2Mbyte via de single indirect pointer, 1Gbyte via een double indirect index en tenslotte 0.5Tbyte via de triple indirect pointer.

De file systemen *ext2* en *ext3* (populair in Linux) maken gebruik van deze allocatie methode. Ext4 werkt daarentegen met *extents*. Een extent is een clusters van opeenvolgende blokken op disk en elke inode bevat een directe verwijzing naar 4 extents. Wanneer er meer dan 4 extents nodig zijn wordt de lokatie van de overige blokken bijgehouden op disk (op basis van H-trees). Werken met extents is een stuk efficiënter voor grote files die bijna niet gefragmenteerd zijn.

3.3 Free-space management

Een probleem dat we tot nu toe onbesproken hebben gelaten, is hoe er vrije ruimte op de disk wordt gelokaliseerd indien er een nieuwe file wordt aangemaakt (of indien een bestaande file groter wordt). Het beheren van deze vrije ruimte wordt aangegeven met de term *free-space management*.

Bit vector of bit map

Een eerste regelmatig gehanteerde methode bestaat erin een binaire vector bij te houden die aangeeft welke disk blokken nog vrij zijn. De lengte van de bit vector stemt bijgevolg overeen met het aantal blokken n op de disk. Is blok nummer k nog vrij, dan is de k -de bit gelijk aan één.

Deze methode wordt voornamelijk gedreven door de beschikbare bit-manipulators in de hardware van een aantal computer systemen (zoals de Intel 80386 en 80486 en een aantal PowerPC Macintosh systemen). Op sommige van deze systemen zal men de bit map woord per woord inlezen in de processor en telkens testen of het woord gelijk is aan nul. Indien dit niet het geval is, wordt de positie van de eerste 1 bit door middel van een efficiënte hardware instructie gelokaliseerd. Het vinden van het eerste vrije blok kan zo snel gebeuren en het is ook eenvoudig een sequentie van vrije blokken op de disk te lokaliseren.

Door de omvang van hedendaagse disks wordt de bit vector wel snel heel erg groot, wat nadelig is daar deze eigenlijk best steeds in het geheugen aanwezig moet zijn. Clustering kan in dit geval opnieuw een oplossing bieden,

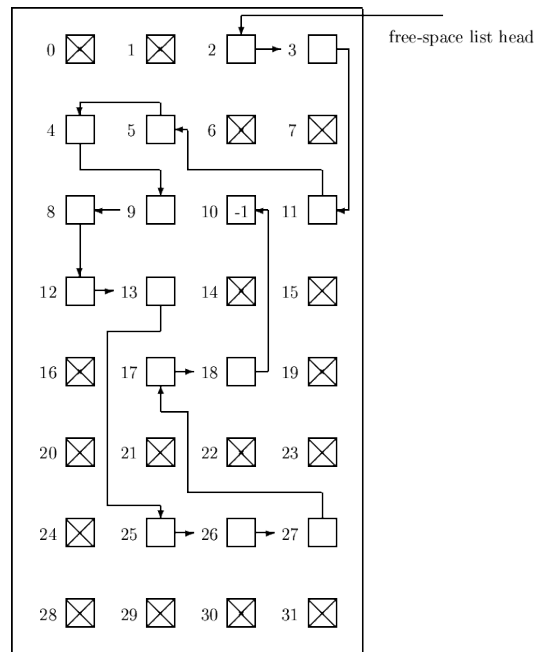


Fig. 61: Vrije ruimte management via een linked list

waarbij men doorgaans grotere clusters zal hanteren naarmate de partitie van de disk groter wordt.

Linked list

Een voor de hand liggende alternatieve aanpak voor het beheren van de vrije ruimte bestaat erin om, net zoals bij linked allocatie, een gelinkte lijst bij te houden van alle vrije data blokken (zie Figuur 61).

In dit geval wordt het blok nummer van het eerste vrije blok op een aparte plaats bijgehouden op de disk, alsook in het geheugen (voor snellere access). Indien er een enkel vrij blok nodig is, dan hoeven we enkel deze pointer te volgen. Wordt een file weggeveegd, dan moeten we de blokken die deze file innam achteraan de lijst toevoegen (wat zeker snel kan indien de allocatie methode eveneens linked was).

Met deze methode is het niet eenvoudig een reeks van opeenvolgende vrije blokken te lokaliseren, want de disk blokken hoeven niet in oplopende volgorde in de lijst aanwezig te zijn. Merk ook op dat, voor de FAT methode, deze gelinkte lijst integraal deel uitmaakt van de file table.

Indexing

Tenslotte kan er ook met indexering gewerkt worden voor het bijhouden van de vrije disk ruimte, door te stellen dat de vrije ruimte een file is zoals elke andere file. In dit geval kan het nuttig zijn om de index wat in te korten door slechts één index te voorzien voor elke reeks van opeenvolgende vrije blokken.

4 Fast file systeem voor UNIX

We eindigen dit hoofdstuk door onze aandacht te richten op enkele bepalende factoren die er voor zullen zorgen dat een file systeem performant is, wat inhoudt dat er data van en naar de disk kan geschreven worden tegen een hoge snelheid. We doen dit aan de hand van het Fast File Systeem (FFS) voor UNIX, voorgesteld in het ACM Transactions on Computer Systems tijdschrift in 1984.

4.1 Het traditionele UNIX file systeem

Zoals alle UNIX file systemen, maakt ook het traditionele UNIX file systeem ontwikkeld door Bell Laboratories, gebruik van inodes. Verder werd er een gelinkte lijst van vrije blokken bijgehouden. Alle blokken hadden een identieke omvang van 512 bytes (en iets later 1024 bytes). Dit impliceerde dat alle data uitwisselingen van en naar de disk gebeurde in blokken van 512 bytes.

Deze traditionele aanpak bleek voor heel wat file systeem intensieve toepassingen (zoals image processing) onvoldoende. Het bleek namelijk dat de gerealiseerde throughput slechts 2% van de disk bandbreedte bedroeg (de bandbreedte is gelijk aan het aantal bytes per track vermenigvuldigd met het aantal revoluties per seconde). De hoofdverklaring hiervoor bestond ten eerste in de betrekkelijk kleine blok grootte van 512 bytes, die al snel werd opgetrokken tot 1024 bytes om tot een throughput van 4% te komen. Even belangrijk was echter het gebruik van de gelinkte lijst voor free-space management. Bij het opzetten van het filesysteem zijn de opeenvolgende vrije blokken in deze free-space lijst vlak na elkaar gelegen op de hard disk. Echter na enkele weken van gemiddeld gebruik, bleek dat deze opeenvolgende blokken, door het veelvuldig verwijderen en aanmaken van files, was omgevormd

tot een lijst van blokken die bijna random over de disk verspreid lagen. Hierdoor ontstonden er veel grotere seek times, waardoor de throughput over deze periode daalde met een factor 6 (van 175 Kbytes/sec tot 30 Kbytes/sec). Merk op, door de resulterende random aard van de blokken waaruit files bestaan, werd ook de ondersteuning van de read-ahead functie nutteloos.

Een ander nadeel van het traditionele file systeem was de manier waarop de inodes en data (binnen de partitie waarop het file systeem betrekking heeft) werden georganiseerd. Een traditionele 150 Mbyte partitie bestond uit 4 Mbyte ruimte voor inodes, gevolgd door 146 Mbyte voor data. Net zoals bij de FAT geeft dit aanleiding tot grote seek times tussen het lezen van de inode informatie en de opgevraagde data zelf. Ook files die behoorden tot éénzelfde directory hadden vaak ver uit elkaar gelegen inodes, waardoor applicaties die gebruik maakten van meerdere files in dezelfde directory, extra vertragingen opliepen.

4.2 Het Fast file systeem

Wanneer we de nadelen van de traditionele aanpak in beschouwing nemen, dan lijkt de oplossing zich vanzelf aan te dienen: we moeten gebruik maken van een grotere blok grootte. Dit verhoogt niet alleen de throughput, maar laat ook toe grotere files aan te spreken zonder dat er gebruik moet gemaakt worden van de *triple indirect* pointer in de inode. Bijvoorbeeld, blokken van 4096 bytes (en pointers van 4 bytes) geven aanleiding tot $(1024)^2$ blokken die we via een dubbele indirecte pointer kunnen aanspreken, elk blok bevat 4096 bytes, wat resulteert in een ondersteuning van files tot 4 Gbyte. Het probleem bij deze benadering is het grote verlies aan opslagcapaciteit, doordat alle files, inclusief de vele kleine files, een veelvoud van 4096 bytes toegewezen krijgen. Zoals mag blijken uit Figuur 62, die is opgemaakt op basis van werkelijke metingen op een multiuser systeem, kan dit resulteren in een capaciteitsverlies van meer dan 45% van de disk. Het Fast file systeem voor UNIX zal dit probleem oplossen door elk blok eveneens op te delen in een aantal fragmenten van vaste grootte (doorgaans 2, 4 of 8, zodat de fragmenten nog minstens 512 bytes groot zijn, deze zijn elk apart adresseerbaar). Voor we hier echter op ingaan, bespreken we eerst even enkele algemene kenmerken van het nieuwe systeem.

% Verlies	Organisatie
0	Enkel Data, geen ruimte tussen files
4.2	Enkel Data, files starten op 512 byte grenzen
6.9	Data + inodes, 512 byte blok UNIX systeem
11.8	Data + inodes, 1024 byte blok UNIX systeem
22.4	Data + inodes, 2048 byte blok UNIX systeem
45.6	Data + inodes, 4096 byte blok UNIX systeem

Fig. 62: Hoeveelheid verloren ruimte als functie van de blok grootte.

Data organisatie

Het FFS werkt met blokken van minstens 4096 bytes, waarbij de blok grootte bij het opzetten van het file systeem gekozen moet worden. Verder zal het systeem de disk partitie waarop het file systeem betrekking heeft, opdelen in een aantal *cylindergroepen*. Zulk een groep bestaat uit een reeks naast elkaar gelegen cylinders van de hard disk (die mogelijk uit een aantal op elkaar gestapelde schijven bestaat, vandaar de naam cylinder). Binnen elke cylindergroep wordt één inode voorzien per 2048 bytes (wat ruim zou moeten volstaan) en wordt er een *bitmap* bijgehouden die aangeeft waar de nog vrije ruimte van deze cylindergroep zich situeert.

Deze bitmap bevat echter meerdere bits per blok, zijnde het aantal fragmenten waarin een blok is opgedeeld. Stel dat de fragmenten 1024 bytes groot zijn, dan kunnen we ons de vraag stellen hoe de werking van dit systeem verschilt van een klassieke oplossing met blokken van 1024 bytes. Om dit te begrijpen moeten we de manier waarop er met deze fragmenten wordt gewerkt van naderbij bekijken. Stel bijvoorbeeld dat er 4 fragmenten per blok zijn en dat fragment 6 tot 9 nog vrij is, alsook fragment 12 tot 15, de bitmap voor de fragmenten 0 tot 15 ziet er dan uit als

1111 1100 0011 0000.

Hoewel elke set van vier opeenvolgende fragmenten in grootte overeenstemt met een enkel blok, zullen de fragmenten 6 tot 9 niet als blok worden beschouwd, daar het nummer van het eerste fragment geen veelvoud is van 4.

Data allocatie

De allocatie gaat nu als volgt te werk. Stel dat we data wensen toe te voegen aan een file, dan kunnen volgende drie mogelijkheden zich voordoen:

1. Er is nog voldoende plaats beschikbaar voor de extra data in het laatste fragment/blok van de file. In dit geval voegen we de data eenvoudigweg toe zonder extra ruimte te alloceren.
2. Indien de ruimte toegekend aan de file enkel bestaat uit volledige blokken, dan vullen we eerst het huidige blok aan, vervolgens voegen we, indien nodig, een aantal volledige blokken toe. Tot slot zoeken we een blok waarin er nog voldoende opeenvolgende fragmenten vrij zijn om het resterende deel van de file in op te slaan (en dat mogelijk al deels door een andere file wordt gebruikt).
3. De reeds toegekende ruimte aan de file bestaat uit een aantal blokken plus minimaal 1 en maximaal 3 fragmenten. In dit geval beschouwen we deze fragmenten eveneens als nieuwe data en hanteren we de procedure uit mogelijkheid 2 om de allocatie uit te voeren. Dit betekent dat deze fragmenten gekopieerd zullen worden naar een andere locatie.

Het verschil met een klassiek UNIX systeem met blokken van 1024 bytes zit dus voornamelijk in de wijze waarop we files uitbereiden indien mogelijkheid 3 zich voordoet. Hoe frequent we fragmenten moeten verplaatsen, hangt in sterke mate af van de wijze waarop de applicaties data wegschrijven of opvragen. Standaard Input/Output library functies zullen trachten steeds met veelvouden van de gehanteerde bloksize te werken.

Free capacity reserve

Verder zal het FFS ook steeds een *reserve* aanleggen van een 5 tot 10% van de beschikbare capaciteit om te voorkomen dat de schijf volledig gevuld geraakt. Dit laatste is noodzakelijk om te voorkomen dat het opvragen van vrije ruimte te veel tijd in beslag zou nemen. Het gereserveerde percentage voor vrije ruimte kan enkel door de systeemadministrator worden aangepast. Deze reserve moeten we natuurlijk wel meetellen als verliescapaciteit van het file systeem. Uit Figuur 62 kunnen we opmaken dat een systeem met blokken van 4096 bytes en fragmenten van 512 bytes, resulteert in een verlies van ongeveer 6.9%. We kunnen dus probleemloos nog 5% reserve aanleggen

zonder meer capaciteit op te offeren dan in een traditioneel systeem met blokken van 1024 bytes (met een verlies van 11.8%).

Blok en inode selectie

Door gebruik te maken van blokken van 4096 bytes, zal de lees- en schrijfsnelheid al danig toenemen. Echter dit alleen volstaat niet om een zeer goede performantie te realiseren. Er moet ook voor gezorgd worden dat de seek times tussen het lezen/schrijven van opeenvolgende blokken op de disk beperkt blijven. Zonder stil te staan bij de details, zal het FFS trachten de eerste 48 Kbyte (dat is, de data vervat in de 12 directe blokken van de inode) in dezelfde cylindergroep te plaatsen. Daarna wordt er voor elke extra Mbyte overgegaan naar een nieuwe cylindergroep. Op deze manier wordt er vermeden dat een enkele file een al te groot deel van een cylindergroep zou opeisen, waardoor de overige files in deze groep niet meer kunnen aangroeien. Er wordt ook steeds getracht verder te gaan met een cylindergroep die een lagere bezettingsgraad heeft dan gemiddeld. Kortom, we moeten trachten te vermijden dat cylindergroepen volledig gevuld geraken, want dit geeft aanleiding tot grotere seek times wanneer we een file in de groep wensen te vergroten.

Wat betreft het toekennen van de inodes aan nieuw aangemaakte files, zal het FFS ernaar streven de inodes van files die tot éénzelfde directory behoren binnen een enkele cylindergroep te plaatsen. Inodes voor directory files worden daarentegen net gezocht in cylindergroepen waarvoor het aantal vrije inodes nog groter is dan gemiddeld.

Performantie

Om de effectiviteit van het FFS te evalueren gaan we de performantie van het FFS vergelijken met deze van het traditionele UNIX systeem. We merken hierbij op dat de resultaten betrekking hebben op een systeem uit 1983 dat reeds meer dan een maand in gebruik was voor de metingen werden uitgevoerd. De resultaten voor de lees- en schrijfsnelheid worden weergegeven in Figuur 63. Het enige verschil tussen de UNIBUS en MASSBUS testen, is de gehanteerde hard disk controller. Vroeger maakte de disk controller geen integraal deel uit van de hard disk, maar bevond deze zich vaak op een aparte controller card. Hierdoor kon men een enkele disk combineren met verschillende controllers.

% File systeem	Bus	Leessnelheid	Leesbandbreedte	% CPU
old 1024	UNIBUS	19 Kb/sec	29/983 (3%)	11%
FFS 4096/1024	UNIBUS	221 Kb/sec	221/983 (22%)	43%
FFS 8192/1024	UNIBUS	233 Kb/sec	233/983 (24%)	29%
FFS 4096/1024	MASSBUS	466 Kb/sec	466/983 (47%)	73%
FFS 8192/1024	MASSBUS	466 Kb/sec	466/983 (47%)	54%
% File systeem	Bus	Schrijfsnelheid	Schrijfbandbreedte	% CPU
old 1024	UNIBUS	48 Kb/sec	48/983 (5%)	29%
FFS 4096/1024	UNIBUS	142 Kb/sec	142/983 (14%)	43%
FFS 8192/1024	UNIBUS	215 Kb/sec	215/983 (22%)	46%
FFS 4096/1024	MASSBUS	323 Kb/sec	323/983 (33%)	94%
FFS 8192/1024	MASSBUS	466 Kb/sec	466/983 (47%)	95%

Fig. 63: Lees- en schrijfsnelheid, gerealiseerde bandbreedte en CPU belasting om UNIX systemen (VAX 11/750)

We zien dat het FFS in staat is om veel hogere lees- en schrijfsnelheden te realiseren, met throughputs tot 47%. In tegenstelling tot het oude systeem blijkt zelfs dat de throughput niet gedaald is ten opzichte van kort na het opzetten van het file systeem. Wanneer we de resultaten uit Figuur 63 eens van naderbij bekijken, dan zien we een aantal opmerkelijke zaken.

De leessnelheid in het traditionele systeem ligt lager dan de schrijfsnelheid, dit is opmerkelijk daar de CPU (twee maal) meer werk heeft bij het schrijven van data dan bij het lezen. Dit wordt echter verklaard doordat de leesopdrachten op dit systeem *synchroon* verlopen, wat inhoudt dat de leesopdrachten in volgorde worden afgehandeld. Het schrijven kan echter *asynchroon*, waardoor de CPU, die capaciteit over heeft, sneller schrijfopdrachten kan genereren, dan er data naar de disk kan worden geschreven. Hierdoor worden er meerdere schrijfopdrachten opgeslagen in de disk cache, wat tot gevolg heeft dat deze in een meer optimale volgorde (d.i., met lagere seek times) weggeschreven kunnen worden. In het FFS systeem liggen de zaken helemaal anders omdat daar de CPU de vertragende factor geworden is, in plaats van de disk. Verder dienen de schrijfopdrachten zich ook in een veel betere volgorde aan (dankzij de invoering van de cilindergroepen), waardoor de mogelijkheid om deze asynchroon af te handelen nog weinig extra voordeel oplevert. Bijgevolg zijn de leesopdrachten wederom sneller, zoals men zou verwachten. Merk op, het fysiek schrijven of lezen van data op de disk is

natuurlijk even snel en wordt bepaald door de draaisnelheid van de disk en de hoeveelheid data per track.

5 Journaling file systemen

Hedendaagse file systemen maken gebruik van *journaling* met als doel om het file systeem snel te kunnen herstellen na een crash of stroomonderbreking. Om de noodzaak van een journal in te zien maken we gebruik van het populaire Linux ext2 file systeem (dat is, het tweede Extended File System). Net zoals in het FFS worden de disk blokken in ext2 opgedeeld in verschillende groepen, waarbij elke groep bestaat uit een aantal inodes en data blokken. Verder bevat elke groep ook een bitmap die aangeeft welke inodes binnen de groep in gebruik zijn en een bitmap die hetzelfde aangeeft voor de data blokken.

Data inconsistentie en onzinnige data

Wanneer we nu een file aanschouwen die bestaat uit een enkel data blok (wat wil zeggen dat enkel de eerste direct pointer in de inode verwijst naar een data blok) en we wensen aan deze file een data blok toe te voegen, dan moeten er drie aanpassingen op de disk gebeuren. Ten eerste moet de data worden geschreven in een vrij data blok, daarnaast moet de tweede direct pointer in de inode van de file verwijzen naar dit data blok en tenslotte moet de bitmap die aangeeft welke data blokken er in gebruik zijn aangepast worden. Er moeten dus drie writes uitgevoerd worden en de controller van de disk zal doorgaans de volgorde van deze writes bepalen.

Stel nu dat er een crash of stroompanne optreedt voordat alle drie de writes hebben plaats gevonden, dan kunnen volgende gevallen zich voordoen:

1. Stel dat enkel de bitmap of de inode is aangepast. In het eerste geval geeft de bitmap aan dat het data blok in gebruik is, maar er is geen enkele inode die hier naar verwijst. In het tweede geval is er wel een verwijzende inode, maar geeft de bitmap aan dat het data blok nog vrij is. In beide gevallen stellen we vast dat het file systeem niet langer consistent is.
2. Wanneer enkel het data blok is geschreven, dan is het file systeem nog wel consistent, maar de geschreven data is verloren (daar er geen enkele

inode verwijst naar de data en we ook niet weten om welk data blok het gaat).

3. Indien zowel de bitmap als de inode is aangepast, zonder dat de data is geschreven, dan is het systeem opnieuw consistent. Er stelt zich echter een ander probleem: de inhoud van het data blok waar de inode naar verwijst is niet aangepast en bevat dus onzinnige informatie (garbage).
4. Indien enkel de bitmap of inode niet is aangepast, dan is er opnieuw sprake van een inconsistentie in het file systeem.

In oudere file systemen (zoals ext2) werden mogelijke inconsistenties in het file systeem vaak opgelost bij het rebooten door gebruikt te maken van specifieke tools (b.v., fsck in ext2). Wanneer bijvoorbeeld een bepaald data blok niet vrij is volgens de bitmap, maar geen enkele inode verwijst naar dit blok, dan kunnen we dit eenvoudig oplossen door de bit in de bitmap aan te passen.

Het mag echter duidelijk zijn dat het opsporen en oplossen van inconsistenties in het file systeem een trage operatie is gezien de omvang van de hedendaagse harde schijven. Vandaar dat meer recente file systemen de noodzaak van zulke tools vermijden door gebruik te maken van een *journal*. Dit is ook het voornaamste verschil tussen het Linux ext2 en ext3 file systeem.

Mogelijke oplossing

Het idee van journaling kent zijn oorsprong in het beheren van databases en bestaat erin dat we eerst op de disk aangeven welke aanpassingen we wensen door te voeren, voordat we deze ook effectief doorvoeren. Met andere woorden, voordat we wijzigingen op de disk aanbrengen, schrijven we eerst elders op de disk wat we gaan doen. De plek waar we dit aangeven noemen we de *log* of *journal*. Vandaar dat men in geval van journaling ook spreekt van *write-ahead logging*.

In ons voorbeeld zullen we de drie vereiste writes dus eerst specificeren als een enkele *transactie* in de log. Wanneer deze transactie is toegevoegd aan de log, voeren we de transactie uit en vervolgens verwijderen we de transactie van de log. Indien er een crash of stroompanne optreedt, volstaat het om de transacties in de log opnieuw uit te voeren (dat is, er gebeurt een replay van deze transacties) om het systeem consistent te maken.

Enige voorzichtigheid is wel geboden bij het schrijven van de transacties naar de log. Het schrijven van een transactie komt eveneens overeen met

meerdere writes, waarbij er een write is die een controle blok schrijft aan het begin van de transactie en een write die een blok schrijft die het einde van de transactie markeert. Het is belangrijk dat deze laatste write als laatste gebeurt, want zodra het controle blok dat het einde van de transactie aangeeft is toegevoegd aan de log, wordt de transactie als geldig beschouwd (dat is, de transactie is committed). Wanneer echter nog niet alle overige writes van de transactie zijn uitgevoerd en het systeem faalt, dan zal de replay van de log structuur mogelijk resulteren in een inconsistentie of in onzinnige data in de file. Dit betekent dat het write die het einde van de transactie aangeeft pas zal worden doorgestuurd nadat alle overige writes van de transactie zijn uitgevoerd. We merken hierbij ook op dat het schrijven van een enkel blok op een disk mag worden beschouwd als een atomaire operatie.

Hoewel het bijhouden van een journal ervoor zorgt dat we snel de data consistent kunnen maken, is er ook een prijs aan verbonden: we moeten alle writes twee maal uitvoeren, wat erg duur is. In de praktijk bestaan er verschillende modes van journaling (in ext3 zijn er 3 modes) en diegene die we tot nu toe beschreven hebben is gekend onder de naam *data journaling*. Een minder dure mode van journaling, genaamd *metadata journaling*, bestaat erin enkel de *metadata* in de log weg te schrijven en writes naar de data blokken onmiddellijk op hun definitieve plaats door te voeren. Hoewel metadata journaling vaker in onzinnige data resulteert, moet bij deze mode niet alle data twee maal geschreven worden. In ons voorbeeld lijkt het verschil klein, daar we maar één data blok hadden, maar in de praktijk zal het file systeem meerdere writes groeperen tot een enkele transactie waardoor de winst veel groter is.

Tot slot wordt er ook nog een onderscheid gemaakt tussen *ordered* en *unordered* metadata journaling. Bij *ordered* metadata journaling worden eerst alle data blokken op hun definitieve plaats geschreven. Pas daarna wordt de transactie toegevoegd aan de log. Vervolgens wordt de metadata definitief geschreven en wordt de transactie verwijderd. In de *unordered* metadata journaling mode wordt eerst de transactie in de log geschreven, vervolgens wordt de transactie uitgevoerd en worden gelijktijdig de data blokken aangepast en tenslotte wordt de transactie uit de log verwijderd. Het mag duidelijk zijn dat de *unordered* mode vaker aanleiding zal geven tot onzinnige data dan de *ordered* mode.

MULTIPROCESSOR ISSUES

In dit hoofdstuk staan we stil bij een aantal extra moeilijkheden die optreden wanneer we werken in een omgeving die bestaat uit meerdere processoren. We bespreken eerst kort een eenvoudige symmetrische multiprocessor (SMP) organisatie. Daarna gaan we wat dieper in op het *cache coherency* probleem dat hierbij optreedt en bespreken we het MESI protocol dat hiervoor een oplossing biedt. Tenslotte kijken we ook naar de verhoogde complexiteit van schedulingbeslissingen in een multiprocessor omgeving.

1 Symmetrische multiprocessor (SMP) organisatie

Systemen met meerdere processoren kunnen op een aantal manieren worden geclassificeerd. Eén van de belangrijkste eigenschappen die hierbij toedragen, is of het geheugen dat deze processoren gebruiken al dan niet wordt gedeeld. Wanneer de processoren het geheugen niet delen, dan zijn de processen duidelijk minder sterk aan elkaar gekoppeld en spreken we veelal van een *cluster*. De specifieke uitdagingen eigen aan cluster systemen zullen in andere cursussen uit de opleiding aan bod komen. Onze latere discussie rond multiprocessor scheduling algoritmen is voor een deel nog wel toepasbaar op clusters. In dit hoofdstuk zullen we ons meer toeleggen op systemen waarbij het geheugen wel wordt gedeeld. Deze systemen kunnen op hun beurt worden opgesplitst in *master/slave* en *symmetric (SMP)* systemen.

In het geval van een *master/slave* systeem zullen alle kernel functionaliteiten uitgevoerd worden op dezelfde processor. Wenst een proces of thread een service van het operating systeem (zoals een I/O call) dan vraagt hij deze service aan bij de master. Het voordeel is dat alle beslissingen die het operating systeem moet nemen, gecentraliseerd zijn op één processor waardoor er betrekkelijk weinig veranderingen nodig zijn ten opzichte van een uniprocessor systeem. Onze aandacht gaan echter voornamelijk uit naar SMP systemen, waarbij de kernel functionaliteit wordt ondersteund door beide (of alle) processoren.

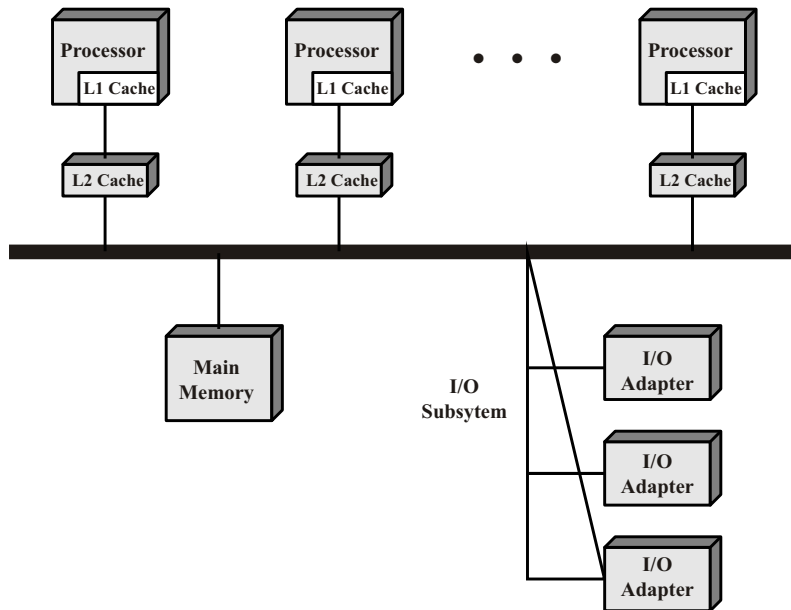


Fig. 64: Symmetrische multiprocessor organisatie [Stallings 03]

Een klassieke opstelling van een SMP systeem zien we in Figuur 64. Hierbij zien we dat de processoren via een bus met elkaar verbonden zijn, alsook met het main geheugen. Wil een bepaald device data over deze bus versturen, dan moet het alleen over deze bus kunnen beschikken. De capaciteit van de bus wordt dus op een dynamische en *time-slotted* manier onder de devices verdeeld. Verder beschikt ook elke processor over zijn eigen cache. In de praktijk zullen er meerdere caches aanwezig zijn, maar voor onze discussie laten we deze voorlopig buiten beschouwing. De opstelling zoals in Figuur 64 is echter niet de enige mogelijkheid. Bijvoorbeeld in een *multi-core* opstelling zullen meerdere processoren op een enkele chip worden geïntegreerd, waarbij elke processor zijn eigen on-chip L1 cache heeft. Deze opstelling heeft als voordeel dat beide processoren erg snel data met elkaar kunnen uitwisselen, wat grote voordelen heeft voor het uitvoeren van een cache coherency protocol (zie verder).

Een derde mogelijke architectuur vinden we terug in het geval van *hyper threading*. In dit geval zal het systeem vanuit het oogpunt van het operating systeem nog steeds bestaan uit meerdere processoren, echter in de realiteit is er slechts een enkele processor beschikbaar. In deze processor zijn de

meeste zaken gedupliceerd, zodat er twee threads (processen) gelijktijdig in de processor aanwezig kunnen zijn. Echter, er is slechts één CPU die de werkelijke berekeningen uitvoert. Het grote voordeel van hyper threading (aanwezig bij de Pentium 4) is dat er geen context switch vereist is om twee threads afwisselend gebruik te laten maken van de CPU. Hierdoor kan de ene thread even de CPU overnemen indien er zich bij de andere thread een oponthoud voordoet (b.v. een cache miss). Bij deze opstelling zal er doorgaans slechts een enkele *L1* cache gebruikt worden.

Een eerste probleem dat zich voordoet bij een multiprocessor architectuur die meerdere *L1* caches hanteert, is dat dezelfde geheugenwoorden in meerdere caches aanwezig kunnen zijn. Wanneer vervolgens een bepaalde processor besluit een write operatie op dit geheugenwoord uit te voeren dan geeft dit snel aanleiding tot inconsistenties. In de volgende sectie bespreken we een specifieke oplossing voor dit probleem, zijnde het MESI protocol. Dit protocol is een hardwarematige oplossing die onder andere gebruikt wordt in commerciële multiprocessor systemen zoals de Pentium 4 en PowerPC.

2 Cache coherency: het MESI protocol

Wanneer elke processor in een SMP setup over een cache beschikt, dan moeten we ervoor zorgen dat de data steeds coherent blijft. Dit betekent dat alle processoren steeds de meest recente versie van een geheugenwoord moeten lezen, het kan namelijk voorvallen dat verschillende threads (of processen), die dezelfde data manipuleren, gelijktijdig in uitvoering zijn.

Het is niet zo moeilijk hiervoor een oplossing te bedenken: zo zou men bij elke write operatie ook het main geheugen kunnen aanpassen (write through). Deze aanpak op zich is niet alleen inefficiënt, maar ook weinig effectief, want zelfs indien een processor een woord in de cache heeft, dan is het nog niet zeker dat dit woord nog niet werd aangepast in het main geheugen door een andere processor. In principe zou men dus een write through kunnen uitvoeren die ook alle andere caches, die dit woord bevatten, aanpast. Deze oplossing is natuurlijk weinig efficiënt, want ze genereert veel trafiek op de bus en kan, in sommige gevallen, de totale performantie van het systeem zelfs zodanig verlagen dat een systeem met een enkele processor effectiever zou werken.

Het MESI protocol zal zorgen voor coherentie, terwijl het toch tracht de hoeveelheid signalisatie over de bus te beperken. Het maakt gebruik

	M Modified	E Exclusive	S Shared	I Invalid
This cache line valid?	Yes	Yes	Yes	No
The memory copy is...	out of date	valid	valid	—
Copies exist in other caches?	No	No	Maybe	Maybe
A write to this line...	does not go to bus	does not go to bus	goes to bus and updates cache	goes directly to bus

Fig. 65: MESI cache lijn toestanden [Stallings 03]

van *snooping*, wat aangeeft dat de cache controllers zelf nagaan, door het versturen en ontvangen van controle signalen over de bus, of een bepaalde cache lijn al dan niet door een andere processor in gebruik is. Een cache systeem dat het MESI protocol hanteert zal voor elke cache lijn twee status bits bijhouden. Met deze twee bits worden dan de vier mogelijke toestanden van de cache lijn weergegeven:

- **Modified:** Deze cache lijn verschilt van de kopij in het main geheugen en geen enkele andere cache bevat deze lijn.
- **Exclusive:** Deze cache lijn verschilt niet van de kopij in het main geheugen en is enkel in deze cache beschikbaar.
- **Shared:** Deze cache lijn is dezelfde als in het main geheugen en mogelijk bevatten ook andere caches deze lijn.
- **Invalid:** Deze cache lijn bevat geen geldige data.

Elke cache lijn bevindt zich dus steeds in één van deze vier toestanden, vandaar dat men dit protocol ook het MESI protocol noemt. Een overzicht van de vier toestanden zien we ook in Figuur 65.

Een diagram dat de verschillende toestandsovergangen weergeeft zien we in Figuur 66. In deze figuur zijn de overgangen veroorzaakt door een lees- of schrijfoperatie op de cache lijn door de processor zelf, links aangegeven en deze veroorzaakt door een signaal ontvangen over de bus rechts. Laten we elke mogelijke operatie even in detail bekijken.

2.1 Read miss en hit

Indien een *read miss* in een cache optreedt, dan betekent dit dat deze processor het gevraagde geheugenwoord niet in zijn cache heeft staan. Noem

deze cache de C_r cache. Het geheugenblok dat dit woord bevat zal dus opgevraagd worden uit het main geheugen. Eerst zal de cache controller van de C_r cache echter een signaal sturen naar de andere processoren (via de bus) om na te gaan of er nog andere caches zijn die deze lijn reeds bevatten. We onderscheiden twee mogelijke reacties:

- Indien er één of meer andere caches zijn die deze lijn bevatten en de toestand van de cache lijn in elk van deze caches is *shared* of *exclusive*, dan zal elk van deze caches een signaal terug sturen om aan te geven dat deze lijn in gebruik is en, indien nodig, de toestand wijzigen naar *shared*. Vervolgens leest de C_r cache de lijn uit het geheugen en stelt de toestand van de ingelezen lijn gelijk aan *shared*.
- Indien er slechts één cache is die deze lijn bevat en de toestand is *modified*, dan zal deze cache eerst het main geheugen aanpassen. Vervolgens stuurt deze cache de aangepaste lijn naar de C_r cache (of hij geeft aan deze uit het main geheugen te lezen) en beide cache lijnen nemen de *shared* toestand aan.

Indien er geen reactie komt, betekent dit dat geen enkele andere cache de lijn in gebruik heeft, waardoor de C_r cache *exclusive* als toestand toekent aan de opgevraagde cache lijn.

Wanneer er een *read hit* optreedt, dan wordt er geen signaal over de bus verstuurd en blijft de toestand van de opgevraagde lijn ook identiek. Read hits zijn dus uiterst snel, wat belangrijk is aangezien het de bedoeling is dat deze situatie zich het vaakst voordoet.

2.2 Write miss en hit

Indien er een *write miss* optreedt, dan zal de processor van deze cache, die we aanduiden als de C_w cache, een signaal versturen over de bus om aan te geven dat hij deze cache lijn wenst te lezen *met de bedoeling ze aan te passen*.

Indien er een cache is die deze lijn bevat en die tevens in de *modified* toestand is (wat betekent dat hij de enige met deze lijn is), dan zal deze eerst de aangepaste versie van deze lijn in het main geheugen plaatsen (dit is nodig daar de aanpassing een INC kan zijn) en vervolgens overgaan naar de *invalid* toestand. Daarna stuurt hij eventueel de aangepaste lijn naar de C_w cache, of laat hij de processor die over de C_w cache beschikt, hetzelfde signaal nogmaals uitzenden over de bus.

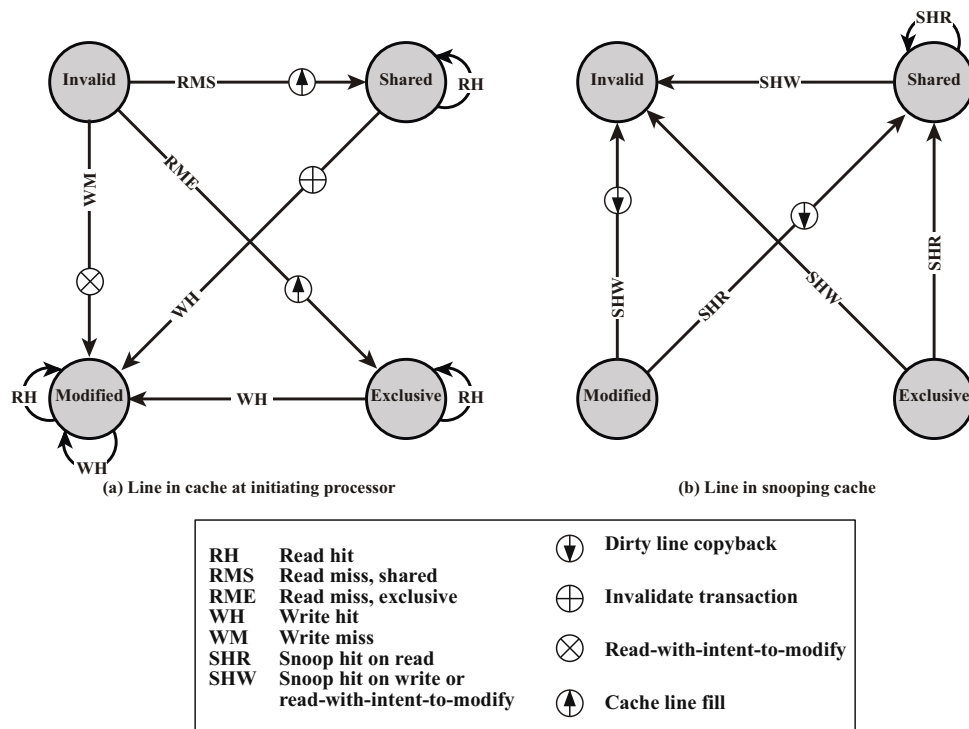


Fig. 66: MESI toestand transitie [Stallings 03]

Zijn er andere caches die de lijn bevatten in de shared of exclusive toestand, dan zullen deze overgaan naar de invalid toestand, daar de C_w cache de lijn zal aanpassen, waardoor de overige versies van deze lijn niet meer van toepassing zijn. In dit geval wordt er dus geen signaal teruggestuurd naar de C_w cache.

Tenslotte bekijken we twee mogelijkheden in het geval van een *write hit*:

- **Modified/Exclusive:** Indien de cache lijn in één van deze toestanden is, dan kan men de cache lijn gewoon aanpassen zonder dat er een signaal over de bus verstuurd hoeft te worden.
- **Shared:** De cache controller van de C_w cache moet eerst zien dat alle andere caches die deze lijn bevatten, hun toestand op invalid zetten. De C_w cache realiseert dit door een signaal over de bus te versturen om aan te geven dat hij de lijn wenst aan te passen. Daarna kan hij de lijn aanpassen en de bijhorende toestand op modified plaatsen.

2.3 L1-L2 cache consistency

Tot nu toe gingen we ervan uit dat er slechts één enkele cache was voor elke processor en dat de controllers hiervan allemaal rechtstreeks via dezelfde bus verbonden waren. In de praktijk zullen het vaak de L2 caches zijn die met de bus verbonden zijn. De L1 caches, die kleiner zijn en een deelverzameling van alle lijnen uit de L2 cache bevatten, hanteren dan een *write through* methode om de toestand van de L2 cache lijnen correct weer te geven. Dit betekent dat elke aanpassing van een lijn in de L1 cache van een processor eveneens onmiddellijk wordt doorgevoerd in de bijhorende L2 cache.

3 Multiprocessor scheduling

Wanneer we spreken over multiprocessor scheduling wordt er doorgaans een onderscheid gemaakt tussen systemen waarin de te schedulen taken volledig los staan van elkaar, waardoor we ze in een willekeurige volgorde over de verschillende processoren kunnen schedulen, en systemen waarbij de taken sterk verbonden zijn doordat ze onderling vaak met elkaar moeten synchroniseren of omdat ze bepaalde resources moeten delen, waardoor ze niet gelijktijdig in de verschillende processoren kunnen worden uitgevoerd. Dit laatste is voornamelijk het geval in de context van thread scheduling.

We beperken ons tot het eerste type van systemen waarbij de taken volledig los staan van elkaar. Verder nemen we ook aan dat er m identieke processoren zijn. We zullen nagaan in hoeverre de resultaten uit het voorgaande hoofdstuk geldig blijven wanneer we over meerdere processoren beschikken. Onze nadruk ligt opnieuw op het short-term schedulen.

3.1 De maximale completion time

Wanneer we onze blik verruimen van uniprocessor systemen naar multiprocessor systemen, dan stelt er zich een nieuwe uitdaging op vlak van short-term scheduling. Namelijk, omdat een taak nooit op twee machines tegelijkertijd kan lopen, valt het vaak voor dat de maximale completion time $C_{max} = \max_{j=1}^n C_j$ van een set van n taken afhankelijk is van de volgorde waarin de taken worden afgehandeld. Bijvoorbeeld, neem drie taken met processing times $p_1 = p_2 = 1$ en $p_3 = 2$. Indien, we twee processoren hebben en we laten deze starten met taak 1 en 2 respectievelijk, dan zal $C_3 = 3$.

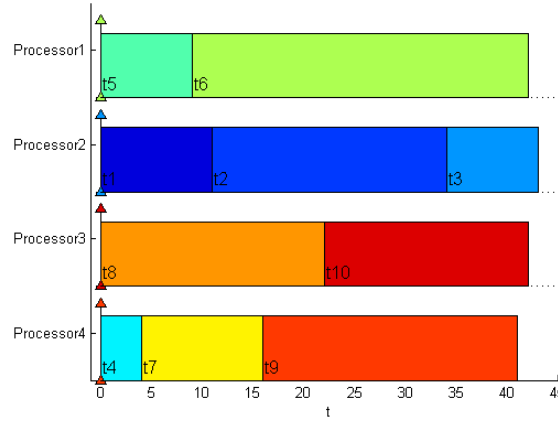


Fig. 67: Optimale oplossing voor $n = 10$ taken en $m = 4$ machines met uitvoertijden: 11, 23, 9, 4, 9, 33, 12, 22, 25 en 20, met $C_{max} = 43$

Laten we echter taak 1 en 3 starten dan zal $C_1 = 1$, $C_2 = 2$ en $C_3 = 2$. Wanneer we maar over een enkele processor beschikken dan is C_{max} natuurlijk steeds gelijk aan 4 (tenzij we de processor een tijd onbenut laten). In Figuur 67 zien we een optimale oplossing voor een probleem bestaande uit 10 taken die te verdelen zijn over 4 processoren wanneer we geen preëmtie toe laten. We starten met enkele benaderingsalgoritmen voor het volgende model

$$Pm || C_{max},$$

dat, net zoals vele andere problemen in multiprocessor scheduling, NP-hard is. Er bestaat wel een (niet praktische) oplossing met polynomiale rekentijd indien we maar een eindig aantal verschillende uitvoertijden toestaan voor de taken. Men kan zelfs een polynomiaal algoritme maken dat maximaal $(1 + \epsilon)$ maal slechter is dan het optimale voor alle $\epsilon > 0$. Zoals we verder zullen zien in de cursus, is het probleem wel eenvoudig oplosbaar indien we preëmtie toelaten.

List Scheduling Algorithm (LS): Het LS algoritme is uiterst eenvoudig en kent bij het vrijkomen van een processor een willekeurige taak aan deze processor toe (die nog niet is uitgevoerd). Het algoritme start eveneens door m taken willekeurig te kiezen. In Figuur 68 zien we het resultaat van dit algoritme toegepast op het probleem uit Figuur 67.

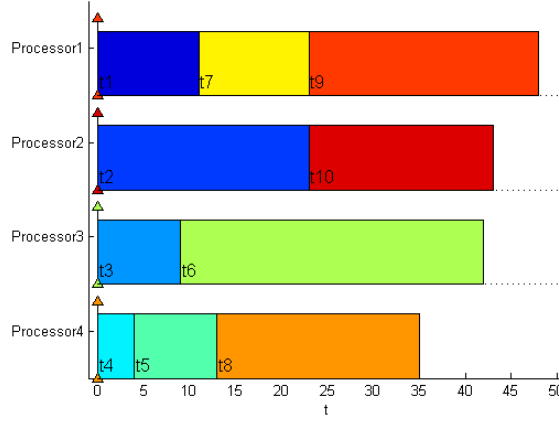


Fig. 68: List scheduling oplossing voor $n = 10$ taken en $m = 4$ machines met uitvoertijden: 11, 23, 9, 4, 9, 33, 12, 22, 25 en 20, met $C_{max} = 48$

Dit algoritme zal maximaal $2 - 1/m$ keer slechter presteren dan het optimale algoritme. Laat C_{max}^* de kleinste mogelijke maximale completion time zijn dat een algoritme kan realiseren voor een set van n taken. Om de performance van het LS algoritme aan te tonen starten we met twee ondergrenzen op C_{max}^* :

$$C_{max}^* \geq p_j,$$

voor alle j en

$$C_{max}^* \geq \sum_{j=1}^n p_j / m,$$

aangezien al het werk door m processoren verricht moet worden. Stel nu dat taak k de laatste is die door het LS algoritme wordt afgewerkt. Dan startte de service van dit proces op tijd $s_k = C_k - p_k$. Alle processoren moeten dus tot tijd s_k zeker bezig geweest zijn met (een deel van) de overige taken. Kortom,

$$C_{max} = C_k = s_k + p_k \leq \sum_{j \neq k} p_j / m + p_k \leq \sum_{j=1}^n p_j / m + \frac{m-1}{m} p_k.$$

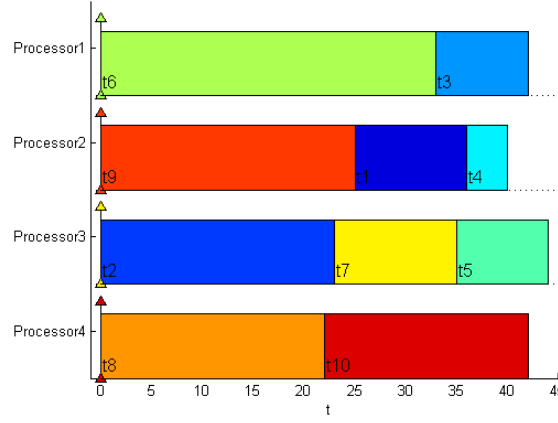


Fig. 69: LPT scheduling oplossing voor $n = 10$ taken en $m = 4$ machines met uitvoertijden: 11, 23, 9, 4, 9, 33, 12, 22, 25 en 20, met $C_{max} = 44$

Gebruikmakend van de twee grenzen voor C_{max}^* verkrijgen we dan

$$C_{max} \leq C_{max}^* + \frac{m-1}{m} C_{max}^* = \frac{2m-1}{m} C_{max}^*.$$

Voor m gelijk aan twee is het LS algoritme dus maximaal 1.5 keer zo slecht. Naarmate we meer processoren nemen kan de performantie tot 2 keer slechter worden, daar $1/m$ naar nul daalt. Dat deze bovengrens voor het LS algoritme niet meer valt te verbeteren zullen we aantonen tijdens de oefeningen.

Longest Processing Time First (LPT): Wanneer we de voorgaande evaluatie in gedachte nemen, dan lijkt het nuttig om ervoor te zorgen dat een zo kort mogelijke taak als laatste wordt afgewerkt. Hoewel dit niet eenvoudig is, kunnen we er wel voor zorgen dat de kortste taak als laatste gestart wordt. Dit is bijvoorbeeld het geval wanneer we de taken van groot naar klein schedulen en is ook precies wat het LPT algoritme doet. Het resultaat van dit algoritme toegepast op het voorbeeld uit Figuur 67 zien we in Figuur 69

We zullen door middel van een vrij eenvoudig argument aantonen dat dit algoritme een $4/3 - 1/3m$ -approximatie is. Voor $m = 2$ is het dus maximaal $7/6$ keer zo slecht. Tijdens de oefeningen zullen we aantonen dat deze bovengrens voor het LPT algoritme niet verder verbeterd kan worden. Het bewijs

start analoog aan dat van het LS algoritme: stel dat taak k de laatste is die door het LPT algoritme wordt afgewerkt, waarbij we de taken nummeren van groot naar klein. Analooog als bij het LS algoritme hebben we

$$C_{max} \leq C_{max}^* + \frac{m-1}{m}p_k.$$

Kortom, als $p_k \leq C_{max}^*/3$ dan zijn we klaar. Anders moeten de taken 1 tot k een lengte groter dan $C_{max}^*/3$ hebben (want de taken worden van groot naar klein uitgevoerd). Een optimale oplossing zal hoogstens twee van deze taken kunnen toewijzen per processor (want 3 van deze taken samen hebben een lengte groter dan C_{max}^*), wat ook impliceert dat $k \leq 2m$. Laten we alle taken die na taak k starten even buiten beschouwing, dan zal de beste manier om deze k taken te schedulen erin bestaan object i , voor $i \leq m$, samen met object $2m+1-i$ te plaatsen (tenzij $2m+1-i > k$, dan stoppen we object i alleen), dit is exact wat het LPT algoritme doet. Met andere woorden, indien $p_k > C_{max}^*/3$ dan is het LPT algoritme optimaal en is er aan de vooropgestelde bovengrens zeker voldaan.

McNaughton's wrap-around rule: Wanneer we wel preëmtie van de taken toelaten dan is het voorgaande probleem eenvoudig oplosbaar. We stellen eerst $D = \max\{\sum_j p_j/m, \max_j p_j\}$. Vervolgens ordenen we de taken in willekeurige volgorde en starten door eerst het schema van machine 1 op te vullen (tot tijd D), dan machine 2, enz. Wanneer een taak j met lengte p_j de laatste t tijdseenheden van een machine k krijgt, met $t < p_j$, dan zal deze taak eveneens de eerste $p_j - t$ tijdseenheden van machine $k+1$ krijgen. Taak j wordt dus eerst $p_j - t$ eenheden op machine $k+1$ uitgevoerd, vervolgens onderbroken en van tijd $D - t$ tot tijd D op machine k . Gezien $D \geq \max_j p_j$, zal een taak nooit op twee machines tegelijkertijd lopen. Verder zijn ook alle taken afgehandeld voordat de m machines allemaal bezet zijn, want $Dm \geq \sum_{j=1}^n p_j$. Omwille van de twee eerder genoemde ondergrenzen voor C_{max} zien we dat deze methode volgend systeem optimaal oplost:

$$1/p_m t n |C_{max}.$$

Links in Figuur 70 zien we deze regel toegepast op het voorbeeld uit Figuur 67, rechts zien we het resultaat indien taak 6 een uitvoertijd van 53 zou hebben in plaats van 33, waardoor $\sum_j p_j/m < \max_j p_j$.

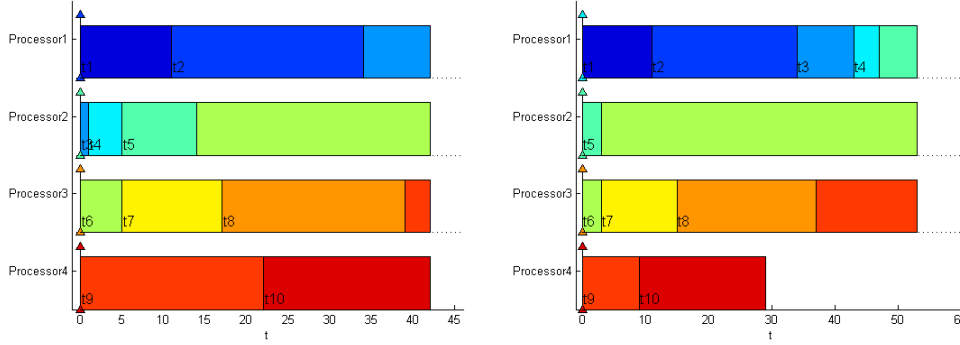


Fig. 70: McNaughton's wrap-around rule voor $n = 10$ taken en $m = 4$ machines met uitvoertijden: 11, 23, 9, 4, 9, 33 of 53, 12, 22, 25 en 20

3.2 SPN en EDF scheduling

Voor een machine met een enkele processor hebben we aangetoond dat Shortest Process Next (SPN) en Earliest Deadline First (EDF) optimaal zijn voor het $1||\sum C_j$ en $1||L_{max}$ model. In deze sectie gaan we even na in hoeverre deze optimaliteit zich laat veralgemenen naar meerdere processoren.

Shortest Process Next: We starten met het SPN algoritme. Stel dat er l_i taken worden uitgevoerd op machine i , voor $i = 1, \dots, m$. Laat κ_{ki} de k -de laatste taak zijn die door machine i wordt uitgevoerd. Dan zien we dat

$$\sum_{j=1}^n C_j = \sum_{i=1}^m \sum_{k=1}^{l_i} C_{\kappa_{ki}} = \sum_{i=1}^m \sum_{k=1}^{l_i} \sum_{x=k}^{l_i} p_{\kappa_{xi}} = \sum_{i=1}^m \sum_{k=1}^{l_i} k p_{\kappa_{ki}}.$$

Kortom, net zoals bij een enkele processor telt de processing time van de k -de laatste taak (op elke machine) k keer mee. De m grootste taken moeten dus als laatste op de m machines lopen, vervolgens de volgende m grootste als voorlaatste, enz. Dit is precies de volgorde van SPN, wat betekent dat SPN scheduling eveneens optimaal is voor

$$Pm||\sum C_j.$$

Dit resultaat geeft aan dat we, afhankelijk van de functie die we wensen te minimaliseren, soms erg verschillend te werk moeten gaan. Voor C_{max} is de

langste taak eerst een goede methode, maar voor de wachttijd moeten we de kortste taak eerst nemen.

We moeten ook enkele kanttekeningen plaatsen bij dit resultaat. Ten eerste zullen we in de praktijk de processing times niet altijd kennen (tenzij voor real-time systemen of in een productieomgeving). Verder is het zo dat de winst die SPN (of SRTN indien we ook aankomsttijden hanteren) oplevert ten opzichte van een eenvoudige FCFS sterk afneemt van zodra er meer processoren in het spel zijn. Zo wordt een winst van 30% al snel gereduceerd tot 5% door een tweede processor in gebruik te nemen. Hetzelfde gaat ook op voor de vergelijking tussen FCFS en Round Robin (RR). Dit is erg belangrijk en geeft ook aan waarom er in vele systemen met meerdere processoren enkel gewerkt wordt met erg eenvoudige scheduling algoritmen. Intuïtief is dit resultaat ook te verwachten, daar een enkele grote taak alle andere taken een verhoogde wachttijd geeft in geval van een enkele processor. Zijn er meerdere processoren dan kunnen de korte taken toch nog snel worden afgehandeld door een andere processor.

Earliest Deadline First: In tegenstelling tot wat men misschien zou verwachten is EDF niet meer optimaal voor het minimaliseren van L_{max} van zodra er meerdere processoren gebruikt worden. Laten we dit even tonen met behulp van een voorbeeld. Stel we hebben drie taken met $p_1 = 1$, $p_2 = 2$ en $p_3 = 3$. Stel verder dat de deadlines gelijk zijn aan $d_1 = 2$, $d_2 = 3$ en $d_3 = 3.5$. EDF zou starten met het uitvoeren van taak 1 en 2 op de twee processoren, waardoor beide taken hun deadline halen. Pas op tijd één zal taak drie aan de beurt komen en hierdoor zijn deadline met 0.5 overschrijden. Hadden we echter gestart met taak 1 en 3 dan hadden alle taken hun deadline gehaald. Voor het $Pm||L_{max}$ model is het nog wel mogelijk om een optimale oplossing te vinden in polynomiale tijd.